Advanced Topics

#	Scenario (Short)	Level	Target Roles
1	Tuning GC for bursty, low-latency e-commerce checkout	L4	Performance Engineer, Backend Core
2	Memory visibility & atomicity in wallet service	L3	Concurrency Engineer
3	JVM on constrained edge devices (smart meters)	L4	IoT Engineer
4	Runtime-pluggable logging via Dynamic Proxy	L3	Backend Core
5	Custom class loading with cyclic plugins	L4	Platform/JVM Engineer
6	JPMS for service-oriented payments codebase	L3	Architect, Backend Core
7	Replace verbose DTOs with Records	L2	Backend Core
8	Cleaner type-specific logic using instanceof pattern	L2	Java Developer
9	Switch pattern matching for heterogeneous inputs	L3	Backend Core
10	Accelerate numeric kernels with Vector API	L4	Performance Engineer
11	Reduce object churn in stock trading engine	L4	Backend Core
12	Modularize university system with JPMS	L3	Architect
13	Avoid deadlocks in hotel booking system	L3	Concurrency Engineer
14	Restrict inheritance in transit app	L2	Java Developer
15	Capture method execution time	L3	Performance Engineer
16	Cleaner query handling with switch patterns	L2	Backend Core
17	Accelerate climate simulation	L4	HPC Engineer
18	Secure deserialization in payment service	L3	Security Engineer
19	Simplify settings with Records	L2	Backend Core
20	Thread-safe random numbers in gaming app	L2	Backend Core
21	GC tuning for sports analytics	L4	Performance Engineer

22	Simplify heterogeneous payloads	L3	Backend Core
23	Thread-safe account updates in trading	L3	Backend Core
24	Break dependency cycles in modules	L3	Architect
25	Vector API in recommendation engine	L4	Data Engineer
26	Prevent insecure deserialization in loan app	L3	Security Engineer
27	Immutable feature flags via Records	L2	Backend Core
28	Scalable random generation (multiplayer)	L3	Game Engineer
29	Enforce API boundaries in healthcare	L3	Architect
30	Replace DTOs in logistics platform	L2	Backend Core
31	Accelerate graph algorithms with SIMD	L4	Data Engineer
32	Eliminate circular deps in enterprise suite	L3	Architect
33	Runtime behavior in plugin CMS	L3	Platform Engineer
34	Validate financial record deserialization	L3	Security Engineer
35	Reduce heap fragmentation in messaging	L4	Performance Engineer
36	Thread safety in order pipeline	L3	Backend Core
37	Faster startup for microservices	L4	Performance Engineer
38	Secure service-to-service JWT calls	L3	Security Engineer
39	Sealed classes for payroll roles	L2	Java Developer
40	Validate nested JSON payloads	L3	API Engineer
41	FFM API native library calls	L4	Systems Engineer
42	Off-heap struct management (Arenas)	L4	Systems Engineer
43	Primitive classes for telemetry (JDK 23)	L4	JVM Engineer
44	Compare Records vs Primitive Classes	L4	Performance Engineer
45	Module import declarations (JDK 23)	L4	Architect
46	Escape analysis for JVM tuning	L4	JVM Engineer
47	Diagnose latency spikes from safepoints	L4	Performance Engineer

48	Advanced module usage (services + reflection)	L4	Architect
49	ClassLoader conflicts / split-packages	L4	Platform Engineer
50	Pattern matching for switch (guards)	L3	Java Developer
51	Concurrent cache for burst traffic	L3	Backend Core
52	Lightweight plugins without restarts	L3	Platform Engineer
53	Zero-copy I/O for file transfer	L4	Systems Engineer
54	JIT warm-up profiling for latency reduction	L4	Performance Engineer
55	Class redefinition at runtime (agents)	L4	JVM Engineer
56	Integrating Vector API with Streams	L4	Data Engineer
57	Analyze object allocation patterns	L4	Performance Engineer
58	Reactive stream backpressure tuning	L3	Backend Core
59	Monitor GC throughput vs pause trade-off	L4	Performance Engineer
60	Pinning memory regions in FFM	L4	Systems Engineer
61	Hot-swap modules in a running JVM	L4	Platform Engineer
62	Memory-safe native interop checks	L4	Systems Engineer
63	Parallel GC vs G1 benchmarking	L4	Performance Engineer
64	Predictable latency with Epsilon GC	L4	Performance Engineer
65	Design immutable dataflow pipeline	L3	Backend Core
66	Secure module reflection access	L4	Security Engineer
67	Automate GC tuning via JMX	L4	Platform Engineer
68	Streaming large objects off-heap	L4	Systems Engineer
69	Heap dump analysis automation	L4	Performance Engineer
70	Diagnose thread contention live	L3	Concurrency Engineer
71	Multi-version modules for compatibility	L4	Architect
72	Hybrid AOT + JIT startup optimization	L4	Performance Engineer

73	Implement virtual threads in microservices	L4	Backend Core
74	Isolate tenant state in multi-tenant JVM	L4	Platform Engineer
75	Secure FFM access with capsules	L4	Systems Engineer
76	Auto-detect safepoint pauses	L4	Performance Engineer
77	Generate runtime proxies for audit logging	L3	Backend Core
78	Monitor native memory usage	L4	Performance Engineer
79	Async GC tuning for throughput	L4	Performance Engineer
80	JVM sandbox security for plugins	L4	Security Engineer
81	Runtime dependency graph inspection	L4	Architect
82	Container-aware JVM resource limits	L3	DevOps Engineer
83	Profile object retention across GCs	L4	Performance Engineer
84	Memory barriers & atomic ops testing	L4	Concurrency Engineer
85	Measure vectorization speed-ups	L4	Data Engineer
86	Safely handle foreign function exceptions	L4	Systems Engineer
87	Custom runtime compiler flags experiment	L4	JVM Engineer
88	Link-time optimizations in JPMS	L4	Architect
89	Module-based testing framework setup	L3	Backend Core
90	Pattern matching switch with guards (JDK 21 final)	L3	Java Developer

Scenario 1: Tuning GC for a bursty, low-latency e-commerce checkout

A flash-sale creates heap surges: short-lived cart/session objects and hot caches. We need to keep GC pauses predictable while sustaining high throughput.

Answer:

I'd choose a concurrent, low-pause collector and size the heap with enough headroom for bursts.

- **Collector choice**: ZGC (preferred on Java 17+) for sub-millisecond pauses; Shenandoah as alternative; G1 if stuck on older setups.
- **Heap sizing**: Measure live set (say 10 GB) and provision 1.5–2× that to absorb bursts. Keep -Xms = -Xmx to avoid resize jitter.
- **Pause target**: ZGC aims <2 ms; Shenandoah ~10 ms; G1 tuned with MaxGCPauseMillis=20.
- **Startup vs steady-state**: Warm caches and pre-touch memory at startup. In steady state, rely on concurrent cycles with enough free headroom.
- **Diagnostics**: Unified GC logging (-Xlog:gc*), JFR allocation profiling, safepoint stats, and G1 summaries (if using G1).

Code:

```
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class CheckoutBurstHarness {
    public static void main(String[] args) throws Exception {
        int threads = Runtime.getRuntime().availableProcessors();
        ExecutorService pool = Executors.newFixedThreadPool(threads);

        Callable<Long> cartWork = () -> {
            long start = System.nanoTime();
            List<byte[]> junk = new ArrayList<>(1024);
            for (int i = 0; i < 1024; i++) {
                junk.add(new byte[1024]); // simulate cart churn
            }
            byte[] cached = new byte[256 * 1024]; // survive longer
            return Duration.ofNanos(System.nanoTime() - start).toMillis();
        };
```

```
        for (int round = 0; round < 60; round++) {
            List<Future<Long>> fs = new ArrayList<>();
            for (int i = 0; i < threads * 200; i++) {
                fs.add(pool.submit(cartWork));
            }
            long p99 = fs.stream().mapToLong(f -> {
                try { return f.get(); } catch (Exception e) { return 0; }
            }).sorted().skip((long)(fs.size() * 0.99)).findFirst().orElse(0);
            System.out.println("Round " + round + " p99(ms)=" + p99);
            Thread.sleep(500);
        }
        pool.shutdown();
    }
}
```

Reasoning to interviewer:

- ZGC/Shenandoah avoid stop-the-world compaction, so flash-sale spikes don't blow up pause times.
- Fixed Xms=Xmx removes resize jitter; pre-touch avoids first-touch page faults.
- G1 tuning with larger young gen absorbs cart churn; earlier IHOP triggers prevent late full GCs.
- Diagnostics link latency regressions to GC phases and allocation hot spots, so tuning is data-driven.

Future scope:

- Try generational ZGC to further isolate short-lived churn.
- Add allocation budgets in CI to catch regressions.
- Offload some state to off-heap stores if heap churn stays high.
- Explore TLAB tuning and critical thread pinning under extreme contention.

Scenario 2: Memory visibility and atomicity in a wallet service

A wallet service has concurrent balance updates and fraud checks. Readers must see the latest confirmed balance, and updates must not interleave incorrectly.

Answer:

I'd split the problem into visibility (publishing latest state) and atomicity (safe updates).

- **Visibility**: Use a volatile field (or AtomicReference) for the last confirmed balance. Volatile ensures happens-before for readers.
- **Atomic debit/credit**: Use AtomicLong with CAS for simple add/subtract, or a lock for multi-step rules. Both prevent torn updates.
- **Pattern**: Writers update AtomicLong and publish to volatile; readers just read the volatile snapshot.

Code:

```
import java.util.concurrent.atomic.AtomicLong;

final class Account {
    private final AtomicLong cents = new AtomicLong(0);
    private volatile long lastConfirmedCents = 0;

    Account(long initial) {
        cents.set(initial);
        lastConfirmedCents = initial;
    }

    void credit(long delta) {
        long updated = cents.addAndGet(delta);
        lastConfirmedCents = updated; // volatile publish
    }

    boolean debit(long delta) {
        while (true) {
            long cur = cents.get();
            long next = cur - delta;
            if (next < 0) return false;
            if (cents.compareAndSet(cur, next)) {
                lastConfirmedCents = next;
                return true;
            }
        }
    }
```

```
    long readForFraud() {
        return lastConfirmedCents; // volatile read
    }
}
```

Reasoning to interviewer:

- The Java Memory Model guarantees volatile writes are visible to later reads, so fraud checks see fresh values.
- CAS via AtomicLong ensures no lost updates under concurrency.
- For business rules needing multiple steps (fees, holds), use ReentrantLock to keep the sequence atomic.
- Keeping account identity stable in a Map avoids publication bugs; only the internal balance field changes.

Future scope:

- For multi-account transfers, use ordered locking or a transaction layer to avoid deadlocks.
- For read-heavy fraud checks, consider ReadWriteLock but keep write sections short.
- Use LongAdder only for metrics, not balances.
- Add an audit log of updates so readers can subscribe to events, not poll.
- Expose a version counter alongside balance for readers to detect changes efficiently.

Scenario 3: JVM on constrained edge devices (smart meters)

Smart meters have tiny RAM and often a single CPU core. Our Java app batches sensor readings and occasionally does TLS handshakes to a gateway. Under brief spikes, the device must stay stable; the total footprint has to stay small.

Answer:

I'd optimize for three things: deterministic pauses on 1 core, a small steady footprint, and predictable startup.

- **GC choice**: use Serial GC. On single-core hardware, "parallel" collectors just contend with the app. Serial GC's stop-the-world pauses are short when the heap is small and allocation rate is modest.
- **Heap policy**: fix -Xms and -Xmx to small, equal values (for example, -Xms32m -Xmx128m) so we don't pay for dynamic heap resizing. If some meters are tighter, I use percentages: -XX:InitialRAMPercentage=10 -XX:MaxRAMPercentage=35.
- **Metaspace/classes**: shrink class footprint with CDS/AppCDS and a slim runtime via jlink. That removes modules we don't use and shares class metadata pages across processes (where applicable).
- **JIT strategy**: prefer a "light JIT" plan: keep Tiered C1 only (-XX:TieredStopAtLevel=1, -XX:CICompilerCount=1). Full C2 isn't worth the compile cost on a meter. Avoid -Xint unless absolutely necessary—it's often too slow for TLS and batching.
- **Startup footprint**: enable CDS (and AppCDS if we can build an archive from a training run). Preload only the classes we actually use; avoid -XX:+AlwaysPreTouch on these devices (it commits memory upfront).
- **Features to disable/trim**: turn off things that add background work or memory maps: -XX:-UsePerfData, -XX:+DisableAttachMechanism, avoid JFR/JVMTI, keep GC logging minimal, and set -Djava.security.properties to a lean config if we ship our own providers.
- **TLS behavior**: at the app layer, reuse connections and enable session resumption so we don't trigger expensive handshakes often. Keep direct buffers bounded if using NIO.

Code:

A tiny bootstrap that fits the footprint goals (Serial GC, light JIT, capped heap). In practice these flags go in your service unit or container manifest:

```
java \
  -XX:+UseSerialGC \
  -Xms64m -Xmx64m \
  -XX:TieredStopAtLevel=1 -XX:CICompilerCount=1 \
  -XX:-UsePerfData -XX:+DisableAttachMechanism \
  -XX:MaxMetaspaceSize=64m \
  -Xshare:on \
  -cp app.jar com.edge.SmartMeterMain
```

A sketch of the batching loop: the design avoids allocation spikes by reusing buffers and using a bounded queue.

```
import java.nio.ByteBuffer;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.TimeUnit;

class BatchSender implements Runnable {
    private final ArrayBlockingQueue<byte[]> sensorQ = new
ArrayBlockingQueue<>(256);
    private final ByteBuffer reuse = ByteBuffer.allocate(8 * 1024); // small reusable
buffer

    public void ingest(byte[] reading) {
        // drop oldest if full—backpressure at the edge beats OOM
        if (!sensorQ.offer(reading)) sensorQ.poll();
        sensorQ.offer(reading);
    }

    @Override public void run() {
        long lastFlush = System.nanoTime();
        while (!Thread.currentThread().isInterrupted()) {
            try {
                byte[] r = sensorQ.poll(200, TimeUnit.MILLISECONDS);
                if (r != null && reuse.remaining() >= r.length + 4) {
                    reuse.putInt(r.length).put(r);
                }
                boolean time = System.nanoTime() - lastFlush >
TimeUnit.SECONDS.toNanos(2);
                boolean full = reuse.remaining() < 512;
                if (r == null && reuse.position() > 0 || time || full) {
                    flush(reuse); // sends via pooled TLS connection with session
resumption
                    reuse.clear();
```

```
                lastFlush = System.nanoTime();
            }
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
        }
    }
}

private void flush(ByteBuffer buf) {
    // minimal allocation; reuse channel and buffers
}
}
```

Reasoning to interviewer:

- Serial GC wins on single-core: there's no GC worker pool stealing cycles from the app, and with a tiny heap, pauses are short and predictable.
- Equal Xms/Xmx avoids "elastic heap" surprises that can thrash paging on small devices.
- CDS/AppCDS and jlink directly reduce resident set size: fewer modules loaded, smaller metaspace, less I/O at boot.
- C1-only JIT gives you "just enough" optimization without multi-threaded compilation overhead.
- Bounded queues and buffer reuse cap allocation rates, so GC has very little to do during spikes.

Future scope:

- If devices vary a lot, maintain two profiles: "tiny" (32–64 MB heap, C1) and "standard" (64–128 MB, C1) with the same code.
- If startup latency is critical, consider a native image build for the data collector component and keep the TLS/gateway client on HotSpot.
- Introduce a circuit-breaker: on repeated TLS failures, back off and persist locally to avoid repeated allocation/IO churn.
- Track a simple health metric (free heap watermark); if it dips below a threshold, temporarily reduce batch size to stabilize.

- ## Scenario 4: Runtime-pluggable logging behaviors via dynamic proxies

  We need to add behaviors at runtime—prepend request-ID, drop DEBUG in prod, buffer and flush on error—without changing the logger classes.

Answer:

I'd define a small Logger SPI (interface), keep a simple concrete logger, and wrap it with a dynamic proxy. The proxy's InvocationHandler becomes the interception point where I can apply a chain of behaviors based on a live config.

- The proxy inspects the method (debug/info/error) and arguments, then runs a list of behaviors before delegating.
- Behaviors are small strategy objects (e.g., "drop DEBUG", "prepend correlation-ID", "buffer until ERROR").
- Toggling is just swapping the behavior list at runtime—no recompilation.

Code:

A minimal, extensible setup.

```
import java.lang.reflect.*;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentLinkedQueue;

interface Logger {
    void debug(String msg);
    void info(String msg);
    void error(String msg, Throwable t);
}

// base implementation (no cross-cutting concerns here)
class SimpleLogger implements Logger {
    private final String name;
    SimpleLogger(String name) { this.name = name; }
    public void debug(String msg) { System.out.println(ts() + " DEBUG [" + name + "]
" + msg); }
    public void info(String msg)  { System.out.println(ts() + " INFO  [" + name + "] " +
msg); }
    public void error(String msg, Throwable t) {
        System.err.println(ts() + " ERROR [" + name + "] " + msg);
        if (t != null) t.printStackTrace(System.err);
    }
```

```
    private String ts() { return Instant.now().toString(); }
}
```

// behavior SPI

```
interface LogBehavior {
    // return null to drop the call; or return possibly-modified args
    Object[] beforeInvoke(Method m, Object[] args);
    default void afterInvoke(Method m, Object result, Throwable thrown) {}
}

// behavior: drop DEBUG in prod
class DropDebugBehavior implements LogBehavior {
    public Object[] beforeInvoke(Method m, Object[] args) {
        if (m.getName().equals("debug")) return null; // drop
        return args;
    }
}
```

// behavior: prepend request ID if present

```
class PrependRequestIdBehavior implements LogBehavior {
    private final ThreadLocal<String> requestIdTL;
    PrependRequestIdBehavior(ThreadLocal<String> requestIdTL) {
this.requestIdTL = requestIdTL; }

    public Object[] beforeInvoke(Method m, Object[] args) {
        String rid = requestIdTL.get();
        if (rid == null || args == null || args.length == 0 || !(args[0] instanceof String))
return args;
        Object[] copy = Arrays.copyOf(args, args.length);
        copy[0] = "[rid=" + rid + "] " + copy[0];
        return copy;
    }
}
```

// behavior: buffer logs; flush buffer when an error happens

```
class BufferUntilErrorBehavior implements LogBehavior {
    private final Queue<String> buffer = new ConcurrentLinkedQueue<>();
    public Object[] beforeInvoke(Method m, Object[] args) {
        if (m.getName().equals("error")) {
            // flush buffer first
            while (!buffer.isEmpty()) {
                System.out.println(buffer.poll());
            }
            return args;
        }
        if (args != null && args.length > 0 && args[0] instanceof String) {
            buffer.offer(args[0].toString());
```

```
        }
        return args;
    }
}
```

// InvocationHandler that applies behaviors, then delegates

```
class LoggerProxy implements InvocationHandler {
    private final Logger target;
    private volatile List<LogBehavior> behaviors;

    LoggerProxy(Logger target, List<LogBehavior> behaviors) {
        this.target = target;
        this.behaviors = List.copyOf(behaviors);
    }

    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        Object[] current = args;
        for (LogBehavior b : behaviors) {
            current = b.beforeInvoke(method, current);
            if (current == null) return null; // dropped
        }
        Object result = null;
        Throwable thrown = null;
        try {
            result = method.invoke(target, current);
            return result;
        } catch (InvocationTargetException ite) {
            thrown = ite.getTargetException();
            throw thrown;
        } finally {
            for (LogBehavior b : behaviors) {
                b.afterInvoke(method, result, thrown);
            }
        }
    }

    void updateBehaviors(List<LogBehavior> newBehaviors) {
        this.behaviors = List.copyOf(newBehaviors);
    }
}
```

// wiring

```
class LoggerFactory {
    private static final ThreadLocal<String> REQUEST_ID = new ThreadLocal<>();
    public static void setRequestId(String id) { REQUEST_ID.set(id); }

    public static Logger get(String name, List<LogBehavior> initial) {
```

```
        SimpleLogger base = new SimpleLogger(name);
        LoggerProxy handler = new LoggerProxy(base, initial);
        return (Logger) Proxy.newProxyInstance(
                Logger.class.getClassLoader(),
                new Class<?>[]{Logger.class},
                handler
        );
    }
}
```

Usage with live toggles (no recompilation):

// pick behaviors based on runtime config flags

```
boolean prod = true;
boolean bufferMode = true;

List<LogBehavior> behaviors = new ArrayList<>();
if (prod) behaviors.add(new DropDebugBehavior());
behaviors.add(new PrependRequestIdBehavior(/* ThreadLocal */ new
ThreadLocal<>()));
if (bufferMode) behaviors.add(new BufferUntilErrorBehavior());

Logger log = LoggerFactory.get("checkout", behaviors);
```

// later, flip flags from a config watcher and call handler.updateBehaviors(...)

Reasoning to interviewer:

- Dynamic proxies let me intercept every Logger API call without touching the implementation—perfect for cross-cutting concerns.
- Modeling each concern as a LogBehavior keeps things testable and composable; the handler just runs the chain.
- Dropping DEBUG is a clean “return null” in beforeInvoke. Prepending request-ID is just modifying the first String arg. Buffer-until-error uses a simple queue and flushes on error.
- Swapping behaviors is an atomic list update, so we can react to config changes (e.g., flip to verbose logging on a single node) at runtime.

Future scope:

- Add a small DSL in config (YAML/JSON) to declare the behavior chain per logger/category.

- Support per-thread MDC via a shared ThreadLocal<Map<String,String>> and a behavior that formats it.
- Add a rate-limit behavior to throttle noisy call-sites without code changes.
- Expose metrics (dropped logs count, buffer size) to Prometheus to spot misconfigurations early.
- For high-perf paths, migrate hotspots to bytecode generation (ByteBuddy) while keeping the same behavior SPI.

Scenario 5: Custom class loading with cyclic plugin references

A reporting engine loads plugins at runtime from separate JARs: “formatters” and “data sources.” Some of these plugins depend on each other’s small utility classes, creating cycles during class resolution. We need safe loading without duplicate definitions, deadlocks, or linkage errors.

Answer:

I’d isolate the engine from plugins and still allow plugins to depend on each other’s exported packages.

- Parent-first only for platform/engine packages (java.*, jakarta.*, logging, and the engine’s own packages).
- Child-first for everything else so a plugin can use its bundled libs without the engine’s versions leaking in.
- A global broker to ensure exactly one Class<?> definition per fully qualified name across all plugin loaders. Others “join” that definition via a future instead of redefining.
- Parallel-capable loaders and per-class locks (getClassLoadingLock) to avoid deadlocks.
- A small “export index” mapping exported package → owning PluginClassLoader so cross-plugin lookups are directed, not brute-forced.

Code:

```
import java.net.URL;
import java.net.URLClassLoader;
import java.util.Set;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.*;

final class ClassLoadingBroker {
    // className -> class definition in progress or completed
    private static final ConcurrentHashMap<String, CompletableFuture<Class<?>>>
inFlight = new ConcurrentHashMap<>();

    static Class<?> defineOnce(String className, Callable<Class<?>> definer)
throws ClassNotFoundException {
        CompletableFuture<Class<?>> f = inFlight.computeIfAbsent(className, k ->
new CompletableFuture<>());
```

```
        boolean iAmOwner = !f.isDone(); // the thread that put the CF is the "owner"
to define
        if (iAmOwner) {
            try {
                Class<?> c = definer.call();
                f.complete(c);
                return c;
            } catch (Throwable t) {
                f.completeExceptionally(t);
                // convert linkage to CNFE for callers outside
                if (t instanceof ClassNotFoundException) throw
(ClassNotFoundException) t;
                throw new ClassNotFoundException("Failed to define " + className, t);
            } finally {
                inFlight.remove(className, f);
            }
        } else {
            try {
                return f.join();
            } catch (CompletionException ce) {
                Throwable cause = ce.getCause() != null ? ce.getCause() : ce;
                if (cause instanceof ClassNotFoundException) throw
(ClassNotFoundException) cause;
                throw new ClassNotFoundException("Failed to join definition for " +
className, cause);
            }
        }
    }
}

final class PluginIndex {
    // exportedPackage -> the loader that owns it
    private final ConcurrentMap<String, PluginClassLoader> exports = new
ConcurrentHashMap<>();

    void registerExports(PluginClassLoader loader, Set<String> exportedPackages)
{
        for (String p : exportedPackages) exports.put(p, loader);
    }

    Optional<PluginClassLoader> ownerOf(String className) {
        int lastDot = className.lastIndexOf('.');
        String pkg = (lastDot < 0) ? "" : className.substring(0, lastDot);
        return Optional.ofNullable(exports.get(pkg));
    }
}

class PluginClassLoader extends URLClassLoader {
    static { ClassLoader.registerAsParallelCapable(); }
```

```
    private final PluginIndex index;
    private final Set<String> parentFirstPackages;
    private final Set<String> myExports;

    PluginClassLoader(URL[] urls,
                      ClassLoader parent,
                      PluginIndex index,
                      Set<String> parentFirstPackages,
                      Set<String> myExports) {
        super(urls, parent);
        this.index = Objects.requireNonNull(index);
        this.parentFirstPackages = Objects.requireNonNull(parentFirstPackages);
        this.myExports = Objects.requireNonNull(myExports);
        index.registerExports(this, myExports);
    }

    @Override
    protected Class<?> loadClass(String name, boolean resolve) throws
ClassNotFoundException {
        final Object lock = getClassLoadingLock(name);
        synchronized (lock) {
            // Fast path: already loaded
            Class<?> c = findLoadedClass(name);
            if (c != null) return c;

            boolean parentFirst = isParentFirst(name);

            if (parentFirst) {
                // engine/platform first
                try {
                    c = super.loadClass(name, false);
                    if (resolve) resolveClass(c);
                    return c;
                } catch (ClassNotFoundException ignore) {
                    // fall through to child
                }
            } else {
                // Try defining locally first (child-first)
                try {
                    c = defineLocallyWithBroker(name);
                    if (resolve) resolveClass(c);
                    return c;
                } catch (ClassNotFoundException ignore) {
                    // not in this jar, try cross-plugin
                    Optional<PluginClassLoader> owner = index.ownerOf(name);
                    if (owner.isPresent() && owner.get() != this) {
                        c = owner.get().loadClass(name, false);
                        if (resolve) resolveClass(c);
                        return c;
                    }
```

```
            }
        }

        // Fallback to parent
        c = super.loadClass(name, false);
        if (resolve) resolveClass(c);
        return c;
    }
  }

  private boolean isParentFirst(String className) {
    // Parent-first for java.*, javax.*, jakarta.*, and engine namespaces
    return className.startsWith("java.")
        || className.startsWith("javax.")
        || className.startsWith("jakarta.")
        || parentFirstPackages.stream().anyMatch(className::startsWith);
  }

  private Class<?> defineLocallyWithBroker(String name) throws
ClassNotFoundException {
    return ClassLoadingBroker.defineOnce(name, () -> {
        // If the class belongs to another plugin's export, don't define it here.
        Optional<PluginClassLoader> owner = index.ownerOf(name);
        if (owner.isPresent() && owner.get() != this) {
            // delegate to owner to keep single definition
            return owner.get().loadClass(name, false);
        }
        // Only define classes actually present in this plugin JAR
        try {
            return findClass(name);
        } catch (ClassNotFoundException e) {
            throw e;
        }
    });
  }
}
```

Reasoning to interviewer:

- The selective delegation strategy keeps the JVM stable: platform and engine APIs are loaded once by the parent; plugins retain control over their own libs, avoiding version clashes.
- The broker makes class definition idempotent across loaders. If two plugins try to load the same class during a cycle, only one defines; the other joins that result. That prevents LinkageError: duplicate class definition.

- registerAsParallelCapable + getClassLoadingLock(name) ensures fine-grained locking per class, which eliminates global-loader deadlocks when A loads B while B loads A.
- The export index turns cross-plugin references into a direct hop instead of “try every loader,” reducing contention and surprises.
- Each plugin has its own loader, which makes hot-unload or versioned reload feasible later without stopping the engine.

Future scope:

- Add “uses/provides” metadata (e.g., SPI) so only declared exported packages are visible; enforce with a small manifest schema.
- Track and reject split packages at registration time to avoid ambiguous ownership.
- Add a “capabilities” graph so a plugin can’t accidentally depend on non-exported internals of another plugin.
- Implement unload hooks and reference tracking to support safe plugin redeploys at runtime.
- Metrics and diagnostics: expose who-defined-what, resolution timings, and cycle detection logs for production debugging.

Scenario 6: Using JPMS for a service-oriented payments codebase

We're modularizing a payments system into three JPMS modules: ledger, payments.api, and settlement. Only public API surfaces should be usable by other modules; internals must be hidden to prevent accidental coupling.

Answer:

I'd make a thin payments.api module that only exports the contracts (interfaces, DTOs). The implementation modules (ledger, settlement) depend on payments.api, but they don't export their internal packages. If we use service loader, payments.api declares the service interfaces; implementations in ledger/settlement "provide" them. We keep internals encapsulated by default (no exports) and only open specific packages for reflection if needed (e.g., JSON mapping).

Code:

```
module-info.java for the API (payments.api):


module com.example.payments.api {
    exports com.example.payments.api;          // interfaces, DTOs
    uses com.example.payments.api.PaymentProcessor; // optional: service
discovery
}

Example API surface:


package com.example.payments.api;

import java.math.BigDecimal;
import java.util.UUID;

public interface PaymentProcessor {
    PaymentResult authorize(UUID merchantId, BigDecimal amount, String
currency);
    PaymentResult capture(UUID authorizationId);
    PaymentResult refund(UUID paymentId, BigDecimal amount);
}

public record PaymentResult(UUID id, boolean success, String message) {}

module-info.java for ledger (implementation, hidden internals):


module com.example.ledger {
```

```
    requires com.example.payments.api;
    requires java.sql;

    // Do NOT export internals. Keep everything encapsulated by default.
    // If we use ServiceLoader:
    provides com.example.payments.api.PaymentProcessor
        with com.example.ledger.LedgerPaymentProcessor;

    // If a serializer (e.g., Jackson) needs reflection, open only that package:
    opens com.example.ledger.serialization to com.fasterxml.jackson.databind;
}
```

module-info.java for settlement (another implementation or collaborator):

```
module com.example.settlement {
    requires com.example.payments.api;
    requires java.net.http;

    // Not exported: implementation details stay private to this module
    provides com.example.payments.api.PaymentProcessor
        with com.example.settlement.SettlementPaymentProcessor;
}
```

Minimal implementation class (ledger):

```
package com.example.ledger;

import com.example.payments.api.*;
import java.math.BigDecimal;
import java.util.UUID;

public final class LedgerPaymentProcessor implements PaymentProcessor {
    @Override
    public PaymentResult authorize(UUID merchantId, BigDecimal amount, String
currency) {
        // persist authorization in ledger tables, return id
        return new PaymentResult(UUID.randomUUID(), true, "AUTH_OK");
    }

    @Override
    public PaymentResult capture(UUID authorizationId) {
        // mark as captured and write journal entries
        return new PaymentResult(authorizationId, true, "CAPTURED");
    }

    @Override
    public PaymentResult refund(UUID paymentId, BigDecimal amount) {
        // create reversal entries
        return new PaymentResult(paymentId, true, "REFUND_OK");
```

```
        }
}

Consuming with ServiceLoader in a client (e.g., a CLI or a REST module):


ServiceLoader<com.example.payments.api.PaymentProcessor> loader =
        ServiceLoader.load(com.example.payments.api.PaymentProcessor.class);

PaymentProcessor impl = loader.findFirst()
        .orElseThrow(() -> new IllegalStateException("No PaymentProcessor
found"));
// use impl.authorize(...), etc.
```

Reasoning to interviewer:

- By default, JPMS hides everything. Only explicitly exported packages are accessible, which prevents accidental imports of “impl” classes.
- Keeping payments.api tiny makes it stable; implementations can evolve independently in ledger/settlement.
- The provides/uses pattern decouples callers from concrete classes. Callers depend only on the interface; implementations are wired by ServiceLoader at runtime, which is a clean form of dependency inversion.
- opens is used narrowly for frameworks that rely on reflection. We don’t open entire modules; we open only specific packages to specific frameworks.
- Because modules declare requires explicitly, the module graph documents dependencies and the compiler/jlink can catch illegal access at build time.

Future scope:

- Add split API surfaces, e.g., com.example.payments.api.spi for optional extension points, keeping core DTOs stable.
- Use jdeps in CI to fail builds if any module accesses a non-exported package or introduces an undeclared dependency.
- Add module versioning and a compatibility policy (semantic version checks) to avoid accidental breaking changes in payments.api.

- Create a “facade” module (com.example.payments.app) that re-exports only selected APIs with requires transitive for convenience, while still hiding internals.
- Produce a custom runtime image with jlink so only needed modules are shipped, reducing attack surface and startup time.

Scenario 7: Replacing verbose DTOs with Records

We have lots of simple, immutable data carriers travelling between services (customer profile, shipment summary, invoice line). They're currently verbose POJOs full of boilerplate. We want to migrate them to Java Records while staying friendly to serializers and other frameworks.

Answer:

I'd move each DTO to a record where the state is just its components. Records give us:

- Built-in constructor, accessors, equals/hashCode, and toString without Lombok or boilerplate.
- Stronger immutability signals: fields are final and accessors are read-only.
- Clearer intent for "data-only" objects in APIs and events.

Key points I'd handle during migration:

- Immutability: records are shallowly immutable. If a component is a mutable type (e.g., List), I'll wrap it defensively (List.copyOf) in a compact constructor.
- Validation and invariants: put guards in the compact constructor so no invalid state leaks.
- Serialization (JSON/Java/Kafka): most modern JSON mappers (Jackson 2.12+/Spring Boot 3+) understand records via the canonical constructor. I'll annotate components if names differ or if null-handling needs control.
- JPA: I won't make JPA entities records (JPA needs a no-arg ctor and often setters). Instead, I'll keep entities as classes and use records for projections/DTOs at boundaries.

Code:

```
// Example: replacing a POJO InvoiceLine with a record
import java.math.BigDecimal;
import java.util.List;
import java.util.Objects;

public record InvoiceLine(
        String sku,
        int quantity,
        BigDecimal unitPrice,
```

```
        List<String> tags // beware: List is mutable by default
) {
    // Compact constructor to enforce invariants / defensive copies
    public InvoiceLine {
        if (sku == null || sku.isBlank()) throw new IllegalArgumentException("sku
required");
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be > 0");
        if (unitPrice == null || unitPrice.signum() < 0) throw new
IllegalArgumentException("unitPrice >= 0");
        tags = (tags == null) ? List.of() : List.copyOf(tags); // defensive copy for
shallow immutability
    }

    // Convenience factory: totals etc.
    public BigDecimal lineTotal() {
        return unitPrice.multiply(BigDecimal.valueOf(quantity));
    }
}

// JSON serialization with Jackson (works out-of-the-box in recent versions):
// If names differ, or you need hints:
// public record CustomerProfile(@JsonProperty("id") String customerId,
@JsonProperty("email") String email) { ... }

// Spring Data / JPA projection example (entity -> DTO):
// @Query("select new com.example.api.ShipmentSummary(s.id, s.status, s.eta)
from Shipment s where s.id=:id")
// ShipmentSummary is a record; Hibernate constructs it via the canonical
constructor.
public record ShipmentSummary(String id, String status, String eta) {}
```

Reasoning to interviewer:

- Using records for DTOs communicates “this is just data,” and the compiler generates the semantics we’d otherwise hand-code.
- The compact constructor is the right place to enforce business invariants and to wrap mutable inputs (e.g., List.copyOf), keeping the record effectively immutable in practice.
- Keeping JPA entities as regular classes sidesteps ORM constraints (no-arg ctor, proxies, lazy setters). Records fit perfectly at module boundaries, REST payloads, Kafka events, and query projections.
- JSON mappers that use constructors (Jackson/Micronaut) naturally bind to the record’s canonical constructor, so we avoid reflection brittleness and maintain stable schemas.

Future scope:

- Add Bean Validation annotations on components (e.g., @NotBlank sku) to centralize validation.
- Introduce dedicated value types (Money, Email) as records to reduce primitive obsession and validation drift.
- Standardize defensive copies for all collection components and document it in a small “DTO guidelines” page.
- For Protobuf/Avro, generate codecs that map cleanly to records (constructor-based) and add contract tests to detect schema drift.

Scenario 8: Cleaner type-specific logic using instanceof pattern matching

A message parser receives generic Objects and must route based on type: Integer counters, String commands, and domain types like Price. Current code does a lot of "if (x instanceof T) { T t = (T) x; … }" noise and casts.

Answer:

I'd use instanceof pattern matching to both test and bind the variable in one step. It removes the extra cast, narrows scope, and makes the dispatch read like intentional routing logic. For domain types, I'll model them as value types (e.g., a Price record), so the parser stays explicit and safe.

Code:

```
// A tiny domain value
public record Price(java.math.BigDecimal amount, String currency) {
    public Price {
        if (amount == null || amount.signum() < 0) throw new
IllegalArgumentException("amount >= 0");
        if (currency == null || currency.isBlank()) throw new
IllegalArgumentException("currency required");
    }
}

public final class MessageParser {

    // Old (noisy) style:
    public String parseOld(Object payload) {
        if (payload instanceof Integer) {
            Integer i = (Integer) payload;
            return "COUNTER=" + i;
        } else if (payload instanceof String) {
            String s = (String) payload;
            return "CMD=" + s.trim().toUpperCase();
        } else if (payload instanceof Price) {
            Price p = (Price) payload;
            return "PRICE=" + p.amount() + " " + p.currency();
        } else {
            return "UNSUPPORTED";
        }
    }

    // New style with instanceof pattern matching (Java 17+):
    public String parse(Object payload) {
        if (payload instanceof Integer i) {
            return "COUNTER=" + i;
```

```
        } else if (payload instanceof String s) {
            return "CMD=" + s.trim().toUpperCase();
        } else if (payload instanceof Price p) {
            return "PRICE=" + p.amount() + " " + p.currency();
        } else {
            return "UNSUPPORTED";
        }
    }
}
```

If we're on Java 21+, we can go one step further with switch on type patterns (even clearer intent):

```
public String parseSwitch(Object payload) {
    return switch (payload) {
        case Integer i -> "COUNTER=" + i;
        case String s  -> "CMD=" + s.trim().toUpperCase();
        case Price p   -> "PRICE=" + p.amount() + " " + p.currency();
        case null      -> "UNSUPPORTED"; // explicit null arm
        default        -> "UNSUPPORTED";
    };
}
```

Reasoning to interviewer:

- Pattern matching narrows and binds in one expression, so we avoid redundant casts and the risk of drifting types after refactors.
- The control flow reads like a router: each arm documents exactly what type is expected and what transformation happens. That reduces subtle ClassCastException paths and makes code review simpler.
- Using a domain value (Price) makes downstream code safer—no ambiguous Pair<BigDecimal,String> or raw Map lookups—so our parser contract is self-documenting.
- With a switch on type patterns (Java 21+), we get an expression that's exhaustive and compiler-checked; adding a new type forces us to handle it explicitly.

Future scope:

- Adopt sealed interfaces for message hierarchies; combined with switch on patterns, we get compile-time exhaustiveness for new message types.

- Centralize routing in a “MessageRouter” that maps types to handlers (Map<Class<?>, Handler<?>>) but still uses pattern matching within handlers for sub-cases.
- Add telemetry: log the matched arm and timing to understand traffic mix and catch misrouted payloads early.
- If parsing external JSON, deserialize directly into the right record/class first (polymorphic deserialization), then route—reduces “Object” handling at the boundaries.

Scenario 9: Using switch pattern matching for heterogeneous inputs

We have a normalization step that receives mixed inputs (numbers, strings, URIs). The current approach uses a mix of switches and if/else type checks, making the flow noisy and error-prone. We want a single, readable dispatch that's explicit about all handled shapes, including null.

Answer:

I'll refactor to a single switch expression with type patterns (Java 21+). Each arm both matches the type and binds a variable, so we avoid manual casts. I'll also add an explicit null arm and a default arm for anything unexpected. The result is exhaustive, intention-revealing, and easy to extend.

Code:

```
import java.net.URI;
import java.math.BigDecimal;

public final class Normalizer {

    // Normalize to a canonical string form
    public static String normalize(Object in) {
        return switch (in) {
            case null -> "null"; // explicit and visible
            case Integer i      -> String.valueOf(i);
            case Long l         -> String.valueOf(l);
            case Double d       -> stripTrailingZeros(BigDecimal.valueOf(d));
            case BigDecimal bd  -> stripTrailingZeros(bd);
            case String s       -> s.trim().isEmpty() ? "" : s.trim();
            case URI uri        -> uri.normalize().toString();
            // group related numeric primitives via Number (careful with
Double/BigDecimal above)
            case Number n       -> n.toString();
            default             -> in.toString(); // known fallback; could also throw if strict
        };
    }

    private static String stripTrailingZeros(BigDecimal bd) {
        return bd.stripTrailingZeros().toPlainString();
    }
}
```

Reasoning to interviewer:

- The switch on type patterns eliminates scattered instanceof + casts and expresses routing as data-shape → action.
- The explicit case null makes null handling obvious and prevents hidden NPEs.
- Ordering matters: more specific numeric cases (Double/BigDecimal) appear before the broader Number, avoiding accidental capture.
- Because this is a switch expression, there's no fall-through; each arm produces a value, so control flow is tight and compiler-checked.
- Extensibility is simple: add a new case arm (e.g., case Path p -> …) and the compiler enforces completeness where needed.

Future scope:

- Introduce domain value types (e.g., Amount, Email) and add dedicated arms for them to push validation to the edges.
- For strictness, replace default -> in.toString() with a default -> throw new IllegalArgumentException(...) and whitelist only approved shapes.
- If inputs evolve into a closed set, use a sealed hierarchy and rely on compiler exhaustiveness (no default arm needed).
- Add small metrics (matched arm counters) to spot unexpected input shapes early in prod.

Scenario 10: Accelerating numeric kernels with the Vector API

A risk engine does heavy math on primitive arrays (scale, add/mul, dot products). Plain for-loops are CPU-bound and not using SIMD. We want portable SIMD without dropping to JNI.

Answer:

I'll rewrite the hot loops with the Vector API (jdk.incubator.vector). I'll pick the preferred species for the element type (e.g., FloatVector.SPECIES_PREFERRED), which maps to the widest SIMD supported by the CPU at runtime. I'll process elements in vector-sized chunks, then handle the tail with a mask. The JIT compiles these to AVX/NEON/etc. where available and falls back gracefully to scalar when not.

Code:

```
// Requires a JDK with the Vector API (incubator in many JDKs).
// Example kernels: scale, axpy (y = a*x + y), dot product.

import jdk.incubator.vector.*;

public final class Kernels {

    private static final FloatVector.Species SPEC =
FloatVector.SPECIES_PREFERRED;

    // y[i] = a * x[i] + y[i]
    public static void axpy(float a, float[] x, float[] y) {
        int i = 0, upperBound = SPEC.loopBound(x.length);
        FloatVector va = FloatVector.broadcast(SPEC, a);

        for (; i < upperBound; i += SPEC.length()) {
            var vx = FloatVector.fromArray(SPEC, x, i);
            var vy = FloatVector.fromArray(SPEC, y, i);
            var vres = vx.fma(va, vy); // va * vx + vy, maps to fused multiply-add when
available
            vres.intoArray(y, i);
        }

        // tail
        if (i < x.length) {
            var m = SPEC.indexInRange(i, x.length);
            var vx = FloatVector.fromArray(SPEC, x, i, m);
            var vy = FloatVector.fromArray(SPEC, y, i, m);
            var vres = vx.fma(va, vy);
            vres.intoArray(y, i, m);
```

```
        }
    }

    // scale: x[i] *= a
    public static void scale(float a, float[] x) {
        int i = 0, ub = SPEC.loopBound(x.length);
        var va = FloatVector.broadcast(SPEC, a);

        for (; i < ub; i += SPEC.length()) {
            var vx = FloatVector.fromArray(SPEC, x, i);
            vx.mul(va).intoArray(x, i);
        }
        if (i < x.length) {
            var m = SPEC.indexInRange(i, x.length);
            var vx = FloatVector.fromArray(SPEC, x, i, m);
            vx.mul(va).intoArray(x, i, m);
        }
    }

    // dot product: sum_i (x[i] * y[i])
    public static float dot(float[] x, float[] y) {
        int i = 0, ub = SPEC.loopBound(x.length);
        var acc = FloatVector.zero(SPEC);

        for (; i < ub; i += SPEC.length()) {
            var vx = FloatVector.fromArray(SPEC, x, i);
            var vy = FloatVector.fromArray(SPEC, y, i);
            acc = vx.fma(vy, acc); // acc += vx * vy
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);

        // tail
        for (; i < x.length; i++) sum += x[i] * y[i];
        return sum;
    }
}
```

Reasoning to interviewer:

- Species selection (SPECIES_PREFERRED) chooses the widest vector size available at runtime (e.g., 256-bit AVX2, 512-bit AVX-512, 128-bit NEON), keeping code portable and fast.
- Loading/storing uses fromArray/intoArray, which the JIT lowers to aligned/unaligned SIMD moves as appropriate; we don’t manage intrinsics by hand.

- The tail is handled via indexInRange masks or a simple scalar cleanup, so correctness isn’t sacrificed when length % vectorWidth != 0.
- Operations like fma, mul, and reduceLanes map directly to SIMD instructions and horizontal reductions; when hardware lacks a feature, the API still runs, just with lower speedups.
- Because this is pure Java, we avoid JNI overhead and keep GC/escape analysis friendly; hot loops become auto-vectorized via explicit vectors rather than relying on sometimes-fragile auto-vectorization.

Future scope:

- Specialize for double[] with DoubleVector where numeric stability matters; use mixed precision intentionally (float vs double) based on the risk model.
- Batch multiple independent dots into a small matrix-vector kernel to improve cache locality and amortize loop overhead.
- Add prefetch-friendly tiling or block sizes for very large arrays to reduce cache misses.
- Expose these kernels behind a simple interface and keep a scalar fallback for older JDKs; add JMH benchmarks in CI to catch regressions across JDK/CPU combos.

Scenario 11: Reducing object churn in a stock-trading engine

In a real-time trading platform, short-lived objects (Price, Order, helper DTOs) explode during peaks, creating GC pressure and jitter. We want to keep throughput high and latency predictable by cutting unnecessary allocations.

Answer:

I'd attack allocations on three fronts: represent data with primitives/compact types, reuse objects in hot paths, and avoid hidden allocations in strings/boxing/streams.

- Prefer primitives over wrappers. Represent money as long cents (or long micros) instead of BigDecimal in hot paths; convert to BigDecimal only at the boundary.
- Avoid boxing. Use primitive arrays and counters (LongAdder/LongAccumulator) and APIs that take primitives; don't put long into List<Long> on the wire path.
- Reuse mutable carriers in the pipeline. Pass a single mutable OrderEvent through the matching engine, reused per slot (ring buffer pattern) instead of creating a new DTO per hop.
- Preallocate and recycle buffers. Keep per-thread StringBuilder/ByteBuffer/CharBuffer via ThreadLocal with sensible max capacity to avoid repeated growth/GC.
- Avoid string churn. Prefer StringBuilder (preset capacity), avoid String.format, and guard logging with logger.isDebugEnabled() so you don't build messages you'll drop.
- No "accidental" streams/Optionals on hot paths. Replace stream pipelines with simple for-loops; remove Optional in inner loops.
- Flyweight/lookup tables for stable data. Venue codes, currency codes, and instrument symbols are reused; intern or resolve to small ids (short/int) once, then carry ids.
- Object pools, but only for truly heavy or external objects (e.g., ByteBuffer, parsers). For simple tiny DTOs, escape analysis + young-gen GC is often cheaper than pooling.
- Cache expensive helpers per thread. Parsers/formatters (e.g., DateTimeFormatter, DecimalFormat) in ThreadLocal to avoid reconstructing.

- Batch work to amortize overhead. Parse N messages into a reusable buffer, then process, reducing per-message allocations.

Code:

```
// 1) Money as primitive + boundary conversion
public final class Money {
    // value in micro-units of currency (e.g., USD micro-dollars)
    private final long micros;
    private final short currencyId; // map USD->1, EUR->2, etc.

    public Money(long micros, short currencyId) {
        if (micros < 0) throw new IllegalArgumentException("negative");
        this.micros = micros;
        this.currencyId = currencyId;
    }
    public long micros() { return micros; }
    public short currencyId() { return currencyId; }
    // convert only at boundary (UI/reporting)
    public java.math.BigDecimal toBigDecimal(int scale) {
        return java.math.BigDecimal.valueOf(micros, scale);
    }
}

// 2) Reusable ring-buffer style event (LMAX-style)
final class OrderEvent {
    // mutable fields reused per slot
    long orderId;
    int instrumentId;
    short side;      // 1=BUY, 2=SELL
    long qty;        // in shares
    long priceMicros;

    void set(long orderId, int instrumentId, short side, long qty, long priceMicros) {
        this.orderId = orderId;
        this.instrumentId = instrumentId;
        this.side = side;
        this.qty = qty;
        this.priceMicros = priceMicros;
    }
}

// 3) Thread-local reusable builders/buffers
final class Builders {
    private static final int DEFAULT = 128;
    private static final ThreadLocal<StringBuilder> SB =
        ThreadLocal.withInitial(() -> new StringBuilder(DEFAULT));
```

```
    static String formatTrade(long orderId, long qty, long priceMicros) {
        StringBuilder sb = SB.get();
        sb.setLength(0);
        sb.append("order=").append(orderId)
          .append(",qty=").append(qty)
          .append(",px=").append(priceMicros);
        String out = sb.toString(); // one allocation for the final String
        if (sb.capacity() > 4096) sb.setLength(DEFAULT); // optional cap policy
        return out;
    }
}

// 4) Avoid boxing in counters
import java.util.concurrent.atomic.LongAdder;
final class Metrics {
    final LongAdder matchedOrders = new LongAdder();
    void onMatched() { matchedOrders.increment(); }
}
```

Reasoning to interviewer:

- Using primitive representations (money in micros, ids for symbols) removes BigDecimal/String churn from the hot path while preserving exactness; we only convert at I/O edges.
- The ring-buffer model keeps a fixed number of OrderEvent objects live; work moves through the buffer without creating new carriers, which flattens allocation spikes during bursts.
- ThreadLocal builders and preallocated buffers avoid repeat growth and GC while remaining contention-free; they're released when the thread dies.
- Guarded logging and avoiding String.format/streams prevent hidden, easy-to-miss allocations that add up at millions of msgs/hour.
- Object pools are used surgically—only where the object is heavy or external. For small objects, modern G1/ZGC handle short-lived allocations well; pooling them can make performance worse due to cache misses and synchronization.

Future scope:

- Adopt a disruptor-style pipeline or SPSC/MPSC queues with fixed slots for all market data paths.
- Off-heap ring buffers (e.g., DirectByteBuffer) for serialization; pin layouts and reuse slices to reduce heap pressure.

- Structured concurrency + virtual threads for I/O fan-out while keeping compute on a small CPU-pinned pool to reduce allocator contention.
- Continuous allocation profiling (JFR, async-profiler alloc events) in canary to catch regressions; set budgets (allocs/msg).
- If accuracy permits, vectorize math on primitive arrays (Java Vector API) for batches of risk calcs to cut both CPU and allocation overhead.

Scenario 12: Modularizing a university management system with JPMS

We want strong boundaries between courses, students, and enrollment. Each area exposes a tight API and hides its internals so other modules can't "accidentally" depend on implementation classes.

Answer:

I'd split the codebase into small API modules and implementation modules. APIs export only contract packages (interfaces/DTOs); impl modules require the APIs and optionally "provide" services via ServiceLoader. Nothing else is exported. Shared types (e.g., common ids, errors) live in a tiny university.common API re-exported where convenient.

- university.common — shared value types and errors (exports).
- university.courses.api — course contracts (exports).
- university.courses.impl — implementations; no exports, only provides.
- university.students.api — student contracts (exports).
- university.students.impl — implementations; no exports, only provides.
- university.enrollment.api — enrollment contracts (exports).
- university.enrollment.impl — business logic; depends on the other APIs, provides enrollment service.
- university.app — composition layer (REST/CLI) that uses the APIs (uses) and wires implementations via ServiceLoader.

Code:

module-info.java for common:

```
module edu.university.common {
    exports edu.university.common;     // StudentId, CourseId, Error codes, etc.
}

module-info.java for courses API:


module edu.university.courses.api {
    requires transitive edu.university.common; // downstream users get common
automatically
    exports edu.university.courses.api;        // Course, CourseCatalog, queries
    uses edu.university.courses.api.CourseCatalog; // optional: discover impl via
ServiceLoader
```

```
}
```

courses API surface (example):

```
package edu.university.courses.api;

import edu.university.common.CourseId;
import java.util.List;

public interface CourseCatalog {
    List<CourseSummary> listByDepartment(String deptCode);
    CourseDetails get(CourseId id);
}

public record CourseIdDTO(String value) {}
public record CourseSummary(CourseId id, String title) {}
public record CourseDetails(CourseId id, String title, int credits) {}
```

module-info.java for courses impl:

```
module edu.university.courses.impl {
    requires edu.university.courses.api;
    requires edu.university.common;
    requires java.sql;

    // Keep internals hidden: NO exports
    provides edu.university.courses.api.CourseCatalog
        with edu.university.courses.impl.JdbcCourseCatalog;

    // If JSON serialization via Jackson is needed internally, open only that package:
    opens edu.university.courses.impl.serialization to
com.fasterxml.jackson.databind;
}
```

students API and impl (similar):

```
module edu.university.students.api {
    requires transitive edu.university.common;
    exports edu.university.students.api;
    uses edu.university.students.api.StudentDirectory;
}

module edu.university.students.impl {
    requires edu.university.students.api;
    requires edu.university.common;
    provides edu.university.students.api.StudentDirectory
        with edu.university.students.impl.JdbcStudentDirectory;
}
```

enrollment API and impl:

```
module edu.university.enrollment.api {
    requires transitive edu.university.common;
    requires transitive edu.university.courses.api;
    requires transitive edu.university.students.api;
    exports edu.university.enrollment.api;
    uses edu.university.enrollment.api.EnrollmentService;
}

module edu.university.enrollment.impl {
    requires edu.university.enrollment.api;
    requires edu.university.courses.api;
    requires edu.university.students.api;
    requires edu.university.common;

    provides edu.university.enrollment.api.EnrollmentService
        with edu.university.enrollment.impl.DefaultEnrollmentService;
}
```

composition/app module:

```
module edu.university.app {
    requires edu.university.common;
    requires edu.university.courses.api;
    requires edu.university.students.api;
    requires edu.university.enrollment.api;

    uses edu.university.courses.api.CourseCatalog;
    uses edu.university.students.api.StudentDirectory;
    uses edu.university.enrollment.api.EnrollmentService;
}
```

Service usage (composition):

```
var catalog =
java.util.ServiceLoader.load(edu.university.courses.api.CourseCatalog.class)
                               .findFirst().orElseThrow();
var directory =
java.util.ServiceLoader.load(edu.university.students.api.StudentDirectory.class)
                                .findFirst().orElseThrow();
var enrollSvc =
java.util.ServiceLoader.load(edu.university.enrollment.api.EnrollmentService.class)
                                .findFirst().orElseThrow();
```

Reasoning to interviewer:

- By default JPMS hides everything; only explicitly exported packages are visible. Keeping impl modules with zero exports prevents “accidental” coupling to implementation classes.
- uses/provides decouples API consumers from implementations; swapping JDBC for JPA or in-memory versions doesn’t ripple across modules.
- requires transitive on API modules forwards common dependencies (like common) to consumers, reducing leakage of implementation details.
- Scoped opens avoids blanket reflection exposure; only specific packages are open to specific frameworks.
- The module graph is explicit, giving compile-time checks and enabling jlink to ship a trimmed runtime.

Future scope:

- Add CI guards with jdeps/Revapi to fail builds on illegal package access or accidental API breaks.
- Version the API modules and document compatibility rules; gate merges on semantic version checks.
- Provide test fixtures as separate test modules (e.g., edu.university.courses.testkit) so black-box tests don’t require opening internals.
- Create an aggregator edu.university.platform module that re-exports only the stable APIs with requires transitive, leaving implementations swappable.

Scenario 13: Avoiding deadlocks in a hotel booking system

Two threads contend for two shared resources: room allocation and payment authorization. If Thread A locks Room then tries Payment, while Thread B locks Payment then tries Room, both can wait forever. We want predictable locking that never deadlocks under load.

Answer:

I'd combine three tactics: a strict global lock order, time-bounded acquisition with fallback, and design that minimizes nested locking.

- Global order: assign a consistent ordering to locks (e.g., by resource id or a rank map). Every code path acquires locks in that order. If roomLock < paymentLock, both features follow Room → Payment in that sequence.
- Timed acquisition: where cross-service coordination is unavoidable, use ReentrantLock.tryLock(timeout, unit) and back off (release what you hold, retry or compensate) instead of blocking forever.
- Reduce nesting: push expensive/slow steps outside critical sections. For example, compute pricing and build the payment request before acquiring the room lock; or serialize all operations per room with a per-room executor so you rarely need two locks at once. For cross-resource flows, consider "two-phase" staging: allocate a provisional hold on the room, perform payment, then finalize with a single lock.

Code:

```
import java.util.concurrent.*;
import java.util.concurrent.locks.*;
import java.util.*;

final class LockRegistry {
    // One lock per resource (room-123, payment-visa, etc.)
    private final ConcurrentMap<String, ReentrantLock> locks = new
ConcurrentHashMap<>();

    ReentrantLock lockOf(String key) {
        return locks.computeIfAbsent(key, k -> new ReentrantLock(true)); // fair lock
reduces starvation
    }

    // Acquire multiple locks in a strict global order (lexicographic on keys)
    List<ReentrantLock> acquireInOrder(List<String> keys) {
```

```
        List<String> sorted = new ArrayList<>(keys);
        Collections.sort(sorted); // global order
        List<ReentrantLock> acquired = new ArrayList<>(sorted.size());
        for (String k : sorted) {
            ReentrantLock l = lockOf(k);
            l.lock();
            acquired.add(l);
        }
        return acquired;
    }

    void releaseAll(List<ReentrantLock> acquired) {
        // Release in reverse order for hygiene
        ListIterator<ReentrantLock> it = acquired.listIterator(acquired.size());
        while (it.hasPrevious()) it.previous().unlock();
    }
}

final class BookingService {
    private final LockRegistry locks = new LockRegistry();

    // Strict lock ordering: ["room:<id>", "payment:<method>"] always in
lexicographic order
    public boolean bookWithPayment(String roomId, String payToken) {
        List<ReentrantLock> acquired = locks.acquireInOrder(List.of("room:" +
roomId, "payment:" + payToken));
        try {
            // room allocation + payment authorization done under consistent ordering
            allocateRoom(roomId);
            authorizePayment(payToken);
            confirm(roomId);
            return true;
        } finally {
            locks.releaseAll(acquired);
        }
    }

    // Alternative: timed acquisition to avoid deadlock and fail fast
    public boolean bookWithTimeout(String roomId, String payToken) throws
InterruptedException {
        ReentrantLock roomLock = locks.lockOf("room:" + roomId);
        ReentrantLock payLock  = locks.lockOf("payment:" + payToken);
        // Determine order once to avoid inversion
        ReentrantLock first = ("payment:" + payToken).compareTo("room:" + roomId)
< 0 ? payLock : roomLock;
        ReentrantLock second = (first == payLock) ? roomLock : payLock;

        if (!first.tryLock(50, TimeUnit.MILLISECONDS)) return false;
        try {
            if (!second.tryLock(50, TimeUnit.MILLISECONDS)) {
```

```
                // back off: release and let caller retry or compensate
                return false;
            }
            try {
                allocateRoom(roomId);
                authorizePayment(payToken);
                confirm(roomId);
                return true;
            } finally {
                second.unlock();
            }
        } finally {
            first.unlock();
        }
    }

    private void allocateRoom(String roomId) { /* fast, no remote calls */ }
    private void authorizePayment(String payToken) { /* quick path only; no blocking
I/O inside lock */ }
    private void confirm(String roomId) { /* persist final state */ }
}

// Design that reduces nested locking:
// Use a per-room single-threaded executor so operations on the same room never
interleave.
// Cross-room operations then rarely require two locks simultaneously.
final class PerRoomSerializers {
    private final ConcurrentMap<String, ExecutorService> perRoom = new
ConcurrentHashMap<>();
    public Future<Boolean> submit(String roomId, Callable<Boolean> task) {
        return perRoom.computeIfAbsent(roomId, id ->
Executors.newSingleThreadExecutor())
                      .submit(task);
    }
}
```

Reasoning to interviewer:

- A strict global order is the simplest, most reliable deadlock prevention: no cycle in the wait-for graph if every thread acquires locks in the same order.
- tryLock(timeout) puts a bounded upper limit on contention. On contention, I release and retry or trigger compensating actions (e.g., cancel payment hold) instead of risking a permanent stall.
- Shrinking critical sections (precompute outside locks, avoid blocking I/O while holding a lock) drastically reduces the time windows where cycles could form.

- Per-room serialization is pragmatic in booking: most conflicts are on the same room; a single-threaded executor per room avoids lock contention altogether for that hotspot.
- Where possible, I replace custom locks with concurrent primitives (e.g., ConcurrentHashMap.compute, StampedLock read dominance) to reduce explicit nested locking.

Future scope:

- Add deadlock detection in tests with simulated inversions and JFR event scanning; fail fast on regressions.
- Replace multi-lock flows with idempotent, message-driven steps (sagas) so each step holds at most one lock and uses retries/compensation for failures.
- Track critical section durations via metrics; alert if a section exceeds a threshold (likely I/O inside a lock).
- For DB-backed state, enforce the same ordering at the SQL level (e.g., consistent “lowest id first” SELECT … FOR UPDATE) to align JVM and DB locking strategies.

Scenario 14: Restricting inheritance with Sealed Classes in a transit app

Only Bus, Train, and Metro are valid vehicle categories. We want the type system to forbid any other extension of the Transport base.

Answer:

I'd model Transport as a sealed interface (or abstract class) and explicitly permit only Bus, Train, and Metro. Those concrete types are final so no one can extend them further. This gives us a closed hierarchy that the compiler understands, which also enables exhaustive pattern matching in switches.

Code:

// Java 17+

```
public sealed interface Transport permits Bus, Train, Metro {
    String routeCode();
    int capacity();
}

public final class Bus implements Transport {
    private final String routeCode;
    private final int capacity;

    public Bus(String routeCode, int capacity) {
        this.routeCode = routeCode;
        this.capacity = capacity;
    }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

public final class Train implements Transport {
    private final String routeCode;
    private final int capacity;
    public Train(String routeCode, int capacity) {
        this.routeCode = routeCode; this.capacity = capacity;
    }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

public final class Metro implements Transport {
    private final String routeCode;
    private final int capacity;
    public Metro(String routeCode, int capacity) {
        this.routeCode = routeCode; this.capacity = capacity;
```

```
    }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

// Usage with exhaustive pattern matching (Java 21+):
public final class FareCalculator {
    public int baseFare(Transport t) {
        return switch (t) {
            case Bus b   -> 10;
            case Train tr-> 15;
            case Metro m -> 12;
        }; // no default needed; compiler knows the sealed set is exhaustive
    }
}
```

If you need an extension point within a permitted type, mark that subtype non-sealed instead of final, and then you can extend only inside that branch:

public non-sealed class Train implements Transport { /* ... */ } // allows more specific trains
public final class ExpressTrain extends Train { /* ok */ }

Reasoning to interviewer:

- sealed with permits closes the hierarchy to a known set of subtypes at compile time. That both documents our policy (“only these vehicles”) and makes it impossible to extend from the outside.
- Marking the concrete classes final locks them down fully; or non-sealed where we want a controlled extension point internally.
- Switch pattern matching becomes safer: the compiler forces us to handle every permitted subtype, so missing cases are caught during builds rather than at runtime.
- This is a strong fit for mobile/transit domains where categories are business-enumerated and shouldn’t drift over time.

Future scope:

- Combine with JPMS: place Transport in a module and keep the permitted types in the same module (or list cross-module permits explicitly) to enforce boundaries.

- Add value-type records for immutable variants (e.g., record BusInfo(...) implements Transport) when the data is pure state.
- Use sealed hierarchies alongside validation at parsing boundaries (reject unknown vehicle types early) for defense in depth.
- Add versioning tests: if we ever add a new vehicle, the exhaustive switch will fail to compile—intentional friction that prompts API review.

Scenario 15: Capturing method execution time with dynamic proxies

We need per-method latency without changing business code. The methods already sit behind interfaces (e.g., TaskRunner, Scheduler). Perfect fit for JDK dynamic proxies: intercept calls at the interface boundary, measure time, log/emit metrics, then forward.

Answer:

I'll wrap target instances with a Proxy.newProxyInstance and an InvocationHandler. The handler records nanoTime() before/after method.invoke, computes the delta, and ships it to logging/metrics. I can make it opt-in via an annotation (e.g., @Timed) or opt-out for noisy methods. Exceptions are rethrown as-is so behavior stays identical. For classes without interfaces, I'd use ByteBuddy/CGLIB—but with pure JDK proxies, we stick to interfaces.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Objects;
import java.util.concurrent.TimeUnit;

// 1) Optional annotation to mark which methods we time
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.TYPE})
@interface Timed {
    String value() default ""; // optional metric name
}

// 2) Contract + impl (business code untouched)
interface TaskRunner {
    void runTask(String taskId);
    @Timed("heavy-op")
    String compute(String input);
}

final class TaskRunnerImpl implements TaskRunner {
    @Override public void runTask(String taskId) {
        // do work...
    }
    @Override public String compute(String input) {
        // simulate CPU
        for (int i = 0; i < 1_000_00; i++) {}
        return input.toUpperCase();
    }
```

```
}

// 3) InvocationHandler that times method calls
final class TimingInvocationHandler implements InvocationHandler {
    private final Object target;

    private TimingInvocationHandler(Object target) {
        this.target = Objects.requireNonNull(target);
    }

    public static <T> T timedProxy(Class<T> iface, T target) {
        return iface.cast(Proxy.newProxyInstance(
                iface.getClassLoader(),
                new Class<?>[]{iface},
                new TimingInvocationHandler(target)));
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        // Decide whether to time: method or type annotated?
        boolean timed = method.isAnnotationPresent(Timed.class)
                || target.getClass().isAnnotationPresent(Timed.class);

        if (!timed) {
            return method.invoke(target, args);
        }

        String metric = method.getAnnotation(Timed.class) != null
                ? method.getAnnotation(Timed.class).value()
                : method.getName();

        long start = System.nanoTime();
        boolean success = false;
        try {
            Object result = method.invoke(target, args);
            success = true;
            return result;
        } catch (InvocationTargetException ite) {
            // unwrap and rethrow user exception
            throw ite.getCause();
        } finally {
            long durNs = System.nanoTime() - start;
            logTiming(metric, method, durNs, success);
        }
    }

    private void logTiming(String metric, Method method, long durNs, boolean
success) {
        // Replace with Micrometer/Dropwizard/SLF4J as needed
```

```
        System.out.printf("metric=%s method=%s success=%s duration=%dms%n",
                metric, method.getName(), success,
TimeUnit.NANOSECONDS.toMillis(durNs));
    }
}

// 4) Usage
class Demo {
    public static void main(String[] args) {
        TaskRunner runner = new TaskRunnerImpl();
        TaskRunner timed = TimingInvocationHandler.timedProxy(TaskRunner.class,
runner);

        timed.runTask("T-42");          // not annotated → no timing log
        String out = timed.compute("abc"); // annotated → timing logged
    }
}
```

Reasoning to interviewer:

- Dynamic proxies let me graft cross-cutting concerns (timing) onto interface calls with zero changes to business classes.
- An annotation keeps the surface explicit: only marked methods get timed; avoids log noise.
- InvocationTargetException unwrapping preserves original exceptions and stack traces, which is important for debugging.
- This plays nicely with dependency injection: wrap beans at creation time and hand out the proxy everywhere else.
- For classes without interfaces, I'd switch to a bytecode proxy (ByteBuddy/CGLIB) or use AOP in the DI container.

Future scope:

- Emit structured metrics (Micrometer) with percentiles and tags (service, method, success/failure).
- Add sampling to avoid overhead on ultra-hot paths, or a threshold to only log slow calls.
- Cache Method → "isTimed/metricName" lookups to avoid reflectively inspecting on every call.

- Combine with context propagation (trace/span ids) so timings tie into distributed tracing.

Scenario 16: Cleaner query handling with switch pattern matching

Users query products by keyword (String), numeric id (Integer/Long), or UUID. The old code branches with instanceof plus casts. We want one concise, safe dispatcher.

Answer:

I'll use a type-pattern switch (Java 17+). Each arm both checks and binds, so there's no separate cast. I'll handle null explicitly, special-case empty strings, and keep a strict default for unsupported shapes.

Code:

```
import java.util.*;
import java.util.UUID;

interface ProductRepo {
    Optional<Product> findById(long id);
    Optional<Product> findByUuid(UUID id);
    List<Product> searchByKeyword(String keyword);
}

final class Product { /* fields omitted */ }

final class QueryService {
    private final ProductRepo repo;

    QueryService(ProductRepo repo) { this.repo = repo; }

    public Object handle(Object query) {
        return switch (query) {
            case null -> List.of(); // or throw IllegalArgumentException
            case String s -> {
                String q = s.trim();
                if (q.isEmpty()) yield List.of();
                // optional: try to parse UUID in string form
                try {
                    UUID uuid = UUID.fromString(q);
                    yield repo.findByUuid(uuid).map(List::of).orElseGet(List::of);
                } catch (IllegalArgumentException ignore) {
                    // not a UUID, treat as keyword
                    yield repo.searchByKeyword(q);
                }
            }
            case Integer i ->
repo.findById(i.longValue()).map(List::of).orElseGet(List::of);
            case Long l    -> repo.findById(l).map(List::of).orElseGet(List::of);
            case UUID u    -> repo.findByUuid(u).map(List::of).orElseGet(List::of);
```

```
            default       -> throw new IllegalArgumentException("Unsupported query
type: " + query.getClass());
        };
    }
}
```

If your inputs already arrive as the right static type (controller layer does parsing), the switch is even simpler:

```
public Object handleTyped(Object query) {
    return switch (query) {
        case String s -> repo.searchByKeyword(s.trim());
        case Long id  -> repo.findById(id).orElse(null);
        case UUID u   -> repo.findByUuid(u).orElse(null);
        case null     -> List.of();
        default       -> throw new IllegalArgumentException("Unsupported");
    };
}
```

Reasoning to interviewer:

- Type-pattern switch eliminates repetitive instanceof + cast pairs and makes dispatch intent crystal-clear.
- Ordering keeps the specific cases before the broad ones; since we don't have a Number arm here, there's no accidental capture.
- Treating String specially allows UUID-in-string to be recognized without a separate endpoint while still supporting plain keywords.
- Using switch expressions avoids fall-through by design; every arm yields a value, so it's hard to make control-flow mistakes.

Future scope:

- If query types become closed, use a sealed interface Query (e.g., ById, ByUuid, ByKeyword) and switch exhaustively with no default.
- Add rate-limit/authorization guards per arm (IDs might be privileged; keywords public).
- Add telemetry counters by matched arm to understand how users search and tune indexes accordingly.
- Move parsing of strings to a separate "query normalizer," so the switch stays purely on types.

Scenario 17: Accelerating climate simulation with the Vector API

We're crunching huge numeric arrays (temperatures, humidity, wind components) and doing lots of scaling, averages, and dot-like math. Scalar for-loops leave SIMD lanes idle. We want clean, portable SIMD in Java without JNI.

Answer:

I'd rewrite the hot loops with the Vector API so each iteration processes a whole "vector species" of lanes at once. At runtime the JIT maps this to AVX/AVX-512/NEON where available; otherwise it safely falls back. The shape is consistent across kernels:

- Pick the preferred species for the element type (FloatVector.SPECIES_PREFERRED or DoubleVector.SPECIES_PREFERRED).
- Process i += species.length() chunks: load → compute (mul/add/fma) → store.
- Handle the tail with a mask (indexInRange) so arrays of any length are correct.
- Use reductions (reduceLanes) for sums/means; use fma where it improves precision/speed.
- Keep arrays primitive and contiguous for cache-friendly streaming.

Code:

```
// Works on JDKs with the Vector API (package name may be jdk.incubator.vector
or jdk.vector, depending on JDK).
import jdk.incubator.vector.*; // swap to jdk.vector.* if on a finalized API

public final class ClimateKernels {
    private static final FloatVector.Species SPEC =
FloatVector.SPECIES_PREFERRED;

    // scale: T[i] = a * T[i]
    public static void scale(float a, float[] T) {
        int i = 0, ub = SPEC.loopBound(T.length);
        FloatVector va = FloatVector.broadcast(SPEC, a);
        for (; i < ub; i += SPEC.length()) {
            var v = FloatVector.fromArray(SPEC, T, i);
            v.mul(va).intoArray(T, i);
        }
        if (i < T.length) { // tail
```

```
            var m = SPEC.indexInRange(i, T.length);
            var v = FloatVector.fromArray(SPEC, T, i, m);
            v.mul(va).intoArray(T, i, m);
        }
    }

    // y = a*x + y (FMA where available) — useful for incremental updates
    public static void axpy(float a, float[] x, float[] y) {
        int i = 0, ub = Math.min(SPEC.loopBound(x.length),
SPEC.loopBound(y.length));
        var va = FloatVector.broadcast(SPEC, a);
        for (; i < ub; i += SPEC.length()) {
            var vx = FloatVector.fromArray(SPEC, x, i);
            var vy = FloatVector.fromArray(SPEC, y, i);
            vx.fma(va, vy).intoArray(y, i); // y = a*x + y
        }
        // tail
        for (; i < Math.min(x.length, y.length); i++) y[i] = a * x[i] + y[i];
    }

    // mean temperature over an array
    public static float mean(float[] T) {
        int i = 0, ub = SPEC.loopBound(T.length);
        var acc = FloatVector.zero(SPEC);
        for (; i < ub; i += SPEC.length()) {
            var v = FloatVector.fromArray(SPEC, T, i);
            acc = acc.add(v);
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < T.length; i++) sum += T[i]; // tail
        return sum / T.length;
    }

    // dot product (e.g., projection along a wind vector)
    public static float dot(float[] a, float[] b) {
        int n = Math.min(a.length, b.length);
        int i = 0, ub = SPEC.loopBound(n);
        var acc = FloatVector.zero(SPEC);
        for (; i < ub; i += SPEC.length()) {
            var va = FloatVector.fromArray(SPEC, a, i);
            var vb = FloatVector.fromArray(SPEC, b, i);
            acc = va.fma(vb, acc); // acc += va * vb
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < n; i++) sum += a[i] * b[i];
        return sum;
    }
}
```

Reasoning to interviewer:

- The species chooses the widest hardware vector at runtime, so the same code uses AVX-512 on one host and NEON on another with no changes.
- Loading with fromArray and storing with intoArray are lowered by the JIT to the right SIMD moves; we don’t micro-manage alignment.
- Masks make tails safe and branch-light; we avoid scalar loops except for a tiny remainder.
- Using fma improves both performance and numerical behavior where supported, which matters for long accumulation chains in climate models.
- Kernels remain pure Java, so we avoid JNI overhead and keep portability and maintainability high.

Future scope:

- Specialize precision per variable: use DoubleVector for sensitive aggregates, FloatVector for bulk fields.
- Tile computations (blocked loops) to keep hot tiles in L1/L2 cache, especially for 2D/3D grids.
- Batch multiple reductions to amortize memory bandwidth (e.g., compute sum and sum-of-squares in one pass).
- Add JMH benchmarks and per-CPU baselines; gate PRs on not regressing SIMD throughput.
- Combine with ForkJoin/structured concurrency to parallelize across tiles, each tile using SIMD inside.

Scenario 18: Securing deserialization in a payment-messaging service

We're deserializing messages between services. Untrusted payloads can try to instantiate dangerous classes or gadget chains. We want to allowlist only the DTOs we expect and reject everything else.

Answer:

I'd apply context-specific deserialization filters (Java's ObjectInputFilter) so each ObjectInputStream only allows approved classes/packages, with tight limits on depth/array sizes. This is enforced per-stream (closest to the boundary), optionally with a conservative global fallback.

- Build a small allowlist: exact classes and safe packages for our payment DTOs.
- Wrap every deserialization site with an ObjectInputStream that sets a filter via setObjectInputFilter.
- In the filter, inspect FilterInfo.serialClass() and decide ALLOWED or REJECTED. Also cap depth, arrayLength, and references to prevent resource abuse.
- Optionally set a restrictive global filter as a backstop and then relax per-stream where needed.

Code:

```
import java.io.*;
import java.util.Set;
import java.util.function.Predicate;

public final class Filters {
    private static final Set<String> ALLOWED = Set.of(
        "com.payments.dto.Transfer",
        "com.payments.dto.AuthRequest",
        "com.payments.dto.AuthResult",
        "java.util.ArrayList",
        "java.util.HashMap"
    );
    private static final String ALLOWED_PACKAGE_PREFIX =
"com.payments.dto.";

    // Per-stream contextual filter
    public static ObjectInputFilter perMessageFilter() {
```

```
        return info -> {
            // Basic structural guards
            if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
            if (info.arrayLength() != -1 && info.arrayLength() > 1_000_000) return
ObjectInputFilter.Status.REJECTED;
            if (info.references() != -1 && info.references() > 2_000_000) return
ObjectInputFilter.Status.REJECTED;

            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitive data

            String name = cl.getName();
            // Block proxies and JDK internal/sensitive types
            if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
            if (name.startsWith("java.lang.invoke.") || name.startsWith("java.security.")
||
                name.startsWith("java.nio.channels.") || name.startsWith("sun.") ||
name.startsWith("jdk.")) {
                return ObjectInputFilter.Status.REJECTED;
            }

            // Allowlist exact classes and our DTO package
            if (ALLOWED.contains(name) ||
name.startsWith(ALLOWED_PACKAGE_PREFIX)) {
                return ObjectInputFilter.Status.ALLOWED;
            }
            return ObjectInputFilter.Status.REJECTED; // default-deny
        };
    }

    // Optional global backstop (applies if no per-stream filter is set)
    public static void installGlobalBackstop() {
        ObjectInputFilter.Config.setSerialFilter(info -> {
            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
            String name = cl.getName();
            // Only dto package by default; everything else rejected
            return name.startsWith("com.payments.dto.")
                ? ObjectInputFilter.Status.ALLOWED
                : ObjectInputFilter.Status.REJECTED;
        });
    }
}

Usage at the boundary:


// At the message consumer boundary:
try (ObjectInputStream ois = new ObjectInputStream(in)) {
    ois.setObjectInputFilter(Filters.perMessageFilter());
```

```
    Object obj = ois.readObject(); // will be blocked if not allowlisted
    // cast to your DTO and proceed
}
```

Pattern-string filters are also viable for simple setups (combine limits and packages in one string):

```
// Example pattern: allow our DTOs and basic collections; cap depth/array size;
deny everything else.
ObjectInputFilter f = ObjectInputFilter.Config.createFilter(
    "maxdepth=20;maxarray=1000000;maxrefs=2000000;" +
    "com.payments.dto.*;java.util.*;!*"
);
ois.setObjectInputFilter(f);
```

Reasoning to interviewer:

- Context-specific filters are closest to the attack surface, so even if another part of the JVM is laxer, the boundary denies unexpected types.
- Default-deny is critical: only known DTOs pass. Everything else—including dynamic proxies and risky JDK types—is rejected.
- Structural limits (depth/arrays/references) cut off resource-exhaustion vectors, not just gadget chains.
- The global backstop helps if someone forgets to set a per-stream filter; still, I prefer explicit per-stream filters tied to the topic/endpoint's schema.

Future scope:

- Migrate wire format to schema-based (JSON with strict mappers, Protobuf/Avro); keep Java serialization only for trusted, intra-JVM hops.
- Add canary tests that attempt to deserialize known "bad" classes and assert rejection.
- Version DTOs explicitly and keep filters in config so rollout doesn't require a redeploy.
- Layer this with code-signing of messages or mTLS between services to reduce exposure even further.

Scenario 19: Simplifying application settings with Records

You've got immutable configuration objects (service name, version, environment). The current POJOs are stuffed with getters, constructors, equals/hashCode, and toString. You want the same immutability and clarity—without the boilerplate.

Answer:

- Records give me a canonical constructor, accessors, equals/hashCode, and toString out of the box.
- Immutability is the default: components are final. For collection fields, I'll defensively copy so the record stays effectively immutable.
- I'll validate in the compact constructor and provide small factory methods for loading from Properties/Maps or environment variables.
- For readability, I'll model environment as an enum and keep optional extras in an unmodifiable map.

Code:

```
import java.util.*;
import java.util.stream.Collectors;

public enum Env { DEV, STAGE, PROD }

public record ServiceConfig(
        String serviceName,
        String version,
        Env environment,
        Map<String, String> extras // optional key/values
) {
    // Compact ctor: validate + defensive copies
    public ServiceConfig {
        if (serviceName == null || serviceName.isBlank())
            throw new IllegalArgumentException("serviceName required");
        if (version == null || version.isBlank())
            throw new IllegalArgumentException("version required");
        if (environment == null)
            throw new IllegalArgumentException("environment required");

        // ensure extras is immutable and has trimmed keys/values
        extras = (extras == null) ? Map.of()
                : Collections.unmodifiableMap(
                    extras.entrySet().stream()
                        .collect(Collectors.toMap(
                            e -> e.getKey().trim(),
```

```
                        e -> e.getValue() == null ? "" : e.getValue().trim()
                    )));
    }

    // Convenience factories
    public static ServiceConfig fromEnv(Map<String, String> env) {
        String name = env.getOrDefault("SERVICE_NAME", "unknown");
        String ver  = env.getOrDefault("SERVICE_VERSION", "0.0.0");
        Env e = Env.valueOf(env.getOrDefault("SERVICE_ENV",
"DEV").toUpperCase(Locale.ROOT));
        Map<String,String> extras = env.entrySet().stream()
                .filter(it -> it.getKey().startsWith("CFG_"))
                .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
        return new ServiceConfig(name, ver, e, extras);
    }

    // “with” helpers for readability (records return a new instance)
    public ServiceConfig withVersion(String newVersion) {
        return new ServiceConfig(serviceName, newVersion, environment, extras);
    }
}

// Usage
class Demo {
    public static void main(String[] args) {
        ServiceConfig cfg = ServiceConfig.fromEnv(System.getenv());
        System.out.println(cfg); // nice toString for logs
        ServiceConfig bumped = cfg.withVersion("1.2.3");
        System.out.println(bumped);
    }
}
```

Reasoning to interviewer:

- Records encode the data-only intent and remove boilerplate while keeping immutability by default.
- The compact constructor centralizes validation and ensures collections are wrapped, so callers can’t mutate internal state.
- The enum for environment prevents typos like "prod " and makes config self-documenting.
- The “with”-style helpers communicate immutability clearly—each change returns a new instance.

Future scope:

- Bind records directly in Spring Boot/Micronaut config classes (constructor binding works great).
- Add Bean Validation annotations on components (e.g., @NotBlank, @Pattern) for consistent config checks.
- Define small value-type records (ServiceName, SemVer) for stronger domain constraints.
- Snapshoot configs for audit by serializing records to JSON (stable, human-readable toString/constructor).

Scenario 20: Thread-safe random numbers in an online gaming app

You need fast, independent random streams for dice rolls and loot tables across many threads. A single shared Random becomes a contention hotspot and harms fairness.

Answer:

- Java 17 introduced the java.util.random PRNG framework with a family of high-quality generators and interfaces (RandomGenerator, RandomGeneratorFactory).
- I'll give each thread its own generator instance—no shared state, no contention.
- I'll pick an algorithm designed for concurrency, e.g., an LXM generator (L64X128MixRandom, L128X256MixRandom) that supports jump/leap/split to create statistically independent substreams.
- For thread assignment, I'll either use a ThreadLocal<RandomGenerator> or generate per-thread streams by "jumping" from a root generator.
- Result: lock-free, cache-friendly, reproducible randomness across threads.

Code:

```
import java.util.random.*;
import java.util.concurrent.*;
import java.util.*;

public final class Rngs {
   // Choose a fast, high-quality generator available in Java 17
   private static final RandomGeneratorFactory<? extends RandomGenerator>
FACTORY =
         RandomGeneratorFactory.of("L64X128MixRandom");

   // Option 1: ThreadLocal RNGs (no contention)
   private static final ThreadLocal<RandomGenerator> TL =
         ThreadLocal.withInitial(() -> FACTORY.create(System.nanoTime()));

   public static int rollDice() {
      // uniform 1..6
      return TL.get().nextInt(1, 7);
   }

   public static boolean lootDrop(double p) {
      return TL.get().nextDouble() < p; // Bernoulli trial
```

```
    }

    // Option 2: Build independent streams via jump() (if supported)
    public static RandomGenerator streamForShard(int shardId) {
        RandomGenerator root = FACTORY.create(123456789L);
        if (root instanceof RandomGenerator.Jumpable j) {
            // Jump deterministically shardId times to create disjoint substreams
            RandomGenerator g = root;
            for (int i = 0; i < shardId; i++) g = j.jump();
            return g;
        }
        // Fallback: seed mixing per shard
        long seed = 0x9E3779B97F4A7C15L ^ (long) shardId *
0xBF58476D1CE4E5B9L;
        return FACTORY.create(seed);
    }

    // Example parallel usage
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(8);
        List<Callable<Integer>> tasks = new ArrayList<>();
        for (int t = 0; t < 8; t++) {
            tasks.add(() -> {
                int sum = 0;
                for (int i = 0; i < 1_000_000; i++) sum += rollDice();
                return sum;
            });
        }
        pool.invokeAll(tasks).forEach(f -> {
            try { System.out.println("sum=" + f.get()); } catch (Exception ignored) {}
        });
        pool.shutdown();
    }
}
```

Reasoning to interviewer:

- The new PRNG API separates algorithm from usage via RandomGenerator and factories, so I can pick a generator with the right speed/quality traits.
- Per-thread generators remove synchronization and false sharing; results scale with cores.
- LXM generators implement Jumpable/Leapable/SplittableGenerator, letting me derive non-overlapping substreams—great for sharding rooms/matches or reproducible simulations.

- Using bounded methods like nextInt(origin, bound) avoids modulo bias and clarifies intent.

Future scope:

- Centralize RNG selection behind a small interface so we can swap algorithms (e.g., L128X256MixRandom for higher period) without touching gameplay code.
- Seed strategy: seed each match/shard from a SecureRandom root, then split/jump for players to guarantee independence.
- Add statistical health checks (dieharder/PractRand offline) and property tests to catch accidental reuse or bias.
- For replays, persist the seed and jump counts so runs are fully reproducible across environments.

Scenario 21: GC tuning for a live sports-streaming analytics app

Live player stats and game events arrive continuously; you need sub-second insights. GC hiccups (stop-the-world pauses, promotion failures) will show up as UI stalls or delayed alerts. The goal is predictable low latency with high throughput.

Answer:

- Choose a low-latency collector by workload profile.
    - If you're on Java 21+, start with generational ZGC (very short pauses, concurrent compaction).
    - If you must stay on older LTS where ZGC isn't ideal, G1 with tight pause targets is the pragmatic default.
- Keep allocation predictable.
    - Flatten objects (prefer primitives/records), reuse buffers, avoid per-event temporary objects in hot paths.
    - Batch parse/transform events to amortize allocations.
- Size the heap for headroom.
    - Give enough space so the collector runs concurrently and calmly (avoid "last-minute" collections).
    - Pin Xms = Xmx in containers to avoid costly heap growth during peak.
- Keep young-gen healthy (for ephemeral events).
    - Ensure short-lived objects die young; reduce needless promotion (avoid long chains/queues of temporaries).
- Instrument and iterate.
    - Turn on GC unified logging + JFR; watch allocation rate, pause percentiles, promotion failures, remembered-set/scan times.

Code:

Minimal ZGC setup (Java 21+, low latency):

```
# Typical ZGC flags for steady sub-10ms pauses at high throughput
java \
  -Xms6g -Xmx6g \                    # fixed heap for stable ergonomics
  -XX:+UseZGC \                      # generational by default on JDK 21+
  -XX:SoftMaxHeapSize=5g \           # ZGC will try to stay near soft max, leaving
headroom
  -Xlog:gc*:file=gc.log:time,uptime,tags,level \
  -XX:+AlwaysPreTouch \              # avoid first-touch page faults (containers)
```

```
  -XX:+DisableExplicitGC \            # ignore accidental System.gc()
  -XX:+ZProactive \                   # background work to keep ahead of demand
  -jar analytics.jar
```

Tuned G1 setup (if ZGC isn't an option):

```
java \
  -Xms6g -Xmx6g \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=50 \           # start with 50ms target; tighten if headroom
allows
  -XX:InitiatingHeapOccupancyPercent=30 \  # start marking earlier to smooth
cycles
  -XX:G1NewSizePercent=30 -XX:G1MaxNewSizePercent=60 \  # roomy young
gen for ephemeral events
  -XX:SurvivorRatio=8 -XX:TargetSurvivorRatio=80 \      # reduce premature
promotion
  -Xlog:gc*:file=gc.log:time,uptime,tags,level \
  -XX:+AlwaysPreTouch -XX:+DisableExplicitGC \
  -jar analytics.jar
```

App-level hot-path hygiene (example):

```
// Reuse per-thread builders to avoid per-event garbage
final class Builders {
  private static final ThreadLocal<StringBuilder> SB =
      ThreadLocal.withInitial(() -> new StringBuilder(256));
  static String formatEvent(Event e) {
    var sb = SB.get(); sb.setLength(0);

sb.append(e.playerId()).append(',').append(e.metric()).append(',').append(e.value()
);
    return sb.toString(); // one allocation (the final String)
  }
}
```

Reasoning to interviewer:

- ZGC (generational on JDK 21+) gives concurrent marking *and* compaction with tiny pauses, ideal for streams where tail latencies matter more than absolute throughput.
- Fixing Xms=Xmx avoids in-flight heap resizing, which can destabilize GC timing under load.
- With G1, early concurrent cycles and a large young gen keep most event garbage in the young space; fewer promotions → fewer costly mixed collections.

- GC logging + JFR lets me correlate allocation spikes with pause percentiles; I'll tune pause targets, IHOP, and young/old sizing based on real allocation rates, not guesses.
- Application changes (buffer reuse, avoiding string/boxing churn) often buy more latency headroom than micro-tweaking GC flags.

Future scope:

- Partition processing by key (playerId/team) to keep lifetimes localized and improve object locality (better cache → fewer GC touches).
- Use the Vector API for numeric kernels (rolling averages, z-scores) to reduce CPU time per event, indirectly easing GC pressure.
- Add back-pressure when GC time > budget (drop to sampling mode or coarser aggregates temporarily).
- Container awareness: set -XX:MaxRAMPercentage sensibly and reserve CPU so concurrent GC has cycles during peaks.

Scenario 22: Simplifying heterogeneous payload handling with pattern matching

Your IoT hub receives mixed payloads: numeric sensor IDs, string alerts, and structured event objects. Current logic repeats instanceof checks followed by casts. You want the same routing but safer and cleaner.

Answer:

I'll use instanceof pattern matching (Java 17+) so each check simultaneously binds the typed variable. That removes duplicate casts, narrows scope, and makes the intent of each branch obvious. Where it fits, I'll also use a sealed hierarchy for structured events to get compiler help on exhaustiveness.

Code:

```
import java.time.Instant;

public record SensorEvent(int id, String type, double reading, Instant ts) {}

public final class IngestRouter {

  public void handle(Object payload) {
    if (payload == null) {
      onMalformed("null payload"); return;
    }

    if (payload instanceof Integer sensorId) {
      // numeric sensor id: fetch latest state
      routeToStateLookup(sensorId);
      return;
    }

    if (payload instanceof String msg) {
      // string alert: normalize + route
      String normalized = msg.trim();
      if (normalized.startsWith("ALERT:")) routeToAlerts(normalized);
      else routeToLogs(normalized);
      return;
    }

    if (payload instanceof SensorEvent evt) {
      // structured domain event
      routeToAnalytics(evt);
      return;
    }

    // fallback
```

```
    onMalformed("unsupported type: " + payload.getClass().getName());
  }

  private void routeToStateLookup(int sensorId) { /* ... */ }
  private void routeToAlerts(String alert) { /* ... */ }
  private void routeToLogs(String text) { /* ... */ }
  private void routeToAnalytics(SensorEvent evt) { /* ... */ }
  private void onMalformed(String reason) { /* ... */ }
}
```

If you're on Java 21, a type-pattern switch further clarifies intent:

```
public void handleSwitch(Object payload) {
  switch (payload) {
    case null                  -> onMalformed("null payload");
    case Integer sensorId      -> routeToStateLookup(sensorId);
    case String msg            -> {
      String s = msg.trim();
      if (s.startsWith("ALERT:")) routeToAlerts(s); else routeToLogs(s);
    }
    case SensorEvent evt       -> routeToAnalytics(evt);
    default                    -> onMalformed("unsupported type: " +
payload.getClass().getName());
  }
}
```

Reasoning to interviewer:

- Pattern matching binds and narrows in the same expression (payload instanceof SensorEvent evt), eliminating cast duplication and reducing the chance of ClassCastException after refactors.
- Branches read like a router by data shape, not by manual casting ceremony; easier to review and extend.
- With a switch on type patterns, there's no fall-through, and it's straightforward to add a new arm for a new payload type later.
- Using a domain record (SensorEvent) makes downstream code explicit and self-documenting compared to loose maps/tuples.

Future scope:

- Model structured payloads with a sealed interface (e.g., Payload permits SensorEvent, Alert, Command) and switch exhaustively—no default needed.

- Push boundary parsing into a “normalizer” (JSON → records) so the router only switches on types, not raw strings.
- Add metrics per arm to see traffic mix and prioritize optimizations (e.g., bulk-path for SensorEvent spikes).
- If alerts grow variants, use nested pattern matching (switch inside the String arm) or promote them to typed records.

Scenario 23: Ensuring thread-safe account updates in a trading platform

Trades settle continuously; multiple workers credit/debit the same account objects at the same time. Without care, two updates can interleave and lose money. We need correctness (no lost updates), visibility (readers see latest), and deadlock-free transfers.

Answer:

- The Java Memory Model guarantees visibility and ordering via “happens-before”: a write that is followed by a read with the proper synchronization (synchronized/locks, volatile, atomics) becomes visible to other threads. I’ll rely on these edges to make every balance update linearizable.
- For single-account updates (credit/debit), I’ll use an AtomicLong with CAS to avoid lost updates and ensure immediate visibility.
- For cross-account operations (transfer, settle fee + balance), I’ll take per-account locks in a global, deterministic order to avoid deadlocks, update both, and publish a version stamp for readers.
- Readers that need very low latency can do an optimistic read (StampedLock) and retry if a concurrent write occurred; otherwise a simple atomic get() works.

Code:

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.ReentrantLock;

final class Account {
   // AtomicLong gives volatile visibility + CAS atomicity
   private final AtomicLong balanceMicros = new AtomicLong(0);
   // Narrow lock per account for multi-field or multi-account invariants
   final ReentrantLock lock = new ReentrantLock();

   long get() { return balanceMicros.get(); }          // visible to all threads
   void add(long delta) {                              // single-account update
      long prev, next;
      do {
         prev = balanceMicros.get();
         next = prev + delta;
         if (next < 0) throw new IllegalStateException("insufficient");
      } while (!balanceMicros.compareAndSet(prev, next)); // retry on contention
```

```
    }
}

final class Ledger {
    private final ConcurrentHashMap<Long, Account> accounts = new
ConcurrentHashMap<>();
    private Account acc(long id) {
        return accounts.computeIfAbsent(id, k -> new Account());
    }

    // credit/debit that touches one account
    void settle(long accountId, long deltaMicros) {
        acc(accountId).add(deltaMicros);
    }

    // transfer between two accounts: lock ordering avoids deadlock
    void transfer(long fromId, long toId, long amountMicros) {
        if (fromId == toId) return;
        Account a = acc(fromId), b = acc(toId);

        Account first = (fromId < toId) ? a : b;
        Account second = (first == a) ? b : a;

        first.lock.lock();
        try {
            second.lock.lock();
            try {
                a.add(-amountMicros);
                b.add(+amountMicros);
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }
}
```

Reasoning to interviewer:

- The JMM gives us the tools:
  - synchronized/locks establish happens-before on lock release → acquire;
  - volatile (and atomics) establish happens-before on write → read and prevent reordering;
  - atomic classes add CAS so updates aren't lost under races.
- Using AtomicLong for single-account updates avoids coarse locking, keeps throughput high, and every successful CAS publishes the new value (visibility).
- For multi-account invariants, we can't rely on two independent CAS operations; we take per-account locks in a globally ordered manner (by account id) to prevent cycles → no deadlocks.
- Reads use get() for fresh values; if we had multi-field snapshots, we'd either read under the same lock or use a versioned immutable snapshot and retry on version change (optimistic read).
- This design keeps the common path (single account credit/debit) lock-free, and the rare path (transfer) correct and deadlock-free.

Future scope:

- Introduce a per-account single-writer executor (mailbox) for ultra-hot accounts; workers send messages, maintaining serial order without locks.
- Add idempotency keys on events so retries never double-apply a mutation.
- Persist with DB transactions that mirror JVM ordering (e.g., lock rows in ascending id) so JVM and DB don't fight.
- Emit audit snapshots via a Flow.Publisher after each successful update for downstream risk systems.

Scenario 24: Breaking dependency cycles in a modular travel booking system

Flights, Hotels, and Billing were split into modules but started referencing each other directly (Flights ↔ Billing ↔ Hotels), creating cycles that complicate builds and cause runtime classpath/module path issues. We need to invert dependencies.

Answer:

- Split each area into a tiny API module and a separate implementation module. Other modules depend only on the APIs, never on each other's implementations.
- Move shared DTOs and value types to a travel.common API.
- Wire implementations with JPMS services (uses/provides) so orchestration depends on abstractions, not concrete classes. This breaks cycles because APIs don't depend back on implementations.

Code:

API modules (no cycles; each exports only its contracts):

```
// travel.common
module com.travel.common {
    exports com.travel.common; // Ids, Money, Dates, Errors
}

// flights.api
module com.travel.flights.api {
    requires transitive com.travel.common;
    exports com.travel.flights.api;          // FlightSearch, FlightOffer DTOs
    uses com.travel.flights.api.FlightService; // optional: consumers can discover
}

// hotels.api
module com.travel.hotels.api {
    requires transitive com.travel.common;
    exports com.travel.hotels.api;
    uses com.travel.hotels.api.HotelService;
}

// billing.api
module com.travel.billing.api {
    requires transitive com.travel.common;
    exports com.travel.billing.api;          // PaymentService, Invoice DTOs
    uses com.travel.billing.api.PaymentService;
}
```

Implementation modules (no exports; they only provide services):

```
// flights.impl
module com.travel.flights.impl {
    requires com.travel.flights.api;
    requires com.travel.common;
    // do NOT depend on billing.impl or hotels.impl
    provides com.travel.flights.api.FlightService
        with com.travel.flights.impl.DefaultFlightService;
}

// hotels.impl
module com.travel.hotels.impl {
    requires com.travel.hotels.api;
    requires com.travel.common;
    provides com.travel.hotels.api.HotelService
        with com.travel.hotels.impl.DefaultHotelService;
}

// billing.impl
module com.travel.billing.impl {
    requires com.travel.billing.api;
    requires com.travel.common;
    provides com.travel.billing.api.PaymentService
        with com.travel.billing.impl.StripePaymentService;
}
```

Composition/orchestrator module (depends on APIs only, discovers impls via ServiceLoader):

```
module com.travel.app {
    requires com.travel.common;
    requires com.travel.flights.api;
    requires com.travel.hotels.api;
    requires com.travel.billing.api;

    uses com.travel.flights.api.FlightService;
    uses com.travel.hotels.api.HotelService;
    uses com.travel.billing.api.PaymentService;
}
```

Example usage in the app:

```
var flights = java.util.ServiceLoader.load(com.travel.flights.api.FlightService.class)
                                     .findFirst().orElseThrow();
var hotels  = java.util.ServiceLoader.load(com.travel.hotels.api.HotelService.class)
                                     .findFirst().orElseThrow();
```

```
var pay     =
java.util.ServiceLoader.load(com.travel.billing.api.PaymentService.class)
                                .findFirst().orElseThrow();
```

// Orchestrate a trip without any module depending on another module's impl

Reasoning to interviewer:

- Cycles disappear when each feature depends only on stable API surfaces; implementations never import each other, so there’s no edge to close a cycle.
- requires transitive on API modules re-exports common to their consumers, keeping dependency graphs clean and avoiding “reach through” to impls.
- provides/uses in JPMS formalizes inversion of control: the app consumes services; providers register at module boundaries. This keeps builds simple and swaps implementations without graph changes.
- Implementation modules export nothing, so accidental coupling to internal classes is impossible at compile time.

Future scope:

- Add CI checks (jdeps, Revapi) to fail builds on new cycles or accidental exports.
- If two APIs genuinely need to collaborate, extract a third, smaller API (e.g., pricing.api) that both depend on, instead of pointing them at each other.
- Produce a jlink runtime image with only API + chosen impl modules for smaller, faster deployments.
- Version APIs and enforce semver in CI to keep the graph stable as teams evolve features independently.

Scenario 25: Using the Vector API in recommendation engines

A recommendation system needs to score user–item similarity (cosine/L2 distance) across thousands of vectors fast. Pure for-loops leave SIMD units idle and can't hit real-time SLAs.

Answer:

I'd rewrite hot numeric kernels (dot product, L2 norm, axpy) using the Java Vector API so each loop iteration operates on a whole "vector species" of lanes in parallel. At runtime the JIT maps these ops to AVX/AVX-512/NEON on the host CPU. The pattern is: choose a species for your element type, load lanes from arrays, compute with vector ops (mul, add, fma), reduce lanes, and mask the tail safely.

- Pick species once per kernel (FloatVector/DoubleVector with SPECIES_PREFERRED).
- Process i += species.length() stride; handle remainder with a mask (indexInRange).
- Use fma where available for speed and better numeric behavior.
- Keep vectors in primitive arrays laid out row-major for cache-friendly streaming.

Code:

```
// Requires a JDK that exposes the Vector API (incubator in many builds).
// Switch to jdk.vector.* if using the finalized package.
import jdk.incubator.vector.*;

public final class RecoMath {
    private static final FloatVector.Species S =
FloatVector.SPECIES_PREFERRED;

    // dot(u, v)
    public static float dot(float[] u, float[] v) {
        int n = Math.min(u.length, v.length);
        int i = 0, ub = S.loopBound(n);
        var acc = FloatVector.zero(S);
        for (; i < ub; i += S.length()) {
            var a = FloatVector.fromArray(S, u, i);
            var b = FloatVector.fromArray(S, v, i);
            acc = a.fma(b, acc);          // acc += a*b
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < n; i++) sum += u[i] * v[i]; // tail
```

```
        return sum;
    }

    // ||u||_2
    public static float norm2(float[] u) {
        int i = 0, ub = S.loopBound(u.length);
        var acc = FloatVector.zero(S);
        for (; i < ub; i += S.length()) {
            var a = FloatVector.fromArray(S, u, i);
            acc = a.fma(a, acc);          // acc += a*a
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < u.length; i++) sum += u[i] * u[i];
        return (float) Math.sqrt(sum);
    }

    // cosine(u, v) = dot(u,v) / (||u|| * ||v||)
    public static float cosine(float[] u, float[] v) {
        float d = dot(u, v);
        float nu = norm2(u), nv = norm2(v);
        return (nu == 0f || nv == 0f) ? 0f : d / (nu * nv);
    }

    // y = a*x + y (useful for SGD/updates)
    public static void axpy(float a, float[] x, float[] y) {
        int n = Math.min(x.length, y.length);
        int i = 0, ub = S.loopBound(n);
        var va = FloatVector.broadcast(S, a);
        for (; i < ub; i += S.length()) {
            var vx = FloatVector.fromArray(S, x, i);
            var vy = FloatVector.fromArray(S, y, i);
            vx.fma(va, vy).intoArray(y, i);
        }
        for (; i < n; i++) y[i] = a * x[i] + y[i];
    }
}
```

Reasoning to interviewer:

- SPECIES_PREFERRED picks the widest lanes supported by the CPU at runtime (portable across AVX2/AVX-512/NEON) and falls back cleanly if narrower.
- Vector loads/stores (fromArray/intoArray) are lowered by the JIT to efficient SIMD moves; we don't micromanage alignment.
- Masked tails keep code branch-light and correct for any length.

- Cosine similarity and dot are memory-bandwidth bound; reducing loop overhead and using FMA helps us get closer to peak throughput.

Future scope:

- Promote sensitive accumulations (e.g., very long vectors) to DoubleVector for numeric stability while keeping features as float.
- Tile batches (compute several dots at once) to improve cache locality when scoring a user against many items.
- Parallelize at a higher level with per-shard tasks; keep SIMD inside each shard for best CPU usage.
- Add JMH baselines in CI; track speedups per CPU family to catch regressions across JDK updates.

Scenario 26: Preventing insecure deserialization in a loan-processing app

Serialized messages flow between services. An attacker could smuggle gadget classes to trigger arbitrary code or resource exhaustion during deserialization. We want strict allowlisting at the boundary.

Answer:

I'd enforce context-specific deserialization filters (ObjectInputFilter) on every ObjectInputStream that reads from the wire. The filter allows only approved DTO packages/classes and caps structure (depth/array size/references). Everything else is rejected. Optionally, set a restrictive global backstop.

- Maintain a small allowlist (loan DTOs + basic collections).
- Apply setObjectInputFilter(...) per stream, as close to the network boundary as possible.
- Deny dynamic proxies, JDK internals, and any class outside approved namespaces.
- Add structural limits to block "megabomb" payloads even if types are allowed.

Code:

```
import java.io.*;
import java.util.Set;

public final class LoanFilters {
    private static final String DTO_PKG = "com.bank.loan.dto.";
    private static final Set<String> EXTRA_OK = Set.of(
        "java.util.ArrayList", "java.util.HashMap"
    );

    public static ObjectInputFilter inboundFilter() {
        return info -> {
            // structural limits
            if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
            if (info.arrayLength() != -1 && info.arrayLength() > 500_000)
                return ObjectInputFilter.Status.REJECTED;
            if (info.references() != -1 && info.references() > 1_000_000)
                return ObjectInputFilter.Status.REJECTED;

            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitives
```

```
            String name = cl.getName();

            // blanket denials
            if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
            if (name.startsWith("java.lang.invoke.") || name.startsWith("sun.")
                    || name.startsWith("jdk.") || name.startsWith("java.rmi.")) {
                return ObjectInputFilter.Status.REJECTED;
            }

            // allowlist
            if (name.startsWith(DTO_PKG) || EXTRA_OK.contains(name)) {
                return ObjectInputFilter.Status.ALLOWED;
            }
            return ObjectInputFilter.Status.REJECTED; // default deny
        };
    }

    // Optional: global backstop if a site forgets to set a filter
    public static void installGlobalBackstop() {
        ObjectInputFilter.Config.setSerialFilter(info -> {
            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
            return cl.getName().startsWith(DTO_PKG)
                ? ObjectInputFilter.Status.ALLOWED
                : ObjectInputFilter.Status.REJECTED;
        });
    }
}
```

Usage at the consumer boundary:

```
// inside your message handler
try (ObjectInputStream ois = new ObjectInputStream(incomingBytes)) {
    ois.setObjectInputFilter(LoanFilters.inboundFilter());
    Object obj = ois.readObject(); // blocked unless allowlisted + within limits
    // cast to expected DTO and proceed
}

Alternative with a pattern string (simple deployments):


// allow only our DTO pkg + basic collections; cap depth/array/refs; deny
everything else
ObjectInputFilter pattern = ObjectInputFilter.Config.createFilter(
    "maxdepth=20;maxarray=500000;maxrefs=1000000;" +
    "com.bank.loan.dto.*;java.util.*;!*"
);
ois.setObjectInputFilter(pattern);
```

Reasoning to interviewer:

- Context filters apply closest to the attack surface and are enforced before object graphs are instantiated, stopping gadget chains early.
- Default-deny with a narrow allowlist is the safest stance; structural caps thwart resource-exhaustion even on allowed classes.
- Keeping filters per topic/endpoint lets each consumer define exactly which DTOs are valid there, limiting blast radius.
- A global backstop covers gaps if a code path forgets to set its own filter.

Future scope:

- Prefer schema-based formats (JSON with strict mappers, Avro/Protobuf); use Java serialization only for trusted intra-JVM hops.
- Externalize allowlists in config so rollout of new DTOs doesn’t require redeploy; add CI tests that attempt to deserialize disallowed classes.
- Sign messages or enforce mTLS between services to reduce tampering risk upstream.
- Track filter decisions and rejections with metrics/logs for early detection of probing attempts.

Scenario 27: Enforcing immutability in feature flag configuration

Feature flags (experiment id, variant rules, rollout %) must never change after publish. The current POJOs are verbose and it's easy to accidentally add setters or leak mutable collections.

Answer:

- Records encode "data-only + immutable" by default: components are final, there are no setters, and the canonical constructor is generated.
- I'll validate invariants in the compact constructor and defensively copy any incoming collections so the record stays effectively immutable.
- I'll model small value types (e.g., ExperimentId, Percentage) as records/enums for clarity and stronger typing.
- Publishing returns a new record instance (functional style), so there's no hidden mutation.

Code:

```
import java.time.Instant;
import java.util.List;
import java.util.Map;

public enum Env { DEV, STAGE, PROD }
public enum Variant { CONTROL, TREATMENT_A, TREATMENT_B }

public record ExperimentId(String value) {
    public ExperimentId {
        if (value == null || value.isBlank()) throw new IllegalArgumentException("id
required");
    }
}

public record Percentage(int value) {
    public Percentage {
        if (value < 0 || value > 100) throw new IllegalArgumentException("0..100");
    }
}

public record FeatureFlag(
        ExperimentId id,
        String featureKey,
        Env environment,
        Map<Variant, Percentage> allocations, // e.g., CONTROL=50,
TREATMENT_A=50
```

```
        List<String> includeUserIds,
        Instant publishedAt
) {
    public FeatureFlag {
        if (featureKey == null || featureKey.isBlank()) throw new
IllegalArgumentException("featureKey required");
        if (environment == null) throw new IllegalArgumentException("env required");
        if (allocations == null || allocations.isEmpty()) throw new
IllegalArgumentException("allocations required");
        int sum = allocations.values().stream().mapToInt(Percentage::value).sum();
        if (sum != 100) throw new IllegalArgumentException("allocations must sum to
100");
        // defensive copies for shallow immutability
        allocations = Map.copyOf(allocations);
        includeUserIds = includeUserIds == null ? List.of() :
List.copyOf(includeUserIds);
        if (publishedAt == null) throw new IllegalArgumentException("publishedAt
required");
    }

    // convenience: return a new flag for another environment (immutability
preserved)
    public FeatureFlag forEnv(Env env) {
        return new FeatureFlag(id, featureKey, env, allocations, includeUserIds,
publishedAt);
    }
}
```

Reasoning to interviewer:

- Records kill the “accidental mutability” vector: no setters and final components by construction.
- The compact constructor centralizes validation (sum=100, required fields) and wraps collections with Map.copyOf/List.copyOf, so callers can’t mutate the internals.
- Using tiny value types (ExperimentId, Percentage) reduces primitive obsession and prevents invalid states from spreading.
- All “changes” create new instances, which plays nicely with audit logs, rollbacks, and cache keys.

Future scope:

- Add Bean Validation annotations on components (e.g., @NotBlank, custom @SumToOneHundred) for standardized checks.

- Persist snapshots as JSON; records serialize cleanly via constructor-based mappers.
- Introduce a signed “manifest” record for published configs (hash + signature) to detect tampering.
- Provide a diff utility between two FeatureFlag records for rollout reviews and approval workflows.

Scenario 28: Scalable random number generation for online multiplayer battles

Many threads need randomness for moves, spawns, and rewards. A single shared Random causes contention and unpredictably slows hot paths.

Answer:

- Java 17's PRNG framework (java.util.random) exposes RandomGenerator + factories and modern algorithms (LXM, Xoshiro, etc.).
- I'll give each thread its own generator instance—via ThreadLocal or by deriving independent substreams using Jumpable/SplittableGenerator—so there's no shared state or locks.
- Choose a fast, high-quality generator like L64X128MixRandom (or L128X256MixRandom for longer period).
- Provide bounded APIs (nextInt(origin, bound)) to avoid modulo bias and keep call sites tidy.

Code:

```
import java.util.random.*;

public final class Rng {
    private static final RandomGeneratorFactory<? extends RandomGenerator> F =
            RandomGeneratorFactory.of("L64X128MixRandom");

    // Per-thread RNG: no contention
    private static final ThreadLocal<RandomGenerator> TL =
            ThreadLocal.withInitial(() -> F.create(seed()));

    private static long seed() {
        // mix time + thread id; for reproducible matches, inject a fixed seed instead
        return System.nanoTime() ^ Thread.currentThread().getId();
    }

    public static int roll(int fromInclusive, int toExclusive) {
        return TL.get().nextInt(fromInclusive, toExclusive);
    }

    public static boolean chance(double p) {
        return TL.get().nextDouble() < p;
    }

    // Deterministic independent stream for a match/shard
```

```
    public static RandomGenerator forMatch(long matchSeed, int playerIndex) {
        RandomGenerator base = F.create(matchSeed);
        if (base instanceof RandomGenerator.Jumpable j) {
            RandomGenerator g = base;
            for (int i = 0; i < playerIndex; i++) g = j.jump(); // disjoint substreams
            return g;
        }
        // Fallback: mix seeds
        return F.create(matchSeed ^ (playerIndex * 0x9E3779B97F4A7C15L));
    }
}
```

Reasoning to interviewer:

- Per-thread generators eliminate contention entirely and scale linearly with cores.
- LXM generators support jump() to carve deterministic, non-overlapping substreams—perfect for per-match or per-player reproducibility and fairness.
- The new API decouples algorithm choice from usage; swapping to a different generator is a one-liner.
- Using bounded methods (nextInt(a,b)) avoids biased math and keeps logic easy to audit.

Future scope:

- Centralize RNG policy (algorithm, seeding) behind a small interface; log seeds for replay/debug.
- Use SecureRandom only for security-sensitive flows (e.g., anti-cheat tokens), keep gameplay on fast PRNGs.
- Add property-based tests to catch accidental stream reuse; monitor entropy consumption per match.
- For large simulations, combine thread partitioning (one RNG per worker) with SIMD math for physics/AI.

Scenario 29: Enforcing API boundaries in healthcare microservices

Patients, Billing, and Reporting must expose stable APIs while keeping implementation details hidden to prevent accidental coupling and unsafe access.

Answer:

- Split each domain into an API module and an implementation module. APIs export only contract packages (interfaces/DTOs) and may declare uses for service discovery. Implementations depend on their API and provide services.
- Shared value types go into a tiny health.common module.
- Consumers depend only on API modules; composition uses ServiceLoader to wire implementations, eliminating impl-to-impl dependencies.

Code:

API modules:

```
// health.common
module com.health.common {
    exports com.health.common; // PatientId, Money, DateRange, Errors
}

// patients.api
module com.health.patients.api {
    requires transitive com.health.common;
    exports com.health.patients.api;              // PatientService, DTOs
    uses com.health.patients.api.PatientService;  // optional: discovery
}

// billing.api
module com.health.billing.api {
    requires transitive com.health.common;
    exports com.health.billing.api;               // PaymentService, Invoice DTOs
    uses com.health.billing.api.PaymentService;
}

// reporting.api
module com.health.reporting.api {
    requires transitive com.health.common;
    requires transitive com.health.patients.api;  // consumes patient summaries
    requires transitive com.health.billing.api;   // consumes billing summaries
    exports com.health.reporting.api;             // ReportService interface
    uses com.health.reporting.api.ReportService;
}
```

Implementation modules (hide internals; only provide):

```
// patients.impl
module com.health.patients.impl {
    requires com.health.patients.api;
    requires com.health.common;
    provides com.health.patients.api.PatientService
        with com.health.patients.impl.SqlPatientService;
    // no exports: internals are inaccessible
}

// billing.impl
module com.health.billing.impl {
    requires com.health.billing.api;
    requires com.health.common;
    provides com.health.billing.api.PaymentService
        with com.health.billing.impl.StripePaymentService;
}

// reporting.impl
module com.health.reporting.impl {
    requires com.health.reporting.api;
    requires com.health.patients.api;
    requires com.health.billing.api;
    requires com.health.common;
    provides com.health.reporting.api.ReportService
        with com.health.reporting.impl.DefaultReportService;
}
```

Composition (gateway/app) depends only on APIs, discovers providers at runtime:

```
module com.health.app {
    requires com.health.common;
    requires com.health.patients.api;
    requires com.health.billing.api;
    requires com.health.reporting.api;

    uses com.health.patients.api.PatientService;
    uses com.health.billing.api.PaymentService;
    uses com.health.reporting.api.ReportService;
}
```

Reasoning to interviewer:

- JPMS hides everything by default; only explicitly exported packages are reachable, so impl classes can’t leak across service boundaries.
- Splitting API vs impl and wiring with uses/provides breaks dependency cycles and lets us swap implementations without graph changes.
- requires transitive on API modules forwards common to consumers, keeping dependency declarations clean.
- With no exports in *.impl, accidental imports of internal classes fail at compile time—exactly the protection we want in regulated healthcare systems.

Future scope:

- Gate CI with jdeps/Revapi: fail builds on new exports, illegal accesses, or breaking API changes.
- Use opens narrowly (to specific frameworks like Jackson/Hibernate) instead of blanket openness.
- Produce a jlink image with only API + selected impls to shrink runtime and attack surface.
- Version API modules and enforce semantic versioning with automated compatibility checks.

Scenario 30: Replacing verbose DTOs in a logistics platform

In a logistics system, DTOs like Parcel, Shipment, and Invoice are just data carriers. Hand-written getters, equals/hashCode, and toString add noise and bugs. We want concise, immutable DTOs that still validate inputs.

Answer:

- I'd migrate DTOs to records. Records generate constructor, accessors, equals/hashCode, and toString automatically, and their components are final.
- I'll keep invariants in the compact constructor and defensively copy collections so objects stay effectively immutable.
- For readability and safety, I'd introduce tiny value types (TrackingId, Money) as records and enums (Currency) where appropriate.
- JSON mappers (Jackson 2.12+/Spring Boot 3+) bind cleanly to records via the canonical constructor; JPA entities remain classes while projections/DTOs become records.

Code:

```
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.Map;

public record TrackingId(String value) {
    public TrackingId {
        if (value == null || value.isBlank()) throw new
IllegalArgumentException("tracking id required");
    }
}
public record Money(BigDecimal amount, String currency) {
    public Money {
        if (amount == null || amount.signum() < 0) throw new
IllegalArgumentException("amount >= 0");
        if (currency == null || currency.isBlank()) throw new
IllegalArgumentException("currency required");
    }
}

public record ParcelDTO(TrackingId trackingId,
                        double weightKg,
                        Map<String,String> attributes) {
    public ParcelDTO {
```

```
        if (weightKg <= 0) throw new IllegalArgumentException("weight must be
positive");
        attributes = (attributes == null) ? Map.of() : Map.copyOf(attributes); //
defensive copy
    }
}

public record ShipmentDTO(String shipmentId,
                          List<ParcelDTO> parcels,
                          Instant createdAt) {
    public ShipmentDTO {
        if (shipmentId == null || shipmentId.isBlank()) throw new
IllegalArgumentException("shipmentId required");
        if (createdAt == null) throw new IllegalArgumentException("createdAt
required");
        parcels = (parcels == null) ? List.of() : List.copyOf(parcels);
    }
}

public record InvoiceDTO(String invoiceNo, Money total, List<TrackingId> lines) {
    public InvoiceDTO {
        if (invoiceNo == null || invoiceNo.isBlank()) throw new
IllegalArgumentException("invoiceNo required");
        if (total == null) throw new IllegalArgumentException("total required");
        lines = (lines == null) ? List.of() : List.copyOf(lines);
    }
}
```

Reasoning to interviewer:

- Records remove boilerplate while making “data-only” intent explicit; fewer moving parts means fewer bugs.
- Compact constructors centralize validation and defensive copying, so DTOs stay immutable even if callers hand us mutable inputs.
- Small value types (TrackingId, Money) reduce primitive obsession and keep rules close to the data.
- We keep persistence models (entities) as classes, but everything crossing service boundaries (REST, messaging) can be a record—leaner and safer.

Future scope:

- Add Bean Validation on components (@NotBlank, @Positive) and let the framework enforce constraints automatically.

- Use record-based projections in Spring Data/JPA queries to skip entity materialization.
- Generate OpenAPI/JSON Schema directly from record DTOs for contract checks.
- Create mapping utilities (entity → record) and contract tests to detect field drift early.

Scenario 31: Accelerating graph algorithms with SIMD

Graph workloads have irregular memory access, but many inner loops still do uniform math: relax edges, accumulate centrality scores, normalize ranks. We can vectorize those kernels with the Java Vector API to use SIMD lanes.

Answer:

- Store the graph in a CSR-like layout (row offsets + neighbor arrays + weights) so edge attributes are contiguous in memory.
- Use the Vector API (FloatVector/DoubleVector with SPECIES_PREFERRED) to process edges in chunks: load weights and source/destination values, compute candidates (e.g., d[src] + w), compare to d[dst], and write back via masked blends.
- Handle remainder elements with masks; keep atomic/conflict-prone updates out of the vectorized kernel (use two-phase relax: compute best per-dst in a scratch array, then commit).
- Similar patterns speed up PageRank (rank = α·M·rank + (1−α)/N) and centrality accumulations (vectorized multiplies/adds and reductions).

Code:

```
// CSR graph: rowPtr[v..v+1) gives the edge range for vertex v
import jdk.incubator.vector.*;

public final class GraphSimd {
    private static final FloatVector.Species S =
FloatVector.SPECIES_PREFERRED;

    // Relax edges of a single vertex v (Bellman-Ford-like step), two-phase into
scratch
    // Inputs: dist[], edgeTo[], edgeWeight[], rowPtr[]
    public static void relaxVertex(int v, float[] dist,
                                   int[] edgeTo, float[] w,
                                   int[] rowPtr,
                                   float[] scratchBest /* length = dist.length, init with +INF */) {
        int start = rowPtr[v], end = rowPtr[v + 1];
        float dv = dist[v];
        int i = start, ub = start + S.loopBound(end - start);

        var vDv = FloatVector.broadcast(S, dv);
        for (; i < ub; i += S.length()) {
            // load destination indices
```

```
            // gather is not directly supported; we batch destinations contiguously by
construction
            var ww = FloatVector.fromArray(S, w, i);              // edge weights
            var cand = ww.add(vDv);                               // candidate distance
            // destinations are contiguous for this vertex's row
            // write: scratchBest[edgeTo[i + lane]] = min(...)
            for (int lane = 0; lane < S.length(); lane++) {
                int dest = edgeTo[i + lane];
                float c = cand.lane(lane);
                float prev = scratchBest[dest];
                if (c < prev) scratchBest[dest] = c;              // scalar min to avoid races
            }
        }
        // tail
        for (; i < end; i++) {
            int dest = edgeTo[i];
            float cand = dv + w[i];
            float prev = scratchBest[dest];
            if (cand < prev) scratchBest[dest] = cand;
        }
    }

    // PageRank vector step: rNext = alpha * (A^T * r) + (1 - alpha)/N
    public static void axpy(float a, float[] x, float[] y) {
        int n = Math.min(x.length, y.length);
        int i = 0, ub = S.loopBound(n);
        var va = FloatVector.broadcast(S, a);
        for (; i < ub; i += S.length()) {
            var vx = FloatVector.fromArray(S, x, i);
            var vy = FloatVector.fromArray(S, y, i);
            vx.fma(va, vy).intoArray(y, i); // y = a*x + y
        }
        for (; i < n; i++) y[i] = a * x[i] + y[i];
    }

    public static float reduceSum(float[] a) {
        int i = 0, ub = S.loopBound(a.length);
        var acc = FloatVector.zero(S);
        for (; i < ub; i += S.length()) {
            acc = acc.add(FloatVector.fromArray(S, a, i));
        }
        float sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < a.length; i++) sum += a[i];
        return sum;
    }
}
```

Reasoning to interviewer:

- CSR packs edge weights contiguously per source, giving the Vector API something to chew on despite overall graph irregularity.
- We vectorize the arithmetic (add/compare) and keep conflicting writes scalar or move to a two-phase scheme to avoid atomics. The big wins come from fewer loop branches and better use of CPU SIMD for the arithmetic heavy parts.
- PageRank/centrality have even cleaner vectorization: they're mostly SpMV-like math and reductions, which map well to broadcast/mul/add/reduce patterns.
- Using SPECIES_PREFERRED keeps the code portable: AVX-512/AVX2/NEON get used automatically; correctness falls back gracefully.

Future scope:

- Tile over vertices/edges to improve cache locality; parallelize tiles with ForkJoin while keeping SIMD within each tile.
- Use DoubleVector for numerically sensitive phases and FloatVector elsewhere.
- Batch multiple SpMV operations (e.g., compute rank and normalization in one pass) to reduce memory traffic.
- Add JMH and real-graph A/B benchmarks to prevent regressions across JDK/CPU changes.

Scenario 32: Eliminating circular module dependencies in an enterprise suite

Auth, Inventory, and Payments accidentally depend on each other (Auth ↔ Payments ↔ Inventory), making builds and upgrades painful. We need to cut cycles and keep internal code from leaking.

Answer:

- Split each domain into a tiny API module and a separate implementation module. Other modules depend only on APIs, never on impls.
- Move common value types to suite.common.
- Wire implementations with JPMS services (uses/provides), so the app composes them without direct impl-to-impl imports. This breaks cycles structurally.

Code:

API modules (only contracts are exported):

```
// suite.common
module com.suite.common {
    exports com.suite.common; // UserId, Money, Errors, DateRange
}

// auth.api
module com.suite.auth.api {
    requires transitive com.suite.common;
    exports com.suite.auth.api;                 // AuthService, Tokens DTOs
    uses com.suite.auth.api.AuthService;        // discovery
}

// inventory.api
module com.suite.inventory.api {
    requires transitive com.suite.common;
    exports com.suite.inventory.api;            // InventoryService, Sku DTOs
    uses com.suite.inventory.api.InventoryService;
}

// payments.api
module com.suite.payments.api {
    requires transitive com.suite.common;
    exports com.suite.payments.api;             // PaymentService, Invoice DTOs
    uses com.suite.payments.api.PaymentService;
}
```

Implementation modules (no exports; they only provide services):

```
module com.suite.auth.impl {
    requires com.suite.auth.api;
    requires com.suite.common;
    provides com.suite.auth.api.AuthService
        with com.suite.auth.impl.JwtAuthService;
}

module com.suite.inventory.impl {
    requires com.suite.inventory.api;
    requires com.suite.common;
    provides com.suite.inventory.api.InventoryService
        with com.suite.inventory.impl.DefaultInventoryService;
}

module com.suite.payments.impl {
    requires com.suite.payments.api;
    requires com.suite.common;
    provides com.suite.payments.api.PaymentService
        with com.suite.payments.impl.StripePaymentService;
}
```

Composition/app module (depends only on APIs; discovers impls):

```
module com.suite.app {
    requires com.suite.common;
    requires com.suite.auth.api;
    requires com.suite.inventory.api;
    requires com.suite.payments.api;

    uses com.suite.auth.api.AuthService;
    uses com.suite.inventory.api.InventoryService;
    uses com.suite.payments.api.PaymentService;
}
```

Reasoning to interviewer:

- JPMS hides everything by default; exporting only API packages prevents accidental coupling to internals.
- Splitting API/impl and using uses/provides inverts dependencies so there's no path to form a cycle. The app composes services; providers don't depend on each other's impls.

- requires transitive on APIs re-exports common to consumers, keeping dependency declarations clean and avoiding “reach-through” to impl modules.
- Builds stabilize because module graphs are acyclic and explicit; jdeps can enforce that in CI.

Future scope:

- Add CI checks (jdeps/Revapi) that fail on new cycles or accidental exports.
- If two domains truly need shared abstractions, extract a fourth API (e.g., pricing.api) instead of cross-linking them.
- Produce a jlink image containing only API + selected impls to shrink the runtime and attack surface.
- Version API modules and enforce semver policy to keep the graph healthy over time.

Scenario 33: Adding runtime behavior in a plugin-based CMS

Third-party plugins (reporting, media export) are loaded dynamically. You want to weave behaviors like logging, metrics, validation, and guards around plugin calls without changing plugin code.

Answer:

- I'll load plugins behind small interfaces (e.g., ReportPlugin, Exporter), then wrap each concrete instance with a JDK dynamic proxy.
- The proxy's InvocationHandler will: validate inputs, time execution, add structured logs/metrics, and map exceptions to CMS-friendly errors.
- Behavior is opt-in via annotations or a registry, and completely transparent to plugin authors.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Objects;
import java.util.concurrent.TimeUnit;

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface Validate {}

public interface ReportPlugin {
    @Validate byte[] generate(String contentId, String format);
}

final class PdfReportPlugin implements ReportPlugin {
    public byte[] generate(String contentId, String format) {
        // plugin's own logic
        return ("PDF:" + contentId + ":" + format).getBytes();
    }
}

final class PluginProxy implements InvocationHandler {
    private final Object target;
    private PluginProxy(Object target) { this.target = Objects.requireNonNull(target);
}

    @SuppressWarnings("unchecked")
    static <T> T wrap(Class<T> api, T impl) {
        return (T) Proxy.newProxyInstance(
            api.getClassLoader(), new Class<?>[]{api}, new PluginProxy(impl));
```

```
    }

    @Override public Object invoke(Object proxy, Method m, Object[] args) throws
Throwable {
        long start = System.nanoTime();
        boolean ok = false;
        try {
            if (needsValidation(m)) validateArgs(m, args);
            Object out = m.invoke(target, args);
            ok = true;
            return out;
        } catch (InvocationTargetException ite) {
            throw ite.getCause(); // preserve plugin's exception semantics
        } finally {
            long tookMs = TimeUnit.NANOSECONDS.toMillis(System.nanoTime() -
start);
            logCall(target.getClass(), m, tookMs, ok);
        }
    }

    private boolean needsValidation(Method m) {
        return m.isAnnotationPresent(Validate.class) ||
target.getClass().isAnnotationPresent(Validate.class);
    }

    private void validateArgs(Method m, Object[] args) {
        // minimal sample: real impl would check formats, size limits, etc.
        if (args != null) for (Object a : args) if (a instanceof String s && s.isBlank())
            throw new IllegalArgumentException("Blank string in " + m.getName());
    }

    private void logCall(Class<?> clazz, Method m, long tookMs, boolean ok) {
        System.out.printf("plugin=%s method=%s ok=%s durationMs=%d%n",
                clazz.getSimpleName(), m.getName(), ok, tookMs);
    }
}

// Usage
class Demo {
    public static void main(String[] args) {
        ReportPlugin raw = new PdfReportPlugin();
        ReportPlugin wrapped = PluginProxy.wrap(ReportPlugin.class, raw);
        wrapped.generate("post-42", "pdf"); // validation + timing + log applied
    }
}
```

Reasoning to interviewer:

- Dynamic proxies let me inject cross-cutting concerns at the interface boundary—no bytecode tools needed, and plugin code stays untouched.
- Annotations make policies explicit per method/type. I can attach more behaviors (circuit breaker, rate limits) in the same handler.
- Exceptions are unwrapped to keep stack traces accurate. The proxy is drop-in: the CMS hands out the proxy instead of the raw instance.

Future scope:

- Cache per-method policies to avoid reflection overhead in hot paths.
- Publish timing/err metrics to Micrometer; sample or threshold to control log volume.
- For classes without interfaces, switch to Byte Buddy or CGLIB.
- Add a policy DSL: e.g., “for plugin X, require auth scope ‘cms.admin’ and a 200ms SLA.”

Scenario 34: Validating deserialization of sensitive financial records

Incoming objects may include account details. You must prevent gadget chains or unapproved types from being instantiated during Java deserialization.

Answer:

- I'll attach a context-specific ObjectInputFilter to every ObjectInputStream at the boundary.
- The filter uses a strict allowlist of DTO packages/classes and structural caps (depth, array size, reference count).
- I'll add a conservative global backstop; per-stream filters can be stricter.

Code:

```
import java.io.*;
import java.util.Set;

public final class DeserGuards {
    private static final String DTO_PKG = "com.bank.financial.dto.";
    private static final Set<String> ALSO_OK = Set.of("java.util.ArrayList",
"java.util.HashMap");

    public static ObjectInputFilter perStreamFilter() {
        return info -> {
            if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
            if (info.arrayLength() != -1 && info.arrayLength() > 500_000) return
ObjectInputFilter.Status.REJECTED;
            if (info.references()  != -1 && info.references()  > 1_000_000) return
ObjectInputFilter.Status.REJECTED;

            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitives

            String n = cl.getName();
            if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
            if (n.startsWith("java.lang.invoke.") || n.startsWith("sun.") ||
n.startsWith("jdk.")
                || n.startsWith("java.rmi.")) return ObjectInputFilter.Status.REJECTED;

            return (n.startsWith(DTO_PKG) || ALSO_OK.contains(n))
                    ? ObjectInputFilter.Status.ALLOWED
                    : ObjectInputFilter.Status.REJECTED;
        };
    }
```

```
    // Optional global guard if no per-stream is set
    public static void installGlobalBackstop() {
        ObjectInputFilter.Config.setSerialFilter(info -> {
            Class<?> cl = info.serialClass();
            if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
            return cl.getName().startsWith(DTO_PKG)
                    ? ObjectInputFilter.Status.ALLOWED
                    : ObjectInputFilter.Status.REJECTED;
        });
    }
}

// Boundary usage
try (ObjectInputStream ois = new ObjectInputStream(inStream)) {
    ois.setObjectInputFilter(DeserGuards.perStreamFilter());
    Object obj = ois.readObject(); // rejected unless allowlisted and within limits
    // proceed
}

Alternative (pattern string in config):


ObjectInputFilter f = ObjectInputFilter.Config.createFilter(
    "maxdepth=20;maxarray=500000;maxrefs=1000000;" +
    "com.bank.financial.dto.*;java.util.*;!*"
);
ois.setObjectInputFilter(f);
```

Reasoning to interviewer:

- Filters run before object graph creation, stopping malicious types early.
- Default-deny with a tight allowlist is the safest stance; structural limits also prevent resource-exhaustion bombs.
- Per-stream filters let each topic/endpoint define its own valid DTO set; the global backstop covers omissions.

Future scope:

- Prefer schema-driven formats (JSON/Avro/Protobuf) with strict mappers.
- Externalize allowlists and test them in CI with “known bad” payloads.
- Sign messages or use mTLS so fewer untrusted payloads reach deserialization.
- Log rejections with minimal details for audit/forensics.

Scenario 35: Reducing heap fragmentation in a messaging service

High-throughput producers/consumers allocate many short-lived and some medium-lived objects. Over time, fragmented regions increase GC frequency and pause variance.

Answer:

- Pick a collector with concurrent compaction to mitigate fragmentation:
  - Java 21+ generational ZGC (default concurrent compaction, tiny pauses).
  - Shenandoah (if available) as another concurrent-compacting option.
  - If you must use G1, tune region sizing and mixed-GC cadence to keep old-gen defragmented.
- Keep allocation patterns friendly: prefer small, short-lived objects (die young), reuse buffers/builders, avoid accidental medium-lifetimes that get promoted.
- Size the heap with headroom and fix Xms=Xmx to avoid resizing churn.

Code (ZGC baseline):

```
java \
  -Xms8g -Xmx8g \                  # fixed heap for stable ergonomics
  -XX:+UseZGC \                    # generational ZGC on JDK 21+
  -XX:SoftMaxHeapSize=7g \         # keep free headroom to avoid last-minute
cycles
  -Xlog:gc*:file=gc.log:time,uptime,level,tags \
  -XX:+AlwaysPreTouch \            # pre-fault pages for stable latencies
  -XX:+ZProactive \                # proactive concurrent work
  -XX:+DisableExplicitGC \         # ignore accidental System.gc()
  -jar messaging.jar
```

Code (G1 tuned when ZGC isn't available):

```
java \
  -Xms8g -Xmx8g \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=50 \            # start target; iterate with data
  -XX:InitiatingHeapOccupancyPercent=30 \   # start cycles earlier to smooth
compaction
  -XX:G1HeapRegionSize=8m \            # pick region size fitting your object
sizes
  -XX:G1ReservePercent=20 \            # free space buffer to reduce allocation
stalls
```

```
  -XX:G1NewSizePercent=30 -XX:G1MaxNewSizePercent=60 \  # roomy young
gen for ephemeral msgs
  -XX:SurvivorRatio=8 -XX:TargetSurvivorRatio=80 \      # avoid premature
promotion
  -XX:+ParallelRefProcEnabled \
  -Xlog:gc*:file=gc.log:time,uptime,level,tags \
  -XX:+AlwaysPreTouch -XX:+DisableExplicitGC \
  -jar messaging.jar
```

App-level hygiene (cutting fragmentation at the source):

```
// Reuse per-thread builders/buffers to avoid churn
final class Buffers {
  private static final ThreadLocal<StringBuilder> SB =
      ThreadLocal.withInitial(() -> new StringBuilder(256));

  static String buildLog(String topic, int size) {
    var sb = SB.get(); sb.setLength(0);
    sb.append("t=").append(topic).append(",sz=").append(size);
    return sb.toString(); // only the final String allocates
  }
}
```

Reasoning to interviewer:

- Concurrent compaction (ZGC/Shenandoah) keeps the heap defragmented continuously with tiny pauses—ideal for long-lived servers with mixed lifetimes.
- Fixed heap and proactive tuning avoid surprise expansions and last-minute mark-compact work that spikes latency.
- With G1, region sizing, reserve percent, and earlier IHOP help avoid fragmentation by keeping space for mixed collections and compaction.
- Reducing promotion by right-sizing the young gen and reusing transient objects lowers old-gen fragmentation pressure.

Future scope:

- Use JFR + GC logs to correlate allocation rate, promotion rate, pause percentiles; add SLO alerts on GC > budget.
- Consider off-heap/pooled ByteBuffers for message serialization to reduce heap pressure.

- Apply string dedup (-XX:+UseStringDeduplication) if the workload repeats many keys/headers.
- Decompose “medium-lived” objects (which often fragment heaps) into either truly short-lived or promote them intentionally via caching to stabilize tenure.

Scenario 36: Thread safety in an e-commerce order pipeline

Orders flow through multiple stages (validate → reserve → charge → ship). Several threads also update shared stats (orders/sec, revenue, per-SKU counters). We need correctness without turning the pipeline into a single lock.

Answer:

- Use contention-friendly atomics for hot counters: LongAdder/DoubleAdder beat AtomicLong under high write concurrency.
- Keep shared maps concurrent: ConcurrentHashMap<K, LongAdder> with computeIfAbsent to avoid double-checked races.
- Prefer immutable messages between stages; mutate local variables, not shared state.
- Shard by key (e.g., SKU or userId) to minimize cross-shard contention; per-key single-writer executors or mailboxes work well.
- Batch/accumulate locally, then flush atomically to shared counters to reduce write amplification.
- Avoid broad synchronized; if you need locks for multi-field invariants, use fine-grained locks with a consistent order.
- For read-heavy stats snapshots, publish immutable copies (or use StampedLock optimistic reads) instead of holding writers back.

Code:

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

// global stats (high contention-friendly)
final class Stats {
  final LongAdder orders = new LongAdder();
  final LongAdder failed  = new LongAdder();
  final DoubleAdder revenue = new DoubleAdder();
  final ConcurrentHashMap<String, LongAdder> perSku = new
ConcurrentHashMap<>();

  void addOrder(String sku, double amount) {
    orders.increment();
    revenue.add(amount);
    perSku.computeIfAbsent(sku, k -> new LongAdder()).increment();
  }
  void addFailure() { failed.increment(); }
```

```
}

// pipeline stage, immutable DTO
record Order(String id, String sku, int qty, double price) {}

// shard updates by SKU to reduce contention
final class ShardedUpdaters {
  private final int shards = Runtime.getRuntime().availableProcessors();
  private final ExecutorService[] execs = new ExecutorService[shards];
  ShardedUpdaters() {
    for (int i=0;i<shards;i++) execs[i] = Executors.newSingleThreadExecutor();
  }
  void submit(Order o, Runnable r) {
    int idx = Math.floorMod(o.sku().hashCode(), shards);
    execs[idx].submit(r); // per-shard single writer ⇒ no locks needed inside r
  }
}

final class Pipeline {
  private final Stats stats = new Stats();
  private final ShardedUpdaters shard = new ShardedUpdaters();

  void onOrder(Order o) {
    // local work, no shared writes yet
    double amount = o.qty() * o.price();

    // accumulate to shared stats via sharded single-writer
    shard.submit(o, () -> stats.addOrder(o.sku(), amount));
  }

  // periodic snapshot for dashboards (no writer blocking)
  Snapshot snapshot() {
    return new Snapshot(stats.orders.sum(), stats.failed.sum(),
stats.revenue.sum());
  }
}
record Snapshot(long orders, long failed, double revenue) {}
```

Reasoning to interviewer:

- LongAdder/DoubleAdder reduce contention by striping—perfect for hot counters updated by many threads.
- ConcurrentHashMap with computeIfAbsent makes per-SKU counters safe without external locks.
- Sharding converts many contending writers into N single writers, eliminating most locking without sacrificing parallelism.

- Immutable DTOs keep stages side-effect-free; shared state is updated in narrow, controlled places.
- Snapshots read via sum() avoid blocking writers; if I needed multi-field consistency, I’d publish an immutable snapshot object on a schedule.

Future scope:

- Add a bounded, lock-free queue (SPSC/MPSC) per shard to further cut scheduler overhead.
- Export metrics via Micrometer; alert on contention (adder contention shows up as CPU spikes/latency).
- For complex multi-field invariants, use per-account/mailbox single-writer actors.
- If a SKU is ultra-hot, isolate it into its own shard dynamically.

Scenario 37: Faster startup for microservices

Dozens of services must scale quickly; cold start is dominated by class loading, verification, and framework initialization.

Answer:

- **Enable Class Data Sharing:** JVM maps pre-processed class metadata from a shared archive. Use base CDS and AppCDS to include your app/framework classes.
- **Lazy/lite initialization:** defer non-critical beans/components, cache warmups, and client pools until first use.
- **Trim the classpath:** remove unused starters/autoconfigs and optional transitive deps.
- **Parallel and incremental class loading:** mark custom class loaders as parallelCapable; avoid heavy reflective scanning at boot.
- **Profile-guided class lists:** generate a classlist from a warm run, then build an AppCDS archive for production.

Code (AppCDS workflow):

```
# 1) Generate a classlist from a representative run
java -Xshare:off -XX:DumpLoadedClassList=app.classlist -jar service.jar --sample-traffic

# 2) Create a shared archive with your app + frameworks
java -Xshare:dump \
    -XX:SharedClassListFile=app.classlist \
    -XX:SharedArchiveFile=app-cds.jsa \
    -cp service.jar:lib/*

# 3) Run using the archive (startup now maps preprocessed metadata)
java -Xshare:on -XX:SharedArchiveFile=app-cds.jsa -jar service.jar
```

Code (lazy init patterns, framework-agnostic):

```
// lazy holder idiom for heavy singletons
final class HeavyClient {
  private HeavyClient() { /* connect/create pools */ }
  private static class Holder { static final HeavyClient I = new HeavyClient(); }
  static HeavyClient get() { return Holder.I; }
}
```

```
// provider pattern to delay construction
interface Provider<T> { T get(); }
final class MetricsProvider implements Provider<Metrics> {
  private volatile Metrics m;
  public Metrics get() {
    Metrics local = m;
    if (local == null) {
      synchronized (this) { if (m == null) m = new Metrics(...); local = m; }
    }
    return local;
  }
}
```

Reasoning to interviewer:

- CDS/AppCDS eliminates repeated bytecode parsing/verification and shares metadata pages across processes—huge wins for fleets.
- Deferring non-critical initialization shortens the critical path to “ready”; most services don’t need all clients warm at t=0.
- Cutting reflection and classpath bloat directly reduces classes loaded and time in <clinit>.
- Parallel-capable class loaders and fewer static initializers reduce single-threaded choke points during boot.

Future scope:

- Consider native images (GraalVM AOT) for specific services where the trade-offs fit (smaller RSS, very fast cold starts).
- Persist warmed caches to disk between deploys (where safe) to avoid recomputation.
- Use jlink to build a trimmed runtime image containing only required modules + CDS archive.
- Automate classlist generation in CI from realistic traffic replays.

Scenario 38: Securing service-to-service calls with JWT

Service A calls Service B over HTTP. Only authorized services should access B's endpoints, and B must verify caller identity and scopes.

Answer:

- Issue signed JWTs (RS256/ES256) from a central issuer (Auth service). Tokens include iss, aud, sub, and fine-grained scope.
- Service A fetches a token (client credentials flow), sends it as Authorization: Bearer <jwt>.
- Service B validates signature using issuer JWKS (cache by kid), checks expiry/aud/scope, and enforces least privilege per route.
- Rotate keys via JWKS; cache keys with TTL, fall back to refresh on kid miss. Prefer mTLS between services in addition to JWT for defense-in-depth.

Code (server-side verification with Nimbus JOSE + JWT):

```
// Gradle/Maven: com.nimbusds:nimbus-jose-jwt
import com.nimbusds.jose.jwk.source.*;
import com.nimbusds.jose.proc.*;
import com.nimbusds.jwt.proc.*;
import com.nimbusds.jose.util.DefaultResourceRetriever;
import com.nimbusds.jwt.SignedJWT;

import javax.servlet.*;
import javax.servlet.http.*;
import java.net.URL;
import java.text.ParseException;
import java.util.Set;

public class JwtAuthFilter implements Filter {
  private final String issuer = "https://auth.example.com";
  private final Set<String> audience = Set.of("service-b");
  private final Set<String> requiredScopes = Set.of("orders.read");

  private final
ConfigurableJWTProcessor<com.nimbusds.jose.proc.SecurityContext> processor;

  public JwtAuthFilter() throws Exception {
    var retriever = new DefaultResourceRetriever(2000, 2000, 512 * 1024);
    var jwkSetURL = new URL(issuer + "/.well-known/jwks.json");
    var keySource = new RemoteJWKSet<>(jwkSetURL, retriever);
    processor = new DefaultJWTProcessor<>();
    JWSKeySelector<com.nimbusds.jose.proc.SecurityContext> selector =
```

```
        new
JWSVerificationKeySelector<>(com.nimbusds.jose.JWSAlgorithm.RS256,
keySource);
    processor.setJWSKeySelector(selector);
  }

  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
      throws java.io.IOException, ServletException {
    HttpServletRequest http = (HttpServletRequest) req;
    String h = http.getHeader("Authorization");
    if (h == null || !h.startsWith("Bearer ")) { ((HttpServletResponse)
res).sendError(401); return; }
    String token = h.substring(7);

    try {
      var claims = processor.process(SignedJWT.parse(token), null);
      // validate issuer, audience, expiry (Nimbus checks exp by default if
configured)
      if (!issuer.equals(claims.getIssuer()) ||
claims.getAudience().stream().noneMatch(audience::contains)) {
        ((HttpServletResponse) res).sendError(403); return;
      }
      var scopes = Set.copyOf(claims.getStringClaim("scope").split(" "));
      if (!scopes.containsAll(requiredScopes)) { ((HttpServletResponse)
res).sendError(403); return; }

      // attach principal to request context, continue
      chain.doFilter(req, res);
    } catch (ParseException | com.nimbusds.jose.proc.BadJOSEException e) {
      ((HttpServletResponse) res).sendError(401);
    } catch (com.nimbusds.jose.JOSEException e) {
      ((HttpServletResponse) res).sendError(500);
    }
  }
}
```

Reasoning to interviewer:

- JWT lets us authenticate the caller without reaching the issuer on each request; integrity is guaranteed by signature verification against JWKS.
- Validating aud binds the token to the intended service; scope/roles enforce authorization per endpoint.
- Caching JWKS keys by kid avoids hot-path network calls while still allowing key rotation.

- Pairing JWT with mTLS (service-mesh or TLS client certs) gives both strong transport security and application-layer authZ.

Future scope:

- Add per-route scopes and purpose-bound tokens with short TTL; rely on refresh via client-credentials flow.
- Use DPoP or MTLS-bound tokens to prevent token replay across callers.
- Propagate distributed tracing ids in headers and include them in audit logs with sub/scope.
- Introduce fine-grained authorization via ABAC (attributes in JWT claims) for sensitive routes.

Scenario 39: Sealed classes for payroll roles

A payroll system only allows two employee categories: FullTime and Contractor. The base Employee must forbid any other subclasses.

Answer:

I'll declare Employee as a sealed type and explicitly list the only permitted subclasses. Each permitted subclass is final (or non-sealed if we want a controlled extension point). This lets the compiler enforce the closed hierarchy and enables exhaustive switches.

Code:

```
// Java 17+
public sealed interface Employee permits FullTime, Contractor {
    String id();
    String name();
    long basePayMicros();
}

public final class FullTime implements Employee {
    private final String id, name;
    private final long annualSalaryMicros;

    public FullTime(String id, String name, long annualSalaryMicros) {
        this.id = id; this.name = name; this.annualSalaryMicros = annualSalaryMicros;
    }
    public String id() { return id; }
    public String name() { return name; }
    public long basePayMicros() { return annualSalaryMicros; }
}

public final class Contractor implements Employee {
    private final String id, name;
    private final long hourlyRateMicros;

    public Contractor(String id, String name, long hourlyRateMicros) {
        this.id = id; this.name = name; this.hourlyRateMicros = hourlyRateMicros;
    }
    public String id() { return id; }
    public String name() { return name; }
    public long basePayMicros() { return hourlyRateMicros; }
}

// Exhaustive switch (Java 21 pattern matching)
static long monthlyCost(Employee e) {
    return switch (e) {
```

```
        case FullTime ft   -> ft.basePayMicros() / 12;
        case Contractor c  -> c.basePayMicros() * 160; // assume 160 hrs
    };
}
```

Reasoning to interviewer:

- Sealed + permits locks the inheritance tree; no third-party class can sneak in another employee type.
- Marking concrete classes final prevents further subclassing; if we ever need internal variants (e.g., PartTime), we can mark one branch non-sealed.
- Exhaustive switches over a sealed hierarchy are compiler-checked—no default branch needed, so missing cases fail fast at build time.

Future scope:

- Combine with JPMS: place Employee and its permitted types in the same module to avoid cross-module surprises.
- Represent pure data variants as records (e.g., record FullTime(...) implements Employee).
- Add validation at construction time (non-negative pay), pushing invariants to the edge.
- If regions differ (e.g., contractor policies), use non-sealed Contractor and constrain subtypes inside payroll’s module.

Scenario 40: Validating nested JSON payloads in user registration

The API accepts nested JSON like { user: {...}, address: {...} }. You want declarative validation before persistence.

Answer:

I'll model the payload as DTOs and use Bean Validation (JSR 380) annotations on fields. For nested objects, I'll use @Valid so the validator descends into the child. Controllers trigger validation with @Valid/@Validated; violations return structured errors.

Code:

```
// DTOs
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;

public class RegistrationRequest {
    @Valid @NotNull
    private UserInfo user;

    @Valid @NotNull
    private Address address;

    // getters/setters or make these records (below)
}

public class UserInfo {
    @NotBlank @Size(max = 100)
    private String fullName;

    @Email @NotBlank
    private String email;

    @NotBlank @Pattern(regexp = "^[A-Za-z0-9_]{6,32}$")
    private String username;

    @NotBlank @Size(min = 8, max = 64)
    private String password;

    // getters/setters
}

public class Address {
    @NotBlank private String line1;
    private String line2;
    @NotBlank private String city;
```

```
    @NotBlank private String state;
    @NotBlank @Pattern(regexp = "^[0-9]{5}(-[0-9]{4})?$")
    private String postalCode;
    @NotBlank private String country;
    // getters/setters
}

// Spring controller example
@PostMapping("/register")
public ResponseEntity<?> register(@Valid @RequestBody RegistrationRequest
req) {
    // If invalid, Spring returns 400 with constraint violations automatically
    // else proceed to persist
    return ResponseEntity.ok("ok");
}
```

Or with records (cleaner, immutable):

```
public record RegistrationRequest(
        @Valid @NotNull UserInfo user,
        @Valid @NotNull Address address) { }

public record UserInfo(
        @NotBlank @Size(max=100) String fullName,
        @Email @NotBlank String email,
        @NotBlank @Pattern(regexp="^[A-Za-z0-9_]{6,32}$") String username,
        @NotBlank @Size(min=8, max=64) String password) { }

public record Address(
        @NotBlank String line1,
        String line2,
        @NotBlank String city,
        @NotBlank String state,
        @NotBlank @Pattern(regexp="^[0-9]{5}(-[0-9]{4})?$") String postalCode,
        @NotBlank String country) { }
```

Reasoning to interviewer:

- @Valid cascades into nested objects, so I get deep validation with minimal code.
- Annotations keep rules close to the data model; controllers stay thin and consistent.
- Using records makes DTOs immutable and reduces boilerplate while still working with constructor binding.

- Frameworks (Spring/Micronaut/Jakarta) translate violations into standard error payloads—no manual checks or branching.

Future scope:

- Add custom constraints (e.g., @PasswordStrength, @CountryPostalCode) and class-level cross-field checks (e.g., username != email).
- Localize messages via ValidationMessages.properties.
- Put validation in a shared module so both REST and batch imports reuse the exact rules.
- Add JSON Schema generation from constraints for client-side pre-validation.

Scenario 41: Tuning GC for a bursty, low-latency e-commerce checkout

Flash sales create sudden waves of short-lived cart objects and cache promotions. You need sub-100ms (often sub-50ms) p99 latencies while sustaining high throughput.

Answer:

I'd start with a low-latency, concurrent-compacting collector and size the heap for headroom. Then iterate with GC logs/JFR under realistic burst loads.

- **Collector choice**
  - Java 21+: generational ZGC is my default for tight tail latencies (concurrent mark+compact, tiny pauses).
  - If ZGC isn't available, G1 with tuned pause targets and young-gen sizing is the pragmatic fallback.
- **Heap sizing strategy**
  - Fix -Xms = -Xmx to avoid in-flight resizing during spikes.
  - Leave 10–20% headroom via SoftMaxHeapSize (ZGC) or G1ReservePercent (G1) so bursts don't hit the wall.
  - Aim for enough young space to absorb peak allocation bursts; prevent premature promotions.
- **Pause-time targets & behavior**
  - ZGC: expect single-digit ms pauses even at multi-GB heaps; focus on keeping concurrent cycles ahead of allocation.
  - G1: start with MaxGCPauseMillis=50, then adjust based on p95/p99 under load; avoid chasing absolute zero pauses.
- **Startup vs steady state**
  - Pre-touch pages (AlwaysPreTouch) and warm caches during startup to avoid first-touch stalls during the first spike.
  - CDS/AppCDS to speed warm-up so the service reaches steady allocation behavior faster.
- **Diagnostics to iterate safely**
  - Enable unified GC logging with detail and safepoint reasons; pipe to a file for analysis.

- Use JFR with GC, allocation, and executor events to correlate allocation spikes with pause outliers.

Code (ZGC baseline, Java 21+):

```
java \
  -Xms8g -Xmx8g \                        # fixed heap, stable ergonomics
  -XX:+UseZGC \                          # generational by default on 21+
  -XX:SoftMaxHeapSize=7g \                    # headroom for bursts
  -XX:+ZProactive \                      # keep ahead of allocation rate
  -XX:+AlwaysPreTouch \                       # avoid page faults at peak
  -XX:+DisableExplicitGC \                    # ignore accidental System.gc()
  -Xlog:gc*,safepoint:file=gc.log:time,uptime,tags,level \
  -XX:StartFlightRecording=filename=jfr.jfr,settings=profile \
  -jar checkout.jar
```

Code (G1 tuned fallback):

```
java \
  -Xms8g -Xmx8g \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=50 \                   # start target; verify against p99
  -XX:InitiatingHeapOccupancyPercent=30 \     # earlier concurrent marking
  -XX:G1NewSizePercent=30 -XX:G1MaxNewSizePercent=60 \  # bigger young for
bursts
  -XX:TargetSurvivorRatio=80 -XX:SurvivorRatio=8 \     # curb early promotion
  -XX:G1ReservePercent=20 \                   # free-space buffer
  -XX:+ParallelRefProcEnabled \
  -XX:+AlwaysPreTouch -XX:+DisableExplicitGC \
  -Xlog:gc*,safepoint:file=gc.log:time,uptime,tags,level \
  -XX:StartFlightRecording=filename=jfr.jfr,settings=profile \
  -jar checkout.jar
```

Reasoning to interviewer:

- Generational ZGC minimizes pause variance with concurrent relocation; for bursty short-lived objects, the young-gen handles the churn while old-gen stays compacted in the background.
- Fixed heap and reserved headroom prevent “cliff” behaviors during flash events; the collector has space to breathe.
- With G1, larger young regions and earlier IHOP smooth cycles; survivor/target ratios reduce accidental promotions that bloat old-gen and trigger mixed collections at bad times.

- GC logs + JFR give the feedback loop: I'll watch allocation rate, young/old collection cadence, promotion failure, and pause distributions, then nudge targets and region sizing accordingly.

Future scope:

- App-level: reuse builders/buffers, cut string/boxing churn, and cap cache object lifetimes to keep objects in young gen.
- Autoscaling: scale out proactively before flash sales so per-instance allocation rate stays below GC stress thresholds.
- Consider Shenandoah as an alternative low-pause collector if that's the platform default.
- Establish SLO burn alerts on p99 pauses and "% time in GC" from JFR; gate releases on load-test baselines to avoid regressions.

Scenario 42: JVM on constrained edge devices (smart meters)

Single-core, small RAM; periodic TLS handshakes create brief spikes. You need predictable behavior and a tiny footprint.

Answer:

- Collector: prefer Serial GC on single-core, tiny heaps—no background threads, minimal overhead, short pauses due to small heap size.
- Heap sizing: keep it small and fixed to avoid resizing; tune young/old so batches die young.
- Class metadata & startup footprint: use jlink to build a trimmed runtime image + CDS/AppCDS to avoid parsing/verification at boot.
- Compilation: limit tiered compilation to reduce warmup CPU; keep JIT light.
- Disable heavyweight features: JFR, heap dumps on OOM, large thread stacks; keep logging minimal and bounded.
- Runtime architecture: bounded queues with backpressure; keep TLS connections warm (HTTP/2 or persistent TLS), and schedule batch uploads away from GC windows.

Code (typical launch flags):

```
java \
  -XX:+UseSerialGC \                    # single-core friendly
  -Xms48m -Xmx48m \                     # fixed, small heap
  -XX:MaxMetaspaceSize=64m \            # cap class metadata
  -XX:+UseCompressedOops -XX:+UseCompressedClassPointers \
  -XX:+TieredCompilation -XX:TieredStopAtLevel=1 \   # C1 only, low JIT overhead
  -XX:+AlwaysActAsServerClassMachine=false \         # keep client-like behavior
  -XX:-HeapDumpOnOutOfMemoryError \
  -Xss256k \                            # small stacks, few threads
  -Xshare:on \                          # CDS enabled (use AppCDS archive if built)
  -Djava.util.logging.config.file=logging.properties \
  -jar smart-meter.jar
```

Build-time footprint reductions:

- jlink: include only needed modules (e.g., java.base, java.net.http, jdk.crypto.ec).
- AppCDS: generate a classlist from a warm run and build an archive with app/framework classes.

- Optional: consider a native image (GraalVM) for some devices if you can trade peak throughput for tiny RSS and instant start.

Runtime practices:

- Preallocate/buffer small batches; reuse ByteBuffer/StringBuilder via ThreadLocal.
- Keep a small young gen bias (Serial automatically manages, but avoid medium-lived objects).
- Use a single scheduler thread; offload TLS handshakes to scheduled windows; keep connections pooled.

Reasoning to interviewer:

- Serial GC avoids concurrent GC threads competing on a single core; with a 30–64 MB heap, pauses are brief.
- Fixed heap and bounded queues enforce a hard ceiling on memory—spikes trigger backpressure, not death spirals.
- CDS/jlink cut startup IO/CPU and lower RSS.
- Limiting to C1 compilation reduces warmup work while still producing decent code for a small app.

Future scope:

- Stagger telemetry batches across devices to avoid gateway thundering herds.
- Add simple alloc budgets via periodic JFR-lite alternatives (or custom counters) if available.
- Tighten TLS by session resumption/HTTP/2 to reduce handshake spikes.
- If you later add multi-threading, consider G1 with a tiny heap only if pause predictability degrades.

Scenario 43: Runtime-pluggable logging behaviors via dynamic proxies

You need to prepend request-id, drop DEBUG in prod, or buffer-then-flush on ERROR—without changing the logger implementation.

Answer:

- Wrap the logger interface with a JDK dynamic proxy.
- The InvocationHandler reads live config (env flags, MDC/ThreadLocal) and applies behaviors: mutate arguments (prepend requestId), short-circuit (drop DEBUG), or buffer messages per request and flush on error().
- Swapping behaviors is just changing config; no recompilation.

Code:

```
import java.lang.reflect.*;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

interface Logger {
  void debug(String msg);
  void info(String msg);
  void error(String msg);
  void error(String msg, Throwable t);
}

// Simple impl (could be slf4j bridge, etc.)
final class ConsoleLogger implements Logger {
  private final String name;
  ConsoleLogger(String name) { this.name = name; }
  public void debug(String m) { System.out.println("DEBUG " + name + " " + m); }
  public void info(String m)  { System.out.println("INFO  " + name + " " + m); }
  public void error(String m) { System.err.println("ERROR " + name + " " + m); }
  public void error(String m, Throwable t) {
    System.err.println("ERROR " + name + " " + m); t.printStackTrace(System.err);
  }
}

// Runtime config toggles (could be refreshed from config server)
final class LogConfig {
  volatile boolean dropDebug = true;
  volatile boolean prependRequestId = true;
  volatile boolean bufferUntilError = true;
}
```

```
final class RequestContext {
  static final ThreadLocal<String> requestId = new ThreadLocal<>();
}

// Handler that injects cross-cutting behaviors
final class LoggingProxy implements InvocationHandler {
  private final Logger target;
  private final LogConfig cfg;
  private final Map<String, List<String>> buffers = new ConcurrentHashMap<>();

  private LoggingProxy(Logger target, LogConfig cfg) {
    this.target = target; this.cfg = cfg;
  }
  static Logger wrap(Logger impl, LogConfig cfg) {
    return (Logger) Proxy.newProxyInstance(
        Logger.class.getClassLoader(), new Class<?>[]{Logger.class},
        new LoggingProxy(impl, cfg));
  }

  @Override public Object invoke(Object proxy, Method m, Object[] args) throws
Throwable {
    String method = m.getName();
    String msg = (args != null && args.length > 0 && args[0] instanceof String) ?
(String) args[0] : "";

    // 1) Drop DEBUG in prod
    if (cfg.dropDebug && method.equals("debug")) return null;

    // 2) Prepend request-id if configured
    if (cfg.prependRequestId) {
      String rid = RequestContext.requestId.get();
      if (rid != null && !rid.isBlank()) {
        msg = "[rid=" + rid + "] " + msg;
        if (args != null && args.length > 0) args[0] = msg;
      }
    }

    // 3) Buffer until error: keep messages per request, flush when error() called
    if (cfg.bufferUntilError) {
      String rid =
Optional.ofNullable(RequestContext.requestId.get()).orElse("nogroup");
      if (method.equals("error")) {
        // flush buffered lines
        List<String> lines = buffers.remove(rid);
        if (lines != null) for (String line : lines) target.info("(pre-error) " + line);
        // then log the error
        return m.invoke(target, args);
      } else if (method.equals("info") || method.equals("debug")) {
        buffers.computeIfAbsent(rid, k -> new ArrayList<>())
               .add(Instant.now() + " " + msg);
```

```
        return null; // buffered, not emitted yet
      }
    }

    return m.invoke(target, args); // default passthrough
  }
}

// Usage
class Demo {
  public static void main(String[] args) {
    LogConfig cfg = new LogConfig();
    Logger base = new ConsoleLogger("Orders");
    Logger log = LoggingProxy.wrap(base, cfg);

    RequestContext.requestId.set("REQ-123");
    log.debug("created draft");      // dropped
    log.info("validating input");    // buffered
    log.info("calling payment");     // buffered
    log.error("payment failed");     // flush buffered, then log error
  }
}
```

Reasoning to interviewer:

- Dynamic proxies intercept calls at the interface boundary—perfect for weaving policy without touching concrete logger classes.
- The handler is responsible for: reading live config, transforming arguments (prepend), deciding to short-circuit (drop), or deferring (buffer/flush).
- Because behavior is driven by LogConfig, toggling features is runtime-only—no rebuilds or restarts.
- Using ThreadLocal (or MDC) makes request-scoped behaviors trivial.

Future scope:

- Swap the ConsoleLogger with a real SLF4J backend; keep the proxy unchanged.
- Add rate-limits or sampling in the handler for noisy endpoints.
- Expose config via dynamic sources (config server) and hot-reload safely.
- Cache Method → behavior decisions to avoid reflection overhead on hot paths.

Scenario 44: Memory visibility and atomicity in a wallet service

Multiple workers update balances and others read them for fraud checks. Reads must observe the latest confirmed writes; debits/credits mustn't interleave and lose money.

Answer:

- Use the Java Memory Model's happens-before edges to guarantee visibility and ordering.
    - Lock release → subsequent lock acquire creates a happens-before edge.
    - Volatile/atomic writes happen-before subsequent reads.
    - Atomic classes give you volatile semantics plus CAS (compare-and-set) for atomicity.
- For publishing the "latest balance" to readers, expose either:
    - a single AtomicLong per account (get() is a volatile read, always seeing the freshest value), or
    - a volatile reference to an immutable snapshot record if you have multiple fields (balance, lastTxnId, version).
- For atomic debit/credit operations, use:
    - AtomicLong + CAS loop for single-field balance updates, or
    - a narrow lock (e.g., ReentrantLock) when multiple fields must change together (or transfers across two accounts—lock in global order to avoid deadlocks).

Code:

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.ReentrantLock;

// (a) Simple, single-field balance with atomicity + visibility
final class Account {
  private final AtomicLong balanceMicros = new AtomicLong();   // volatile
semantics
  final ReentrantLock lock = new ReentrantLock();              // for multi-field ops

  long readBalance() {                  // readers see latest due to volatile read
    return balanceMicros.get();
```

```
  }
  void credit(long delta) {              // CAS ensures no lost updates
    long p, n;
    do { p = balanceMicros.get(); n = p + delta; } while
(!balanceMicros.compareAndSet(p, n));
  }
  void debit(long delta) {
    long p, n;
    do {
      p = balanceMicros.get(); n = p - delta;
      if (n < 0) throw new IllegalStateException("insufficient funds");
    } while (!balanceMicros.compareAndSet(p, n));
  }
}

// (a) Publishing a multi-field immutable snapshot via volatile
record Snapshot(long balanceMicros, long version, String lastTxnId) {}
final class RichAccount {
  private volatile Snapshot snap = new Snapshot(0, 0, ""); // volatile publish
  final ReentrantLock lock = new ReentrantLock();

  Snapshot read() { return snap; }  // always the latest published snapshot

  void apply(long delta, String txnId) {
    lock.lock();                   // lock orders updates and creates HB on unlock
    try {
      long newBal = snap.balanceMicros() + delta;
      if (newBal < 0) throw new IllegalStateException("insufficient funds");
      snap = new Snapshot(newBal, snap.version() + 1, txnId);   // single volatile
write publishes atomically
    } finally {
      lock.unlock();
    }
  }
}

// (b) Cross-account transfer with deterministic lock ordering
final class Ledger {
  private final ConcurrentHashMap<Long, Account> accounts = new
ConcurrentHashMap<>();
  private Account acc(long id) { return accounts.computeIfAbsent(id, k -> new
Account()); }

  void transfer(long fromId, long toId, long amount) {
    if (fromId == toId) return;
    Account a = acc(fromId), b = acc(toId);
    Account first = (fromId < toId) ? a : b;
    Account second = (first == a) ? b : a;

    first.lock.lock();
```

```
    try {
      second.lock.lock();
      try {
        a.debit(amount);
        b.credit(amount);
      } finally {
        second.lock.unlock();
      }
    } finally {
      first.lock.unlock();
    }
  }
}
```

Reasoning to interviewer:

- Atomic/volatile reads give immediate visibility of the newest balance without extra locks.
- CAS ensures debit/credit atomicity—no lost updates under contention.
- When multiple fields must move together, a short critical section guarded by a lock provides ordering; the unlock publishes the new state (HB edge).
- Deterministic lock ordering prevents deadlocks for multi-account operations.

Future scope:

- Use a per-account single-writer mailbox for ultra-hot accounts to remove locks entirely.
- Attach idempotency keys to events to avoid double application on retries.
- Mirror JVM ordering at the DB layer (e.g., SELECT … FOR UPDATE in account-id order).
- Expose a Flow.Publisher<Snapshot> to fraud/analytics so they always consume the latest published version.

Scenario 45: Custom class loading with cyclic plugin references

A reporting engine loads “formatters” and “data sources” as separate JAR plugins. Some plugins reuse each other’s small utility classes, so A may reference B while B also references A. We must avoid duplicate class definitions, deadlocks, and linkage errors.

Answer:

- Parent-first only for platform/engine namespaces (java.*, jakarta.*, logging, and the engine’s own packages).
- Child-first for everything else so a plugin can use its bundled libs without being shadowed by the engine.
- A global class-definition broker: for any FQCN, exactly one loader defines it; others join that result via a future. This avoids LinkageError: duplicate class definition.
- registerAsParallelCapable + per-class locks (getClassLoadingLock) to avoid global loader locks and cyclic deadlocks.
- A tiny “export index” mapping exported packages → owning PluginClassLoader to route cross-plugin lookups directly (no blind probing).

Code:

```
import java.net.URL;
import java.net.URLClassLoader;
import java.util.*;
import java.util.concurrent.*;

final class ClassLoadingBroker {
  private static final ConcurrentHashMap<String, CompletableFuture<Class<?>>>
inFlight = new ConcurrentHashMap<>();

  static Class<?> defineOnce(String name, Callable<Class<?>> definer) throws
ClassNotFoundException {
    var f = inFlight.computeIfAbsent(name, k -> new CompletableFuture<>());
    boolean owner = !f.isDone();
    if (owner) {
      try {
        var c = definer.call();
        f.complete(c);
        return c;
      } catch (Throwable t) {
        f.completeExceptionally(t);
        throw (t instanceof ClassNotFoundException) ? (ClassNotFoundException) t
```

```
            : new ClassNotFoundException("Failed to define " + name, t);
      } finally {
        inFlight.remove(name, f);
      }
    } else {
      try { return f.join(); }
      catch (CompletionException ce) {
        var cause = (ce.getCause() != null) ? ce.getCause() : ce;
        throw new ClassNotFoundException("Join failed for " + name, cause);
      }
    }
  }
}

final class PluginIndex {
  private final ConcurrentMap<String, PluginClassLoader> exports = new
ConcurrentHashMap<>();
  void register(PluginClassLoader ldr, Set<String> exportedPkgs) {
exportedPkgs.forEach(p -> exports.put(p, ldr)); }
  Optional<PluginClassLoader> ownerOf(String className) {
    int i = className.lastIndexOf('.');
    String pkg = (i < 0) ? "" : className.substring(0, i);
    return Optional.ofNullable(exports.get(pkg));
  }
}

final class PluginClassLoader extends URLClassLoader {
  static { ClassLoader.registerAsParallelCapable(); }

  private final PluginIndex index;
  private final Set<String> parentFirstPrefixes; // e.g., ["java.", "jakarta.",
"com.myengine."]
  private final Set<String> myExports;

  PluginClassLoader(URL[] urls, ClassLoader parent, PluginIndex index,
                    Set<String> parentFirstPrefixes, Set<String> myExports) {
    super(urls, parent);
    this.index = index;
    this.parentFirstPrefixes = parentFirstPrefixes;
    this.myExports = myExports;
    index.register(this, myExports);
  }

  @Override protected Class<?> loadClass(String name, boolean resolve) throws
ClassNotFoundException {
    final Object lock = getClassLoadingLock(name);
    synchronized (lock) {
      Class<?> c = findLoadedClass(name);
      if (c != null) return c;
```

```
      boolean parentFirst =
parentFirstPrefixes.stream().anyMatch(name::startsWith);
      if (parentFirst) {
        try { c = super.loadClass(name, false); if (resolve) resolveClass(c); return c; }
        catch (ClassNotFoundException ignore) {}
      } else {
        try {
          c = defineLocallyWithBroker(name);           // child-first
          if (resolve) resolveClass(c);
          return c;
        } catch (ClassNotFoundException ignore) {
          var owner = index.ownerOf(name);
          if (owner.isPresent() && owner.get() != this) {
            c = owner.get().loadClass(name, false);     // direct cross-plugin hop
            if (resolve) resolveClass(c);
            return c;
          }
        }
      }
      c = super.loadClass(name, false);               // fallback to parent
      if (resolve) resolveClass(c);
      return c;
    }
  }

  private Class<?> defineLocallyWithBroker(String name) throws
ClassNotFoundException {
    return ClassLoadingBroker.defineOnce(name, () -> {
      // don't split exported ownership
      var owner = index.ownerOf(name);
      if (owner.isPresent() && owner.get() != this) return
owner.get().loadClass(name, false);
      return findClass(name); // define only if actually present in this JAR
    });
  }
}
```

Reasoning to interviewer:

- The selective delegation keeps platform classes unique while letting plugins bring their own libs safely.
- The broker makes class definition idempotent across loaders; races collapse into the same Class<?>.
- Parallel-capable + per-class locks remove the “one giant loader lock” that typically deadlocks on cycles.

- The export index prevents O(N) probing across loaders; cross-plugin references become a single directed hop.
- One loader per plugin allows future hot-redeploy/unload without JVM restarts.

Future scope:

- Enforce declared exports/uses in plugin manifests; reject split packages at registration time.
- Add metrics/tracing around class resolution to catch slow or cyclic chains early.
- Support versioned side-by-side plugins by scoping exports with a version key.
- Add unload hooks and reference tracking for safe plugin replacement at runtime.

Scenario 46: Using Java Platform Modules (JPMS) for a service-oriented codebase

Payments is split into payments.api (contracts), ledger (implementation), and settlement (implementation). Only the API should be consumable; internals must stay hidden.

Answer:

- Keep the API tiny and exported. Implementations depend on it but export nothing.
- Use provides … with … in impls and uses in a composition layer if you want ServiceLoader-based wiring.
- Open only the minimal packages (if needed) to specific frameworks for reflection.

Code:

module-info.java (API):

```
module com.payments.api {
  exports com.payments.api;                  // interfaces + DTOs only
  uses com.payments.api.PaymentProcessor;    // optional: discovery
}
```

API surface example:

```
package com.payments.api;
import java.math.BigDecimal;
import java.util.UUID;

public interface PaymentProcessor {
  PaymentResult authorize(UUID merchantId, BigDecimal amount, String
currency);
  PaymentResult capture(UUID authId);
  PaymentResult refund(UUID paymentId, BigDecimal amount);
}
public record PaymentResult(UUID id, boolean success, String message) {}
```

module-info.java (ledger impl):

```
module com.payments.ledger {
  requires com.payments.api;
  requires java.sql;
```

```
  provides com.payments.api.PaymentProcessor
      with com.payments.ledger.LedgerPaymentProcessor;

  // Only if needed for JSON/etc.; restrict scope
  opens com.payments.ledger.serialization to com.fasterxml.jackson.databind;
}
```

module-info.java (settlement impl):

```
module com.payments.settlement {
  requires com.payments.api;
  requires java.net.http;

  provides com.payments.api.PaymentProcessor
      with com.payments.settlement.SettlementPaymentProcessor;
}
```

Composition/app module (depends on API only):

```
module com.payments.app {
  requires com.payments.api;
  uses com.payments.api.PaymentProcessor;
}
```

Reasoning to interviewer:

- JPMS hides everything by default; exporting only API packages prevents accidental imports of impl classes at compile time.
- uses/provides cleanly inverts control: consumers depend on the interface, implementations register themselves.
- Narrow opens avoids blanket reflection exposure while keeping frameworks happy.
- The module graph documents dependencies and prevents cycles, stabilizing builds.

Future scope:

- Add jdeps and Revapi checks in CI to block illegal accesses and accidental API breaks.
- Introduce a facade module that re-exports only approved parts of com.payments.api with requires transitive where convenient.

- Version com.payments.api and enforce a compatibility policy to keep downstreams safe.
- Use jlink to ship a trimmed runtime image containing just the needed modules.

Scenario 47: Replacing verbose DTOs with Records

Services exchange simple, immutable carriers (customer profile, shipment summary, invoice line). Boilerplate POJOs (getters/setters/equals/hashCode/toString) obscure intent and invite mistakes.

Answer:

- Convert DTOs to records: components are final, accessors are generated, and equals/hashCode/toString come for free.
- Use a compact constructor to enforce invariants and wrap mutable inputs (collections) with defensive copies—records are shallowly immutable.
- Serialization: modern JSON mappers (Jackson 2.12+/Spring Boot 3+) bind to the canonical constructor out of the box. For JPA, keep entities as classes; use records for projections/REST/Kafka.
- If component names differ from JSON fields, use annotations (e.g., @JsonProperty) on record components.

Code:

```
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public record InvoiceLine(String sku, int quantity, BigDecimal unitPrice,
List<String> tags) {
  public InvoiceLine {
    if (sku == null || sku.isBlank()) throw new IllegalArgumentException("sku
required");
    if (quantity <= 0) throw new IllegalArgumentException("quantity > 0");
    if (unitPrice == null || unitPrice.signum() < 0) throw new
IllegalArgumentException("unitPrice >= 0");
    tags = (tags == null) ? List.of() : List.copyOf(tags);   // defensive copy
  }
  public BigDecimal lineTotal() { return
unitPrice.multiply(BigDecimal.valueOf(quantity)); }
}

public record CustomerProfile(String id, String email, Map<String, String> prefs) {
  public CustomerProfile {
    if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
    if (email == null || email.isBlank()) throw new IllegalArgumentException("email
required");
    prefs = (prefs == null) ? Map.of() : Map.copyOf(prefs);
```

```
  }
}

// Spring Data projection example (entity → DTO record)
// @Query("select new com.example.ShipmentSummary(s.id, s.status, s.eta) from
Shipment s")
public record ShipmentSummary(String id, String status, String eta) {}
```

Reasoning to interviewer:

- Records express “this is data” in the type system and remove repetitive code, improving readability and correctness.
- The compact constructor centralizes validation and ensures callers can’t mutate internal state through shared lists/maps.
- JSON mappers use constructor binding, so we avoid reflection-heavy setters; schemas stay stable and explicit.
- JPA entities stay as classes (no-arg ctor, proxies) while boundaries (REST/events/projections) benefit from records’ immutability and concision.

Future scope:

- Add Bean Validation annotations on components (@NotBlank, @Positive) for uniform checks.
- Introduce value-type records for common primitives (Money, Email) to reduce validation drift.
- Generate OpenAPI/JSON Schema from record DTOs; add contract tests to detect drift.
- Standardize “DTO guidelines” (defensive copies, invariants, naming) to keep the codebase consistent.

Scenario 48: Cleaner type-specific logic using instanceof pattern matching

We route generic Object payloads differently for Integer counters, String commands, and a domain type Price. The old style repeats instanceof then casts, which is noisy and brittle.

Answer:

- Pattern matching binds and narrows in one step: if (obj instanceof Price p) { ... }.
- It removes redundant casts, keeps variables scoped to the branch, and makes intent obvious.
- With Java 21, a type-pattern switch can make the dispatch even clearer and non-fallthrough.

Code:

```
// Domain value
public record Price(java.math.BigDecimal amount, String currency) {
    public Price {
        if (amount == null || amount.signum() < 0) throw new
IllegalArgumentException("amount >= 0");
        if (currency == null || currency.isBlank()) throw new
IllegalArgumentException("currency required");
    }
}

public final class Parser {
    // Old (noisy) approach
    public String routeOld(Object payload) {
        if (payload instanceof Integer) {
            Integer i = (Integer) payload;
            return "COUNTER=" + i;
        } else if (payload instanceof String) {
            String s = (String) payload;
            return "CMD=" + s.trim().toUpperCase();
        } else if (payload instanceof Price) {
            Price p = (Price) payload;
            return "PRICE=" + p.amount() + " " + p.currency();
        }
        return "UNSUPPORTED";
    }

    // Java 17+ instanceof pattern matching
    public String route(Object payload) {
```

```
        if (payload instanceof Integer i)      return "COUNTER=" + i;
        if (payload instanceof String s)       return "CMD=" + s.trim().toUpperCase();
        if (payload instanceof Price p)        return "PRICE=" + p.amount() + " " +
p.currency();
        return "UNSUPPORTED";
    }

    // Java 21+ type-pattern switch (even cleaner)
    public String routeSwitch(Object payload) {
        return switch (payload) {
            case Integer i -> "COUNTER=" + i;
            case String s  -> "CMD=" + s.trim().toUpperCase();
            case Price p   -> "PRICE=" + p.amount() + " " + p.currency();
            case null      -> "UNSUPPORTED";
            default        -> "UNSUPPORTED";
        };
    }
}
```

Reasoning to interviewer:

- Pattern matching narrows and binds in the same expression; no chance of mismatching the instanceof type and the cast type after refactors.
- Control flow reads like a router by “shape” of data, not by casting ceremony, which cuts review time and reduces ClassCastException risk.
- A switch on type patterns is exhaustive and non-fallthrough, so adding a new domain type is a compiler-enforced change.

Future scope:

- Use a sealed hierarchy (e.g., Message permits Counter, Command, PriceMsg) to remove the default arm entirely and force exhaustive handling.
- Log matched arms for observability (traffic mix), and add metrics for tuning throughput.
- Push parsing of external bytes/JSON into typed records first, then route on types—less “Object”-handling in core code.

Scenario 49: Using switch pattern matching for heterogeneous inputs

We normalize mixed inputs (numbers, strings, URIs). Current code is a switch + scattered if/else chains.

Answer:

- Use a Java 21 type-pattern switch so each arm both matches and binds the variable.
- Include a case null to make null-handling explicit.
- Order specific cases before broader ones (e.g., Double/BigDecimal before Number) to avoid accidental capture.
- Since inputs are open-ended, keep a conservative default; with a sealed set, you'd omit it.

Code:

```
import java.math.BigDecimal;
import java.net.URI;

public final class Normalizer {
    public static String normalize(Object in) {
        return switch (in) {
            case null              -> "null";
            case Integer i         -> String.valueOf(i);
            case Long l            -> String.valueOf(l);
            case Double d          -> strip(BigDecimal.valueOf(d));
            case BigDecimal bd     -> strip(bd);
            case String s          -> s.trim();
            case URI uri           -> uri.normalize().toString();
            // broader fallback for other numeric wrappers
            case Number n          -> n.toString();
            default                -> in.toString();
        };
    }
    private static String strip(BigDecimal bd) {
        return bd.stripTrailingZeros().toPlainString();
    }
}
```

Reasoning to interviewer:

- Each arm declares intent and binds a typed variable, eliminating separate casts and reducing branching noise.
- Explicit case null prevents hidden NPEs and documents policy.
- Specific-over-general ordering avoids the Number arm swallowing Double/BigDecimal logic.
- As a switch expression, there’s no fall-through; each arm must yield a value, which the compiler checks.

Future scope:

- Add domain-specific arms (e.g., Email, Amount) as records to push validation into types.
- If the input set becomes closed, introduce a sealed interface and remove the default arm for compiler-checked exhaustiveness.
- Add small counters per arm to detect unexpected shapes in production.

Scenario 50: Accelerating numeric kernels with the Vector API

A risk engine hits CPU limits on array math (scale, axpy, dot). Scalar loops leave SIMD lanes idle.

Answer:

- Use the Java Vector API to operate on “species-sized” chunks at a time.
- Choose the preferred species for the element type (FloatVector/DoubleVector with SPECIES_PREFERRED).
- Loop by i += species.length(): load (fromArray), compute (mul, add, fma), store (intoArray).
- Handle tails with masks (indexInRange) so any length works.
- Use reduceLanes for sums and fma where available (fused multiply-add).

Code:

```
// Use jdk.incubator.vector.* on older JDKs; jdk.vector.* once finalized.
import jdk.incubator.vector.*;

public final class Kernels {
    private static final DoubleVector.Species S =
DoubleVector.SPECIES_PREFERRED;

    // y = a*x + y  (AXPY)
    public static void axpy(double a, double[] x, double[] y) {
        int n = Math.min(x.length, y.length);
        int i = 0, ub = S.loopBound(n);
        var va = DoubleVector.broadcast(S, a);
        for (; i < ub; i += S.length()) {
            var vx = DoubleVector.fromArray(S, x, i);
            var vy = DoubleVector.fromArray(S, y, i);
            vx.fma(va, vy).intoArray(y, i);       // y = a*x + y
        }
        for (; i < n; i++) y[i] = a * x[i] + y[i]; // tail
    }

    // scale: x[i] *= a
    public static void scale(double a, double[] x) {
        int i = 0, ub = S.loopBound(x.length);
        var va = DoubleVector.broadcast(S, a);
        for (; i < ub; i += S.length()) {
            var vx = DoubleVector.fromArray(S, x, i);
            vx.mul(va).intoArray(x, i);
        }
```

```
        if (i < x.length) {
            var m  = S.indexInRange(i, x.length);        // mask
            var vx = DoubleVector.fromArray(S, x, i, m);
            vx.mul(va).intoArray(x, i, m);
        }
    }

    // dot product
    public static double dot(double[] a, double[] b) {
        int n = Math.min(a.length, b.length);
        int i = 0, ub = S.loopBound(n);
        var acc = DoubleVector.zero(S);
        for (; i < ub; i += S.length()) {
            var va = DoubleVector.fromArray(S, a, i);
            var vb = DoubleVector.fromArray(S, b, i);
            acc = va.fma(vb, acc);                       // acc += va*vb
        }
        double sum = acc.reduceLanes(VectorOperators.ADD);
        for (; i < n; i++) sum += a[i] * b[i];           // tail
        return sum;
    }
}
```

Reasoning to interviewer:

- SPECIES_PREFERRED picks the widest SIMD width available at runtime (AVX2/AVX-512/NEON), keeping code portable and fast.
- Loads/stores and lane ops are JIT-lowered to hardware SIMD; we don't micromanage alignment.
- Masks make tails branch-light and safe; reduceLanes does efficient horizontal reductions.
- fma often improves both speed and numeric accuracy for accumulation-heavy kernels.

Future scope:

- Use FloatVector for bandwidth-bound paths; keep DoubleVector for numerically sensitive reductions.
- Tile and parallelize: split arrays into cache-sized tiles across threads; keep SIMD inside each tile.
- Fuse passes (e.g., compute sum and sum-of-squares together) to cut memory traffic.

- Add JMH baselines and run on target CPU families to catch regressions across JDK upgrades.

Scenario 51: Reducing object churn in a stock-trading engine

A trading platform ingests millions of orders daily. Short-lived objects (Price, Order, calculation helpers) flood the heap, spiking GC and hurting latency.

Answer:

- Reuse where safe: thread-local builders (StringBuilder, DecimalFormat) and object pools for high-churn types (e.g., OrderEvent).
- Prefer primitives over wrappers to avoid boxing (int vs Integer, double vs Double).
- Use StringBuilder or Formatter instead of repeated string concatenations in loops.
- Design immutable value objects (e.g., Price) as records but keep them light and cache common constants (e.g., Price.ZERO).
- For serialization/deserialization, use ByteBuffer or pooled buffers instead of allocating new byte arrays for every message.

Code:

```
// Thread-local builder reuse
final class Builders {
  private static final ThreadLocal<StringBuilder> TL = ThreadLocal.withInitial(() ->
new StringBuilder(256));
  static String formatTrade(String id, double price, int qty) {
    var sb = TL.get(); sb.setLength(0);
    return sb.append("id=").append(id)
             .append(",p=").append(price)
             .append(",q=").append(qty)
             .toString(); // only final String is new
  }
}

// Object pool for hot types
final class OrderEvent {
  String id; double px; int qty;
  void reset(String id, double px, int qty) { this.id=id; this.px=px; this.qty=qty; }
}
final class Pool {
  private final ArrayBlockingQueue<OrderEvent> q = new
ArrayBlockingQueue<>(1000);
  OrderEvent borrow(String id, double px, int qty) {
    OrderEvent e = q.poll();
    if (e == null) e = new OrderEvent();
```

```
    e.reset(id, px, qty); return e;
  }
  void release(OrderEvent e) { q.offer(e); }
}
```

Reasoning to interviewer:

- Thread-local builders cut transient allocations in logging/formatting paths.
- Pools amortize allocation costs for hot types but must be bounded to avoid memory leaks.
- Avoiding boxing saves both allocations and cache pressure in numeric hot loops.
- Immutable lightweight records (Price, TradeId) reduce bugs without bloating GC pressure.

Future scope:

- Profile allocations with JFR/GC logs; target the top allocation sites.
- Consider off-heap buffers (e.g., Agrona) for extreme throughput.
- Introduce zero-GC serialization (FlatBuffers, SBE) for wire formats.
- Explore -XX:+UseStringDeduplication if many duplicate symbols flood the heap.

Scenario 52: Modularizing a university management system with JPMS

Modules: courses, students, enrollment. Each should expose stable APIs but hide internals.

Answer:

- Split each domain into *.api and *.impl modules. API exports only interfaces and DTOs; impls provide them via provides.
- Shared value types (StudentId, CourseId) go into university.common.
- Enrollment depends on APIs of courses and students, not their impls.

Code:

```
// university.common
module university.common {
  exports university.common; // ids, shared DTOs
}

// students.api
module university.students.api {
  requires transitive university.common;
  exports university.students.api;
  uses university.students.api.StudentService;
}

// students.impl
module university.students.impl {
  requires university.students.api;
  provides university.students.api.StudentService
      with university.students.impl.StudentServiceImpl;
}

// courses.api
module university.courses.api {
  requires transitive university.common;
  exports university.courses.api;
  uses university.courses.api.CourseService;
}

// courses.impl
module university.courses.impl {
  requires university.courses.api;
  provides university.courses.api.CourseService
      with university.courses.impl.CourseServiceImpl;
}
```

```
// enrollment.api
module university.enrollment.api {
  requires transitive university.common;
  requires transitive university.students.api;
  requires transitive university.courses.api;
  exports university.enrollment.api;
  uses university.enrollment.api.EnrollmentService;
}

// enrollment.impl
module university.enrollment.impl {
  requires university.enrollment.api;
  requires university.students.api;
  requires university.courses.api;
  provides university.enrollment.api.EnrollmentService
      with university.enrollment.impl.EnrollmentServiceImpl;
}
```

Reasoning to interviewer:

- JPMS hides everything by default—only explicitly exported packages are visible.
- By separating API/impl, other modules can't accidentally depend on internals.
- uses/provides allows discovery and wiring via ServiceLoader, eliminating hard-coded impl dependencies.
- requires transitive lets enrollment re-export student/course APIs to its consumers cleanly.

Future scope:

- Add jdeps/Revapi in CI to catch illegal dependencies.
- Restrict reflection with narrow opens only for frameworks (e.g., Jackson, Hibernate).
- Provide facade modules (like university.admin) that re-export specific APIs for integration points.
- Use jlink to bundle trimmed runtime images for deployments.

Scenario 53: Avoiding deadlocks in a hotel booking system

Booking threads lock both room allocation and payment authorization. Different acquisition order risks deadlock.

Answer:

- Always acquire locks in a global, consistent order (e.g., RoomLock before PaymentLock).
- Alternatively, use tryLock(timeout) to back off instead of waiting indefinitely.
- Prefer higher-level concurrency constructs (e.g., StampedLock for read-heavy state, or transactional queues/actors) instead of manual lock composition.
- Reduce nested locking by designing workflows that split room reservation and payment into separate atomic steps with compensations (sagas).

Code:

```
import java.util.concurrent.locks.*;

final class Booking {
  private final Lock roomLock = new ReentrantLock();
  private final Lock payLock  = new ReentrantLock();

  void bookRoom(String user) throws InterruptedException {
    // consistent order: room → payment
    roomLock.lock();
    try {
      payLock.lock();
      try {
        System.out.println("Allocating + charging for " + user);
      } finally { payLock.unlock(); }
    } finally { roomLock.unlock(); }
  }

  boolean tryBookRoom(String user) throws InterruptedException {
    if (roomLock.tryLock(100, java.util.concurrent.TimeUnit.MILLISECONDS)) {
      try {
        if (payLock.tryLock(100, java.util.concurrent.TimeUnit.MILLISECONDS)) {
          try {
            System.out.println("Allocated + charged for " + user);
            return true;
          } finally { payLock.unlock(); }
        }
      } finally { roomLock.unlock(); }
    }
```

```
    return false; // fallback / retry
  }
}
```

Reasoning to interviewer:

- Consistent lock ordering prevents cycles by construction.
- tryLock with timeout avoids indefinite waits—threads can retry or abort gracefully.
- Breaking workflows into smaller atomic units (reserve room, then charge, with compensation if payment fails) reduces multi-lock spans entirely.
- Actor-like per-room executors eliminate explicit locking and serialize operations per resource.

Future scope:

- Add monitoring for lock contention and detect potential deadlock paths in testing (thread dumps, deadlock detectors).
- Switch to optimistic concurrency (StampedLock) for read-heavy flows like availability queries.
- For distributed booking across services, use sagas or distributed transactions to avoid global locks.
- Integrate metrics/logging on lock wait times to spot bottlenecks before they escalate.

Scenario 54: Restricting inheritance with Sealed Classes in a transit app

Only Bus, Train, and Metro are valid vehicle categories. The base type must forbid any other subclasses.

Answer:

I'd model Transport as a sealed interface (or abstract class) and explicitly permit only Bus, Train, and Metro. Make those concrete types final (or non-sealed if you want a controlled extension point). This encodes the business rule in the type system and enables exhaustive pattern matching.

Code:

```
// Java 17+
public sealed interface Transport permits Bus, Train, Metro {
    String routeCode();
    int capacity();
}

public final class Bus implements Transport {
    private final String routeCode; private final int capacity;
    public Bus(String routeCode, int capacity) { this.routeCode = routeCode;
this.capacity = capacity; }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

public final class Train implements Transport {
    private final String routeCode; private final int capacity;
    public Train(String routeCode, int capacity) { this.routeCode = routeCode;
this.capacity = capacity; }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

public final class Metro implements Transport {
    private final String routeCode; private final int capacity;
    public Metro(String routeCode, int capacity) { this.routeCode = routeCode;
this.capacity = capacity; }
    public String routeCode() { return routeCode; }
    public int capacity() { return capacity; }
}

// Exhaustive usage (Java 21 pattern switch)
int baseFare(Transport t) {
    return switch (t) {
```

```
        case Bus b   -> 10;
        case Train tr-> 15;
        case Metro m -> 12;
    };
}
```

Reasoning to interviewer:

- sealed … permits … closes the hierarchy at compile time; the compiler rejects any other subtype.
- Marking leaves final prevents further subclassing; use non-sealed only where you intend an internal branch to grow.
- Exhaustive switches become safer—missing a case fails compilation rather than at runtime.

Future scope:

- Place all permitted types in the same JPMS module to keep boundaries tight.
- Convert simple variants to records for immutability and concise code.
- Add factory methods to validate route codes and capacities at construction time.

Scenario 55: Capturing method execution time with dynamic proxies

You need per-method latency metrics without modifying the business classes that implement your scheduling interfaces.

Answer:

JDK dynamic proxies can wrap any interface implementation. The InvocationHandler records System.nanoTime() before/after method.invoke, logs the duration, and rethrows the original exception. You can scope timing to annotated methods (e.g., @Timed) to avoid noise.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.concurrent.TimeUnit;

@Retention(RetentionPolicy.RUNTIME) @Target({ElementType.METHOD,
ElementType.TYPE})
@interface Timed { String value() default ""; }

interface TaskRunner {
    void run(String taskId);
    @Timed("heavy-op") String compute(String input);
}

final class TaskRunnerImpl implements TaskRunner {
    public void run(String taskId) { /* work */ }
    public String compute(String input) { return input.toUpperCase(); }
}

final class TimingProxy implements InvocationHandler {
    private final Object target;
    private TimingProxy(Object target) { this.target = target; }

    @SuppressWarnings("unchecked")
    static <T> T wrap(Class<T> api, T target) {
        return (T) Proxy.newProxyInstance(api.getClassLoader(), new
Class<?>[]{api}, new TimingProxy(target));
    }

    public Object invoke(Object proxy, Method m, Object[] args) throws Throwable {
        boolean timed = m.isAnnotationPresent(Timed.class) ||
target.getClass().isAnnotationPresent(Timed.class);
        if (!timed) return m.invoke(target, args);
```

```
        String metric = m.getAnnotation(Timed.class) != null ?
m.getAnnotation(Timed.class).value() : m.getName();
        long start = System.nanoTime();
        try {
            return m.invoke(target, args);
        } catch (InvocationTargetException ite) {
            throw ite.getCause();
        } finally {
            long ms = TimeUnit.NANOSECONDS.toMillis(System.nanoTime() - start);
            System.out.printf("metric=%s method=%s durationMs=%d%n", metric,
m.getName(), ms);
        }
    }
}

// Usage
TaskRunner runner = TimingProxy.wrap(TaskRunner.class, new
TaskRunnerImpl());
runner.run("T-1");                // not timed
runner.compute("abc");            // timed and logged
```

Reasoning to interviewer:

- Proxies intercept at the interface boundary, so cross-cutting concerns (timing, tracing) are added with zero changes to business logic.
- Using an annotation keeps the timing surface explicit and avoids flooding logs.
- Unwrapping InvocationTargetException preserves original stack traces for debuggability.

Future scope:

- Emit metrics via Micrometer and sample in hot paths.
- Cache “isTimed/metricName” decisions per Method to cut reflection overhead.
- For classes without interfaces, switch to Byte Buddy/CGLIB or DI container AOP.

Scenario 56: Cleaner query handling with switch pattern matching

Search can arrive as keyword String, numeric ID, or UUID. The if-else + casts are repetitive and error-prone.

Answer:

Use Java 21’s pattern matching switch on Object. Each case both checks and binds, eliminating separate casts. Handle null explicitly, and keep a strict default for unsupported shapes.

Code:

```
import java.util.*;
import java.util.UUID;

interface ProductRepo {
    Optional<Product> findById(long id);
    Optional<Product> findByUuid(UUID id);
    List<Product> searchByKeyword(String keyword);
}
final class Product { /* ... */ }

final class QueryService {
    private final ProductRepo repo;
    QueryService(ProductRepo repo) { this.repo = repo; }

    public Object handle(Object query) {
        return switch (query) {
            case null           -> List.of(); // or throw IllegalArgumentException
            case String s       -> {
                String q = s.trim();
                if (q.isEmpty()) yield List.of();
                try {
                    UUID u = UUID.fromString(q);
                    yield repo.findByUuid(u).map(List::of).orElseGet(List::of);
                } catch (IllegalArgumentException ex) {
                    yield repo.searchByKeyword(q);
                }
            }
            case Integer i      -> repo.findById(i.longValue()).orElse(null);
            case Long id        -> repo.findById(id).orElse(null);
            case UUID u         -> repo.findByUuid(u).orElse(null);
            default             -> throw new IllegalArgumentException("Unsupported: " +
query.getClass());
        };
    }
}
```

Reasoning to interviewer:

- The switch is exhaustive and non-fallthrough; each arm declares exactly how to handle that type.
- Binding in-place (case UUID u ->) avoids cast drift and reduces ClassCastException risks during refactors.
- Treating String specially lets you support both keyword searches and UUID-in-string transparently.

Future scope:

- Close the set with a sealed Query hierarchy (ById, ByUuid, ByKeyword) and remove the default arm for compiler-enforced exhaustiveness.
- Add rate limiting/authorization per arm (ID lookups might be privileged).
- Log per-arm counters to understand user behavior and tune indexes.

Scenario 57: Accelerating climate simulation with the Vector API

Big numeric arrays (temperature, humidity, wind) are bandwidth-bound; scalar loops leave SIMD lanes idle. I'd vectorize hot kernels so each iteration processes a whole "species" of lanes.

Answer:

- Pick the preferred species for the element type (FloatVector or DoubleVector with SPECIES_PREFERRED).
- Vector loop: load fromArray → compute (mul, add, fma) → store intoArray.
- Handle the remainder with a mask (indexInRange).
- Use reduceLanes for sums/means and fma for accuracy/speed.

Code:

```
// If your JDK uses the incubator: import jdk.incubator.vector.*;
// Otherwise: import jdk.vector.*;
import jdk.incubator.vector.*;

public final class ClimateKernels {
  private static final FloatVector.Species S = FloatVector.SPECIES_PREFERRED;

  // scale: T[i] = a * T[i]
  public static void scale(float a, float[] T) {
    var va = FloatVector.broadcast(S, a);
    int i = 0, ub = S.loopBound(T.length);
    for (; i < ub; i += S.length()) {
      FloatVector.fromArray(S, T, i).mul(va).intoArray(T, i);
    }
    if (i < T.length) {
      var m = S.indexInRange(i, T.length);
      FloatVector.fromArray(S, T, i, m).mul(va).intoArray(T, i, m);
    }
  }

  // y = a*x + y (FMA where available)
  public static void axpy(float a, float[] x, float[] y) {
    var va = FloatVector.broadcast(S, a);
    int n = Math.min(x.length, y.length), i = 0, ub = S.loopBound(n);
    for (; i < ub; i += S.length()) {
      var vx = FloatVector.fromArray(S, x, i);
      var vy = FloatVector.fromArray(S, y, i);
      vx.fma(va, vy).intoArray(y, i);
    }
    for (; i < n; i++) y[i] = a * x[i] + y[i];
```

```
  }

  // mean temperature
  public static float mean(float[] T) {
    int i = 0, ub = S.loopBound(T.length);
    var acc = FloatVector.zero(S);
    for (; i < ub; i += S.length()) acc = acc.add(FloatVector.fromArray(S, T, i));
    float sum = acc.reduceLanes(VectorOperators.ADD);
    for (; i < T.length; i++) sum += T[i];
    return sum / T.length;
  }
}
```

Reasoning to interviewer:

- SPECIES_PREFERRED maps to AVX/AVX-512/NEON widths at runtime—portable speedups.
- Masks keep tails correct without branchy code.
- fma improves both throughput and numerical stability in accumulation-heavy models.

Future scope:

- Use DoubleVector for sensitive reductions, FloatVector for bulk fields.
- Tile 2D/3D grids to improve cache locality; parallelize tiles with ForkJoin while keeping SIMD inside each tile.
- Fuse passes (e.g., sum and sum-of-squares in one sweep) to cut memory traffic; baseline with JMH.

Scenario 58: Securing deserialization in a payment-messaging service

Serialized payloads cross service boundaries; you must default-deny and allow only known DTOs, with structural limits to stop resource bombs.

Answer:

- Attach a context-specific ObjectInputFilter to every ObjectInputStream at the boundary.
- Allow only DTO packages/classes; reject proxies/JDK internals.
- Cap maxdepth, maxarray, and maxrefs.
- Optionally set a restrictive global backstop filter.

Code (per-stream filter):

```
import java.io.*;
import java.util.Set;

public final class DeserFilters {
  private static final String DTO_PKG = "com.payments.dto.";
  private static final Set<String> ALSO_OK = Set.of("java.util.ArrayList",
"java.util.HashMap");

  public static ObjectInputFilter inboundFilter() {
    return info -> {
      if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
      if (info.arrayLength() != -1 && info.arrayLength() > 500_000) return
ObjectInputFilter.Status.REJECTED;
      if (info.references()  != -1 && info.references()  > 1_000_000) return
ObjectInputFilter.Status.REJECTED;

      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitives
      String n = cl.getName();

      if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
      if (n.startsWith("sun.") || n.startsWith("jdk.") || n.startsWith("java.lang.invoke.")
          || n.startsWith("java.rmi.")) return ObjectInputFilter.Status.REJECTED;

      return (n.startsWith(DTO_PKG) || ALSO_OK.contains(n))
          ? ObjectInputFilter.Status.ALLOWED
          : ObjectInputFilter.Status.REJECTED;
    };
  }
```

```
  // Optional global backstop
  public static void installGlobalBackstop() {
    ObjectInputFilter.Config.setSerialFilter(info -> {
      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
      return cl.getName().startsWith(DTO_PKG)
          ? ObjectInputFilter.Status.ALLOWED
          : ObjectInputFilter.Status.REJECTED;
    });
  }
}

// Usage at the boundary:
try (ObjectInputStream ois = new ObjectInputStream(in)) {
  ois.setObjectInputFilter(DeserFilters.inboundFilter());
  Object obj = ois.readObject(); // rejected unless allowlisted + within limits
}
```

Alternative (pattern string):

```
ObjectInputFilter f = ObjectInputFilter.Config.createFilter(

"maxdepth=20;maxarray=500000;maxrefs=1000000;com.payments.dto.*;java.util.*
;!*"
);
ois.setObjectInputFilter(f);
```

Reasoning to interviewer:

- Filters run before object graph instantiation—gadget chains are stopped early.
- Default-deny with a tight allowlist plus structural caps addresses both code-execution and resource-exhaustion vectors.
- Per-stream filters let each topic define valid DTOs; the global backstop covers omissions.

Future scope:

- Prefer schema formats (JSON/Avro/Protobuf) with strict mappers; reserve Java serialization for trusted hops.
- CI tests that attempt to deserialize known-bad classes; externalize allowlists for safe rollout.
- Pair with mTLS or signed messages for defense-in-depth.

Scenario 59: Simplifying application settings with Records

Config objects (service name, version, environment) are immutable by policy; POJOs are bloated with boilerplate. Records express “data-only + immutable” cleanly.

Answer:

- Make configs records; components are final with generated accessors/equals/hashCode/toString.
- Validate in the compact constructor; defensively copy collections to keep effective immutability.
- Provide tiny value types/enums (Env, SemVer) for clarity.
- Supply convenience factories (from env vars/Properties).

Code:

```
import java.util.*;
import java.util.stream.Collectors;

public enum Env { DEV, STAGE, PROD }

public record ServiceConfig(
    String serviceName,
    String version,
    Env environment,
    Map<String, String> extras // optional K/Vs like CFG_*
) {
  public ServiceConfig {
    if (serviceName == null || serviceName.isBlank()) throw new
IllegalArgumentException("serviceName required");
    if (version == null || version.isBlank()) throw new
IllegalArgumentException("version required");
    if (environment == null) throw new IllegalArgumentException("environment
required");
    extras = extras == null ? Map.of()
           : Collections.unmodifiableMap(
               extras.entrySet().stream()
                   .collect(Collectors.toMap(
                     e -> e.getKey().trim(),
                     e -> e.getValue() == null ? "" : e.getValue().trim()
                   )));
  }

  public static ServiceConfig fromEnv(Map<String,String> env) {
    String name = env.getOrDefault("SERVICE_NAME", "unknown");
```

```
    String ver  = env.getOrDefault("SERVICE_VERSION", "0.0.0");
    Env e = Env.valueOf(env.getOrDefault("SERVICE_ENV",
"DEV").toUpperCase(Locale.ROOT));
    Map<String,String> extras = env.entrySet().stream()
      .filter(it -> it.getKey().startsWith("CFG_"))
      .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
    return new ServiceConfig(name, ver, e, extras);
  }

  // "with" convenience
  public ServiceConfig withVersion(String newVersion) {
    return new ServiceConfig(serviceName, newVersion, environment, extras);
  }
}
```

Reasoning to interviewer:

- Records cut boilerplate while enforcing immutability by construction; there are no setters and components are final.
- The compact constructor centralizes validation and wraps maps/lists so callers can’t mutate internals.
- Constructor binding plays well with modern frameworks (e.g., Spring Boot constructor binding for config).

Future scope:

- Add Bean Validation annotations (@NotBlank, @Pattern) on components for uniform checks.
- Standardize value types (record ServiceName, record SemVer) for stronger domain guarantees.
- Serialize records directly to JSON for audit snapshots; diff configs safely during rollout.

Scenario 60: Thread-safe random numbers in an online gaming app

You need fast, independent RNG streams for dice rolls and loot drops on many threads. A single shared Random is a contention hotspot.

Answer

- I'll use Java 17's java.util.random framework: choose a modern generator via RandomGeneratorFactory, then give each thread its own instance (no shared state, no locks).
- For reproducible matches, I'll derive independent substreams using Jumpable/Leapable so players/shards don't overlap in sequences.
- I'll use bounded methods like nextInt(origin, bound) to avoid modulo bias and keep intent explicit.

Code

```
import java.util.random.*;

public final class Rngs {
  private static final RandomGeneratorFactory<? extends RandomGenerator> F =
      RandomGeneratorFactory.of("L64X128MixRandom"); // fast, high-quality LXM

  // Per-thread RNG: zero contention
  private static final ThreadLocal<RandomGenerator> TL =
      ThreadLocal.withInitial(() -> F.create(seed()));

  private static long seed() {
    // For reproducible sessions, inject a fixed seed instead of time-based
    return System.nanoTime() ^ Thread.currentThread().getId();
  }

  // Game utilities
  public static int rollDice() {
    return TL.get().nextInt(1, 7); // 1..6 inclusive
  }
  public static boolean lootDrop(double p) {
    return TL.get().nextDouble() < p; // Bernoulli(p)
  }

  // Deterministic, non-overlapping stream for a match/shard
  public static RandomGenerator forMatch(long matchSeed, int streamIndex) {
    RandomGenerator g = F.create(matchSeed);
    if (g instanceof RandomGenerator.Jumpable j) {
      for (int i = 0; i < streamIndex; i++) g = j.jump();
```

```
      return g; // each index → disjoint substream
    }
    // Fallback: mixed seed
    long mixed = matchSeed ^ (0x9E3779B97F4A7C15L * streamIndex);
    return F.create(mixed);
  }
}
```

Reasoning to interviewer

- Per-thread generators eliminate contention entirely and scale linearly with cores.
- LXM generators support jump() so I can deterministically partition randomness by player/match—great for fairness and replay.
- The new API cleanly separates algorithm choice from usage; swapping generators is a one-liner.
- Bounded methods (nextInt(a,b)) avoid biased arithmetic and keep call sites readable.

Future scope

- Use SecureRandom only for security-sensitive tokens; keep gameplay on fast PRNGs.
- Centralize seeding policy and log seeds for deterministic replays.
- Add property tests/stat checks to catch accidental RNG reuse across threads.
- Expose RNGs via DI so test environments can inject deterministic generators.

Scenario 61: GC tuning for a live sports-streaming analytics app

Continuous player stats and event streams demand sub-second latency. GC hiccups will surface as stalls in dashboards and alerts.

Answer

- Collector choice: on Java 21+, start with generational ZGC (concurrent mark + compact with tiny pauses). If not available, use G1 tuned for short pauses.
- Heap strategy: fix -Xms = -Xmx to avoid resizing; keep 10–20% headroom (ZGC SoftMaxHeapSize, G1 G1ReservePercent) so bursts don't trigger last-minute compaction.
- Young-gen sizing: ensure ephemeral event objects die young; avoid premature promotions.
- Diagnostics: enable unified GC logs and JFR to correlate allocation spikes with pause outliers; iterate with data, not guesses.

Code

ZGC baseline (Java 21+):

```
java \
 -Xms6g -Xmx6g \
 -XX:+UseZGC \
 -XX:SoftMaxHeapSize=5g \
 -XX:+ZProactive \
 -XX:+AlwaysPreTouch \
 -XX:+DisableExplicitGC \
 -Xlog:gc*,safepoint:file=gc.log:time,uptime,tags,level \
 -XX:StartFlightRecording=filename=jfr.jfr,settings=profile \
 -jar analytics.jar
```

G1 fallback:

```
java \
 -Xms6g -Xmx6g \
 -XX:+UseG1GC \
 -XX:MaxGCPauseMillis=50 \
 -XX:InitiatingHeapOccupancyPercent=30 \
 -XX:G1NewSizePercent=30 -XX:G1MaxNewSizePercent=60 \
 -XX:TargetSurvivorRatio=80 -XX:SurvivorRatio=8 \
 -XX:G1ReservePercent=20 \
 -XX:+ParallelRefProcEnabled \
 -XX:+AlwaysPreTouch -XX:+DisableExplicitGC \
 -Xlog:gc*,safepoint:file=gc.log:time,uptime,tags,level \
```

```
 -XX:StartFlightRecording=filename=jfr.jfr,settings=profile \
 -jar analytics.jar
```

App-level hot-path hygiene (example):

```
final class Builders {
  private static final ThreadLocal<StringBuilder> SB =
      ThreadLocal.withInitial(() -> new StringBuilder(256));
  static String format(Event e) {
    var sb = SB.get(); sb.setLength(0);
    return
sb.append(e.player()).append(',').append(e.metric()).append(',').append(e.value()).
toString();
  }
}
```

Reasoning to interviewer

- ZGC's concurrent compaction keeps pause variance tiny; with fixed heaps and proactive cycles, bursts won't cliff-edge into long STWs.
- With G1, earlier IHOP and a bigger young gen smooth mixed collections and reduce promotions.
- GC logs + JFR make it easy to tune using p95/p99 pauses, allocation rate, and promotion failures, not intuition.
- Reusing builders/buffers cuts allocation rate—the cheapest GC is the one you don't force to run.

Future scope

- Add back-pressure when GC time exceeds budget (temporarily downsample events).
- Consider -XX:+UseStringDeduplication if repeated keys inflate memory.
- Autoscale ahead of marquee games to keep per-instance allocation below stress thresholds.
- Gate releases on load-test baselines; alert on regression in pause percentiles.

Scenario 62: Simplifying heterogeneous payload handling with pattern matching

The IoT hub ingests mixed payloads: numeric sensor IDs, string alerts, and structured SensorEvent objects. Manual casts clutter the code and risk ClassCastException.

Answer

- I'll use instanceof pattern matching so each check both narrows and binds the variable—no separate cast lines.
- On Java 21, I can upgrade to a type-pattern switch for clearer, non-fallthrough routing with an explicit null arm.

Code

```
import java.time.Instant;

public record SensorEvent(int id, String type, double reading, Instant ts) {}

public final class Router {
  // Java 17+ if-chain with pattern variables
  public void handle(Object payload) {
    if (payload == null) { onMalformed("null"); return; }

    if (payload instanceof Integer sensorId) {
      routeToStateLookup(sensorId); return;
    }
    if (payload instanceof String msg) {
      var s = msg.trim();
      if (s.startsWith("ALERT:")) routeToAlerts(s); else routeToLogs(s); return;
    }
    if (payload instanceof SensorEvent evt) {
      routeToAnalytics(evt); return;
    }
    onMalformed("unsupported: " + payload.getClass().getName());
  }

  // Java 21+ type-pattern switch
  public void handleSwitch(Object payload) {
    switch (payload) {
      case null                -> onMalformed("null");
      case Integer sensorId    -> routeToStateLookup(sensorId);
      case String msg          -> {
        var s = msg.trim();
        if (s.startsWith("ALERT:")) routeToAlerts(s); else routeToLogs(s);
      }
```

```
      case SensorEvent evt       -> routeToAnalytics(evt);
      default                    -> onMalformed("unsupported: " +
payload.getClass().getName());
    }
  }

  // sinks
  private void routeToStateLookup(int id) { /* ... */ }
  private void routeToAlerts(String s)     { /* ... */ }
  private void routeToLogs(String s)       { /* ... */ }
  private void routeToAnalytics(SensorEvent e) { /* ... */ }
  private void onMalformed(String why)     { /* ... */ }
}
```

Reasoning to interviewer

- Pattern matching binds and narrows in one step (payload instanceof SensorEvent evt), which removes cast duplication and prevents drift after refactors.
- A type-pattern switch is exhaustive and non-fallthrough; adding a new payload type becomes an intentional code change.
- Explicit case null documents policy and avoids hidden NPEs in handlers.

Future scope

- Model structured payloads under a sealed interface (Payload permits SensorEvent, Alert, Command) to remove the default and let the compiler enforce exhaustiveness.
- Push parsing from bytes/JSON into typed records first, so the router operates only on well-formed types.
- Add per-arm metrics (e.g., alerts vs events mix) to guide capacity planning and optimizations.

Scenario 63: Ensuring thread-safe account updates in a trading platform

Trades settle concurrently; readers (fraud checks, risk) must see the latest committed balance, and updates can't clobber each other.

Answer:

I rely on the Java Memory Model's happens-before rules to guarantee visibility and ordering:

- Lock release → subsequent lock acquire (on the same lock) publishes changes.
- Volatile/atomic writes happen-before later reads of the same variable.
- Atomic classes add CAS so increments/decrements don't lose updates.

For single-field balances I use AtomicLong so get() is a volatile read and compareAndSet gives atomic debit/credit. For multi-field state (balance + lastTxnId + version), I publish an immutable snapshot through a volatile field so readers get a consistent view. Transfers (two accounts) need per-account locks acquired in a deterministic order to prevent deadlocks.

Code:

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.ReentrantLock;

// Single-field account: atomic updates + fresh reads
final class Account {
  private final AtomicLong balanceMicros = new AtomicLong(0);
  final ReentrantLock lock = new ReentrantLock(); // for multi-account ops

  long read() { return balanceMicros.get(); } // volatile read (fresh)

  void credit(long delta) {
    long p, n;
    do { p = balanceMicros.get(); n = p + delta; }
    while (!balanceMicros.compareAndSet(p, n));
  }

  void debit(long delta) {
    long p, n;
    do {
```

```
      p = balanceMicros.get(); n = p - delta;
      if (n < 0) throw new IllegalStateException("insufficient");
    } while (!balanceMicros.compareAndSet(p, n));
  }
}

// Multi-field snapshot (consistent reads)
record Snapshot(long balanceMicros, long version, String lastTxnId) {}
final class RichAccount {
  private volatile Snapshot snap = new Snapshot(0, 0, ""); // volatile publish
  final ReentrantLock lock = new ReentrantLock();

  Snapshot read() { return snap; } // always the last published snapshot

  void apply(long delta, String txnId) {
    lock.lock();
    try {
      long newBal = snap.balanceMicros() + delta;
      if (newBal < 0) throw new IllegalStateException("insufficient");
      snap = new Snapshot(newBal, snap.version() + 1, txnId); // atomic publish
    } finally { lock.unlock(); }
  }
}

final class Ledger {
  private final ConcurrentHashMap<Long, Account> accs = new
ConcurrentHashMap<>();
  private Account acc(long id) { return accs.computeIfAbsent(id, k -> new
Account()); }

  // Lock ordering by id avoids deadlocks
  void transfer(long fromId, long toId, long amount) {
    if (fromId == toId) return;
    Account a = acc(fromId), b = acc(toId);
    Account first = (fromId < toId) ? a : b, second = (first == a) ? b : a;

    first.lock.lock();
    try {
      second.lock.lock();
      try {
        a.debit(amount);
        b.credit(amount);
      } finally { second.lock.unlock(); }
    } finally { first.lock.unlock(); }
  }
}
```

Reasoning to interviewer:

- Atomic balances keep the hot path lock-free yet linearizable.
- A volatile snapshot reference ensures readers see a whole, self-consistent state without locking.
- Deterministic lock ordering eliminates cycles in two-account ops.
- The JMM guarantees that successful CAS and volatile writes are visible to later reads, satisfying fraud/risk freshness.

Future scope:

- Push ultra-hot accounts onto a single-writer mailbox (actor) for strictly ordered writes without locks.
- Idempotency keys on events to tolerate retries.
- Align DB layer with JVM ordering (e.g., lock rows in ascending account id).
- Publish a Flow.Publisher<Snapshot> for near-real-time fraud analytics.

Scenario 64: Breaking dependency cycles in a modular travel booking system

Flights, Hotels, and Billing started to depend on each other directly, creating cycles and fragile builds. I cut cycles by splitting each domain into API vs implementation modules and wiring via services.

Answer:

- Introduce a small com.travel.common for shared value types.
- For each domain, create *.api (exports only contracts/DTOs) and *.impl (no exports; provides the service).
- The app/orchestrator depends only on APIs and discovers implementations with ServiceLoader. No impl depends on another impl, so the graph is acyclic.

Code (module-info excerpts):

```
// common
module com.travel.common { exports com.travel.common; }

// flights.api
module com.travel.flights.api {
  requires transitive com.travel.common;
  exports com.travel.flights.api;              // FlightService, DTOs
  uses com.travel.flights.api.FlightService;
}

// flights.impl
module com.travel.flights.impl {
  requires com.travel.flights.api;
  provides com.travel.flights.api.FlightService
      with com.travel.flights.impl.DefaultFlightService;
}

// hotels.api
module com.travel.hotels.api {
  requires transitive com.travel.common;
  exports com.travel.hotels.api;
  uses com.travel.hotels.api.HotelService;
}

// hotels.impl
module com.travel.hotels.impl {
  requires com.travel.hotels.api;
  provides com.travel.hotels.api.HotelService
      with com.travel.hotels.impl.DefaultHotelService;
```

```
}

// billing.api
module com.travel.billing.api {
  requires transitive com.travel.common;
  exports com.travel.billing.api;              // PaymentService, Invoice DTOs
  uses com.travel.billing.api.PaymentService;
}

// billing.impl
module com.travel.billing.impl {
  requires com.travel.billing.api;
  provides com.travel.billing.api.PaymentService
      with com.travel.billing.impl.StripePaymentService;
}

// app (composition)
module com.travel.app {
  requires com.travel.common;
  requires com.travel.flights.api;
  requires com.travel.hotels.api;
  requires com.travel.billing.api;

  uses com.travel.flights.api.FlightService;
  uses com.travel.hotels.api.HotelService;
  uses com.travel.billing.api.PaymentService;
}
```

Reasoning to interviewer:

- JPMS hides everything by default; exporting only API packages prevents accidental impl coupling.
- uses/provides flips dependencies: the app pulls services; providers don't link to each other. That removes the edges that form cycles.
- requires transitive keeps downstream declarations clean (consumers of *.api see common automatically).

Future scope:

- Add CI guards (jdeps, Revapi) to fail on new cycles or accidental exports.
- Use narrow opens ... to <mapper> instead of blanket opens for reflection.
- Version the APIs with semver and run binary compatibility checks.
- Ship a trimmed runtime via jlink with only API + chosen impls.

Scenario 65: Using the Vector API in recommendation engines

We score user–item similarity (cosine/L2) over big float arrays. Scalar loops underutilize SIMD. I vectorize the hot kernels with the Vector API.

Answer:

- Choose the preferred species for the element type (FloatVector.SPECIES_PREFERRED or DoubleVector for sensitive sums).
- Process in chunks of species.length() lanes: load → compute (mul, add, fma) → store.
- Handle the tail via a mask (indexInRange) so arbitrary lengths work.
- Use reduceLanes for reductions and fma to fuse multiply-adds.

Code:

```
// On older JDKs, use jdk.incubator.vector.*; on newer, jdk.vector.*
import jdk.incubator.vector.*;

public final class RecoKernels {
  private static final FloatVector.Species S = FloatVector.SPECIES_PREFERRED;

  public static float dot(float[] a, float[] b) {
    int n = Math.min(a.length, b.length);
    int i = 0, ub = S.loopBound(n);
    var acc = FloatVector.zero(S);
    for (; i < ub; i += S.length()) {
      var va = FloatVector.fromArray(S, a, i);
      var vb = FloatVector.fromArray(S, b, i);
      acc = va.fma(vb, acc); // acc += va*vb
    }
    float sum = acc.reduceLanes(VectorOperators.ADD);
    for (; i < n; i++) sum += a[i] * b[i]; // tail
    return sum;
  }

  public static float norm2(float[] a) {
    int i = 0, ub = S.loopBound(a.length);
    var acc = FloatVector.zero(S);
    for (; i < ub; i += S.length()) {
      var v = FloatVector.fromArray(S, a, i);
      acc = v.fma(v, acc); // acc += v*v
    }
    float sum = acc.reduceLanes(VectorOperators.ADD);
    for (; i < a.length; i++) sum += a[i] * a[i];
    return (float) Math.sqrt(sum);
```

```
  }

  public static float cosine(float[] u, float[] v) {
    float d = dot(u, v);
    float nu = norm2(u), nv = norm2(v);
    return (nu == 0f || nv == 0f) ? 0f : d / (nu * nv);
  }

  // y = a*x + y (useful for SGD)
  public static void axpy(float a, float[] x, float[] y) {
    int n = Math.min(x.length, y.length);
    int i = 0, ub = S.loopBound(n);
    var va = FloatVector.broadcast(S, a);
    for (; i < ub; i += S.length()) {
      var vx = FloatVector.fromArray(S, x, i);
      var vy = FloatVector.fromArray(S, y, i);
      vx.fma(va, vy).intoArray(y, i);
    }
    for (; i < n; i++) y[i] = a * x[i] + y[i];
  }
}
```

Reasoning to interviewer:

- SPECIES_PREFERRED picks AVX/AVX-512/NEON widths at runtime—fast and portable.
- Vector loads/stores and fused ops are JIT-lowered to hardware SIMD; we don’t hand-code intrinsics.
- Masks make tails safe and branch-light; reductions are efficient via reduceLanes.
- Cosine and dot are often memory-bound; SIMD cuts loop overhead and gets us closer to bandwidth limits.

Future scope:

- Use DoubleVector just for accumulation to improve numeric stability on long vectors.
- Tile and parallelize (per-user/per-item shards) with SIMD inside each shard.
- Fuse passes (dot + norms) to reduce memory traffic; benchmark with JMH on target CPUs.
- Keep vectors in row-major primitive arrays for cache-friendly streaming.

Scenario 66: Preventing insecure deserialization in a loan-processing app

Attack surface: objects arriving over the wire. Goal: default-deny anything except our DTOs and reject oversized, deeply nested graphs that can DoS the JVM.

Answer

- I put a context-specific ObjectInputFilter on every ObjectInputStream at the boundary.
- The filter allowlists only our DTO packages and basic collections, then caps structure (depth, array length, reference count).
- I also reject dynamic proxies and risky JDK internals up front.
- Optionally, I install a strict global backstop so any missed path is still protected.

Code

```
import java.io.*;
import java.util.Set;

public final class DeserializationGuards {
  private static final String DTO_PKG = "com.bank.loan.dto.";
  private static final Set<String> ALSO_OK = Set.of("java.util.ArrayList",
"java.util.HashMap");

  public static ObjectInputFilter perStreamFilter() {
    return info -> {
      // structural limits
      if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
      if (info.arrayLength() != -1 && info.arrayLength() > 500_000) return
ObjectInputFilter.Status.REJECTED;
      if (info.references()  != -1 && info.references()  > 1_000_000) return
ObjectInputFilter.Status.REJECTED;

      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitives

      String n = cl.getName();
      if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
      if (n.startsWith("java.lang.invoke.") || n.startsWith("sun.") || n.startsWith("jdk.")
          || n.startsWith("java.rmi.")) return ObjectInputFilter.Status.REJECTED;

      return (n.startsWith(DTO_PKG) || ALSO_OK.contains(n))
          ? ObjectInputFilter.Status.ALLOWED
```

```
          : ObjectInputFilter.Status.REJECTED;
    };
  }

  // Optional global backstop
  public static void installGlobalBackstop() {
    ObjectInputFilter.Config.setSerialFilter(info -> {
      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
      return cl.getName().startsWith(DTO_PKG)
          ? ObjectInputFilter.Status.ALLOWED
          : ObjectInputFilter.Status.REJECTED;
    });
  }
}

// Usage at the network edge:
try (ObjectInputStream ois = new ObjectInputStream(incomingBytes)) {
  ois.setObjectInputFilter(DeserializationGuards.perStreamFilter());
  Object obj = ois.readObject(); // only allowlisted + structurally sane objects pass
}

Pattern string variant (simple deployments):

ObjectInputFilter f = ObjectInputFilter.Config.createFilter(
    "maxdepth=20;maxarray=500000;maxrefs=1000000;" +
    "com.bank.loan.dto.*;java.util.*;!*");
ois.setObjectInputFilter(f);
```

Reasoning to interviewer

- Filters run before object graph creation—gadget chains never get a chance to instantiate.
- Default-deny with a tiny allowlist plus structural caps closes both code-execution and resource-exhaustion paths.
- Keeping the filter “per stream” lets me tailor accept lists per topic/endpoint; the global backstop is insurance.

Future scope

- Prefer schema formats (JSON/Avro/Protobuf) with strict mappers; keep Java serialization only for trusted hops.
- Externalize the allowlist and add CI tests that attempt to deserialize known-bad types.

- Pair with mTLS or signed payloads so fewer malicious inputs reach the deserializer.
- Emit metrics on rejections to catch probing early.

Scenario 67: Enforcing immutability in feature-flag configuration with Records

Flags must be immutable once published (experiment id, variant allocations, rollout lists). Setters and mutable collections are where mistakes creep in.

Answer

- Records give me final components, generated accessors, and no setters by design—clean, immutable DTOs.
- I validate invariants in the compact constructor (e.g., allocations sum to 100) and defensively copy any incoming collections so nothing external can mutate internals.
- Small value types (e.g., ExperimentId, Percentage) push rules into the type system.

Code

```
import java.time.Instant;
import java.util.*;

public enum Env { DEV, STAGE, PROD }
public enum Variant { CONTROL, TREATMENT_A, TREATMENT_B }

public record ExperimentId(String value) {
  public ExperimentId {
    if (value == null || value.isBlank()) throw new IllegalArgumentException("id
required");
  }
}
public record Percentage(int value) {
  public Percentage {
    if (value < 0 || value > 100) throw new IllegalArgumentException("0..100");
  }
}

public record FeatureFlag(
    ExperimentId id,
    String featureKey,
    Env environment,
    Map<Variant, Percentage> allocations, // e.g., CONTROL=50,
TREATMENT_A=50
    List<String> includeUserIds,
    Instant publishedAt
) {
```

```
  public FeatureFlag {
    if (featureKey == null || featureKey.isBlank()) throw new
IllegalArgumentException("featureKey required");
    if (environment == null) throw new IllegalArgumentException("env required");
    if (publishedAt == null) throw new IllegalArgumentException("publishedAt
required");
    if (allocations == null || allocations.isEmpty()) throw new
IllegalArgumentException("allocations required");
    int sum = allocations.values().stream().mapToInt(Percentage::value).sum();
    if (sum != 100) throw new IllegalArgumentException("allocations must sum to
100");

    allocations   = Map.copyOf(allocations);                    // defensive
    includeUserIds = (includeUserIds == null) ? List.of() :
List.copyOf(includeUserIds);
  }

  // If we need the same flag in a different env, return a new instance
  public FeatureFlag forEnv(Env env) {
    return new FeatureFlag(id, featureKey, env, allocations, includeUserIds,
publishedAt);
  }
}
```

Reasoning to interviewer

- Records encode “data-only + immutable” intent; the compiler prevents accidental setters.
- The compact constructor centralizes validation and the copyOf wrappers ensure shallow immutability holds even if the caller keeps their original collections.
- Returning new records for changes (e.g., forEnv) keeps the publish-once rule intact and plays well with auditing.

Future scope

- Add Bean Validation on components (@NotBlank, custom @SumToHundred) for uniform error messages.
- Serialize records directly to JSON for audit snapshots; sign the snapshot to detect tampering.
- Provide diff utilities between two FeatureFlag records for rollout reviews.

- Store flags in a content-addressed cache using the record’s serialized form as the key.

Scenario 68: Scalable random number generation for online multiplayer battles

Many threads need RNGs for spawns/rewards. A single shared Random becomes a lock hotspot.

Answer

- I'll use Java 17's PRNG framework (java.util.random): pick a modern generator (e.g., LXM family) via RandomGeneratorFactory, then give each thread its own instance—no contention.
- For reproducible matches and fairness, derive independent substreams with Jumpable/Leapable so players/shards don't overlap sequences.
- Use bounded APIs like nextInt(origin, bound) to avoid modulo bias.

Code

```
import java.util.random.*;

public final class BattleRng {
  private static final RandomGeneratorFactory<? extends RandomGenerator> F =
      RandomGeneratorFactory.of("L64X128MixRandom");

  // Per-thread RNG — lock-free, cache-friendly
  private static final ThreadLocal<RandomGenerator> TL =
      ThreadLocal.withInitial(() -> F.create(seed()));

  private static long seed() {
    // For deterministic tests, inject a fixed seed
    return System.nanoTime() ^ Thread.currentThread().getId();
    // or use SecureRandom once to derive a root seed
  }

  public static int roll(int fromInclusive, int toExclusive) {
    return TL.get().nextInt(fromInclusive, toExclusive);
  }
  public static boolean chance(double p) {
    return TL.get().nextDouble() < p;
  }

  // Deterministic, non-overlapping stream per match/player
  public static RandomGenerator streamFor(long matchSeed, int playerIndex) {
    RandomGenerator g = F.create(matchSeed);
    if (g instanceof RandomGenerator.Jumpable j) {
      for (int i = 0; i < playerIndex; i++) g = j.jump();
```

```
      return g; // each index gets a disjoint substream
    }
    long mixed = matchSeed ^ (0x9E3779B97F4A7C15L * playerIndex);
    return F.create(mixed);
  }
}
```

Reasoning to interviewer

- Per-thread generators remove synchronization entirely; throughput scales with cores.
- LXM generators support jump() so I can deterministically partition randomness (per player, per shard) and reproduce matches for debugging.
- The API separates algorithm choice from usage; swapping to L128X256MixRandom is a one-liner if we need a longer period.

Future scope

- Centralize RNG policy behind an interface; log seeds and stream indices for replay.
- Use SecureRandom only for security-sensitive flows (tokens/anti-cheat), not gameplay RNG.
- Property-based tests to catch accidental RNG reuse; lightweight health checks to ensure independence across threads.
- Consider binding tokens to TLS sessions or mTLS for extra replay protection outside RNG concerns.

Scenario 69: Enforcing API boundaries in healthcare microservices

Patients, Billing, and Reporting must expose only stable APIs. Any accidental reach into implementation classes should fail at compile time.

Answer

I'd split each domain into a tiny API module and a separate implementation module. Only API packages are exported; implementations export nothing and register via JPMS services (provides/uses). Shared value types (PatientId, Money, DateRange) go in a health.common module. The app/gateway depends on APIs only and discovers providers with ServiceLoader. This structure gives strong encapsulation and an acyclic module graph.

Code

module-info.java (common + APIs):

```
// health.common
module com.health.common {
  exports com.health.common; // PatientId, Money, DateRange, Errors
}

// patients.api
module com.health.patients.api {
  requires transitive com.health.common;
  exports com.health.patients.api;            // PatientService, DTOs
  uses com.health.patients.api.PatientService;
}

// billing.api
module com.health.billing.api {
  requires transitive com.health.common;
  exports com.health.billing.api;             // PaymentService, Invoice DTOs
  uses com.health.billing.api.PaymentService;
}

// reporting.api
module com.health.reporting.api {
  requires transitive com.health.common;
  requires transitive com.health.patients.api;
  requires transitive com.health.billing.api;
  exports com.health.reporting.api;           // ReportService
  uses com.health.reporting.api.ReportService;
}
```

module-info.java (implementations):

```
// patients.impl
module com.health.patients.impl {
  requires com.health.patients.api;
  requires com.health.common;
  provides com.health.patients.api.PatientService
      with com.health.patients.impl.SqlPatientService;
  // If a framework needs reflection, open narrowly:
  opens com.health.patients.impl.serialization to com.fasterxml.jackson.databind;
}

// billing.impl
module com.health.billing.impl {
  requires com.health.billing.api;
  requires com.health.common;
  provides com.health.billing.api.PaymentService
      with com.health.billing.impl.StripePaymentService;
}

// reporting.impl
module com.health.reporting.impl {
  requires com.health.reporting.api;
  requires com.health.patients.api;
  requires com.health.billing.api;
  provides com.health.reporting.api.ReportService
      with com.health.reporting.impl.DefaultReportService;
}
```

Composition (gateway/app):

```
module com.health.app {
  requires com.health.common;
  requires com.health.patients.api;
  requires com.health.billing.api;
  requires com.health.reporting.api;

  uses com.health.patients.api.PatientService;
  uses com.health.billing.api.PaymentService;
  uses com.health.reporting.api.ReportService;
}
```

Reasoning to interviewer

- JPMS hides everything by default; exporting only API packages prevents accidental coupling to internals.
- uses/provides formalizes inversion of control, so implementations don’t depend on each other and cycles can’t form.

- requires transitive on APIs forwards common to consumers, keeping dependency lists clean.
- Narrow opens … to keeps frameworks working without punching global holes in encapsulation.

Future scope

- Fail CI on illegal access or cycles via jdeps and Revapi; gate releases on API compatibility.
- Produce a jlink image containing only API + selected impls to reduce footprint and attack surface.
- Introduce versioned API modules and semantic version checks to protect downstream services.

Scenario 70: Replacing verbose DTOs in a logistics platform

Parcels, shipments, and invoices are simple carriers yet full of getters, setters, and boilerplate. The intent is immutable data.

Answer

I'd migrate DTOs to records. Records generate constructor, accessors, equals/hashCode, and toString. I keep invariants in a compact constructor and defensively copy incoming collections so they stay effectively immutable. JSON mappers (Jackson 2.12+/Spring Boot 3) bind to the canonical constructor; JPA entities remain classes, but projections/REST payloads become records.

Code

```
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.Map;

public record TrackingId(String value) {
  public TrackingId {
    if (value == null || value.isBlank()) throw new IllegalArgumentException("tracking
id required");
  }
}
public record Money(BigDecimal amount, String currency) {
  public Money {
    if (amount == null || amount.signum() < 0) throw new
IllegalArgumentException("amount >= 0");
    if (currency == null || currency.isBlank()) throw new
IllegalArgumentException("currency required");
  }
}

public record ParcelDTO(TrackingId trackingId,
                        double weightKg,
                        Map<String,String> attributes) {
  public ParcelDTO {
    if (trackingId == null) throw new IllegalArgumentException("trackingId required");
    if (weightKg <= 0) throw new IllegalArgumentException("weight > 0");
    attributes = (attributes == null) ? Map.of() : Map.copyOf(attributes); // defensive
copy
  }
}

public record ShipmentDTO(String shipmentId,
```

```
                    List<ParcelDTO> parcels,
                    Instant createdAt) {
  public ShipmentDTO {
    if (shipmentId == null || shipmentId.isBlank()) throw new
IllegalArgumentException("shipmentId required");
    if (createdAt == null) throw new IllegalArgumentException("createdAt required");
    parcels = (parcels == null) ? List.of() : List.copyOf(parcels);
  }
}

public record InvoiceDTO(String invoiceNo, Money total, List<TrackingId> lines) {
  public InvoiceDTO {
    if (invoiceNo == null || invoiceNo.isBlank()) throw new
IllegalArgumentException("invoiceNo required");
    if (total == null) throw new IllegalArgumentException("total required");
    lines = (lines == null) ? List.of() : List.copyOf(lines);
  }
}
```

Reasoning to interviewer

- Records encode “data-only + immutable” in the type, eliminating accidental setters.
- Compact constructors centralize validation and ensure callers can’t mutate internal collections later.
- We keep persistence entities as classes (to satisfy ORM proxies), but everything crossing boundaries (REST/Kafka) benefits from records’ clarity and safety.

Future scope

- Add Bean Validation on components (@NotBlank, @Positive) for uniform error messages.
- Generate OpenAPI/JSON Schema from record DTOs to lock interfaces.
- Create mapping helpers (entity → record) and contract tests to catch field drift early.

Scenario 71: Accelerating graph algorithms with SIMD

Graph workloads are irregular, but inner loops still do uniform math (relax edges, accumulate ranks). Scalar loops underuse the CPU's SIMD units.

Answer

I'd store the graph in a CSR layout (row offsets + neighbor array + weights) and vectorize arithmetic-friendly parts with the Vector API. For relaxations, process contiguous per-source edge blocks in SIMD: load weights, add source distance, and write improvements via a two-phase scheme to avoid atomic conflicts. PageRank and centrality (SpMV-like) map even more directly to broadcast/mul/add/reduce.

Code

```
// If your JDK still uses the incubator: import jdk.incubator.vector.*;
// Otherwise: import jdk.vector.*;
import jdk.incubator.vector.*;

public final class GraphSimd {
  private static final FloatVector.Species S = FloatVector.SPECIES_PREFERRED;

  // CSR arrays: rowPtr[v..v+1) -> edges of v; edgeTo[i], weight[i]
  // Two-phase relax: compute best candidate per destination in scratch to avoid
atomics
  public static void relaxVertex(int v, float[] dist,
                                 int[] edgeTo, float[] weight,
                                 int[] rowPtr,
                                 float[] scratchBest /* init with +INF */) {
    int start = rowPtr[v], end = rowPtr[v + 1];
    float dv = dist[v];

    int i = start, ub = start + S.loopBound(end - start);
    var vDv = FloatVector.broadcast(S, dv);

    for (; i < ub; i += S.length()) {
      var ww = FloatVector.fromArray(S, weight, i);
      var cand = ww.add(vDv);                  // candidate = dv + w
      // destinations are contiguous for a vertex's row; commit per lane
      for (int lane = 0; lane < S.length(); lane++) {
        int dest = edgeTo[i + lane];
        float c = cand.lane(lane);
        float prev = scratchBest[dest];
        if (c < prev) scratchBest[dest] = c;    // scalar min (no atomics)
      }
    }
    // tail
```

```
    for (; i < end; i++) {
      int dest = edgeTo[i];
      float cand = dv + weight[i];
      if (cand < scratchBest[dest]) scratchBest[dest] = cand;
    }
  }

  // PageRank-like axpy: y = a*x + y
  public static void axpy(float a, float[] x, float[] y) {
    int n = Math.min(x.length, y.length);
    int i = 0, ub = S.loopBound(n);
    var va = FloatVector.broadcast(S, a);
    for (; i < ub; i += S.length()) {
      var vx = FloatVector.fromArray(S, x, i);
      var vy = FloatVector.fromArray(S, y, i);
      vx.fma(va, vy).intoArray(y, i);
    }
    for (; i < n; i++) y[i] = a * x[i] + y[i];
  }

  public static float reduceSum(float[] a) {
    int i = 0, ub = S.loopBound(a.length);
    var acc = FloatVector.zero(S);
    for (; i < ub; i += S.length()) acc = acc.add(FloatVector.fromArray(S, a, i));
    float sum = acc.reduceLanes(VectorOperators.ADD);
    for (; i < a.length; i++) sum += a[i];
    return sum;
  }
}
```

Reasoning to interviewer

- CSR packs a vertex's outgoing edges contiguously, giving the Vector API something regular to work on despite overall graph irregularity.
- Arithmetic is vectorized; conflicting writes are handled in a second phase (or using per-partition buffers) to avoid atomics.
- SPECIES_PREFERRED picks AVX/AVX-512/NEON at runtime—portable, fast.
- PageRank/centrality are SpMV-style; broadcasts, fma, and reduceLanes map nicely to SIMD and cut loop overhead.

Future scope

- Tile edges/vertices to improve cache locality; parallelize tiles across threads while keeping SIMD inside each tile.
- Use DoubleVector for numerically sensitive reductions and FloatVector elsewhere.
- Fuse passes (e.g., compute rank and normalization together) to reduce memory traffic; benchmark with JMH on target CPUs.

Scenario 72: Eliminating circular module dependencies in an enterprise suite

Auth, Inventory, and Payments accidentally ended up depending on each other (Auth ↔ Payments ↔ Inventory). That creates upgrade pain and confusing build graphs. I want a structure where only stable APIs are reachable and implementations don't see each other at all.

Answer:

I'll split each domain into a tiny API module and a separate implementation module. Only the API packages are exported. Implementations register via JPMS services (provides … with …) and export nothing. Shared value types live in com.suite.common. The application composes services by depending only on the APIs and using ServiceLoader—which breaks cycles by construction.

Code:

```
// com.suite.common
module com.suite.common {
  exports com.suite.common; // UserId, Money, Errors
}

// com.suite.auth.api
module com.suite.auth.api {
  requires transitive com.suite.common;
  exports com.suite.auth.api;              // AuthService, DTOs
  uses com.suite.auth.api.AuthService;     // discovery
}

// com.suite.auth.impl
module com.suite.auth.impl {
  requires com.suite.auth.api;
  requires com.suite.common;
  provides com.suite.auth.api.AuthService
      with com.suite.auth.impl.JwtAuthService;
  // If reflection is needed, open narrowly:
  // opens com.suite.auth.impl.serialization to com.fasterxml.jackson.databind;
}

// com.suite.inventory.api
module com.suite.inventory.api {
  requires transitive com.suite.common;
  exports com.suite.inventory.api;         // InventoryService, Sku DTOs
  uses com.suite.inventory.api.InventoryService;
}
```

```
// com.suite.inventory.impl
module com.suite.inventory.impl {
  requires com.suite.inventory.api;
  provides com.suite.inventory.api.InventoryService
      with com.suite.inventory.impl.DefaultInventoryService;
}

// com.suite.payments.api
module com.suite.payments.api {
  requires transitive com.suite.common;
  exports com.suite.payments.api;           // PaymentService, Invoice DTOs
  uses com.suite.payments.api.PaymentService;
}

// com.suite.payments.impl
module com.suite.payments.impl {
  requires com.suite.payments.api;
  provides com.suite.payments.api.PaymentService
      with com.suite.payments.impl.StripePaymentService;
}

// Composition layer
module com.suite.app {
  requires com.suite.common;
  requires com.suite.auth.api;
  requires com.suite.inventory.api;
  requires com.suite.payments.api;

  uses com.suite.auth.api.AuthService;
  uses com.suite.inventory.api.InventoryService;
  uses com.suite.payments.api.PaymentService;
}
```

Reasoning to interviewer:

- The module graph becomes acyclic because implementations don't depend on each other—only the app pulls them in.
- JPMS hides everything by default; exporting only API packages prevents "reaching into" internals.
- requires transitive on APIs forwards com.suite.common, so consumers don't need to redeclare it.
- uses/provides is a clean inversion of control: swap impls without touching dependents.

Future scope:

- Add jdeps and Revapi checks in CI to fail on new cycles or accidental exports.
- Use opens … to only for specific frameworks (Jackson/Hibernate), never blanket openness.
- Produce a jlink image with just API + chosen impls to cut footprint and attack surface.
- Version API modules and run binary-compat checks to guard downstreams.

Scenario 73: Adding runtime behavior in a plugin-based CMS

Plugins (reporting, media export) are loaded at runtime. I want to weave behaviors like logging, validation, or request-ID tagging around plugin calls without touching plugin code.

Answer:

I'll expose plugins via small interfaces and wrap instances with a JDK dynamic proxy. The InvocationHandler becomes a tiny pipeline: check config/annotations, decorate arguments (prepend request-id), short-circuit low-level logs in prod, or buffer-and-flush on error. Since the proxy sits at the interface boundary, plugin classes remain untouched.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;
import java.util.concurrent.TimeUnit;

@Retention(RetentionPolicy.RUNTIME) @Target({ElementType.TYPE,
ElementType.METHOD})
@interface Validate {}

interface ExportPlugin {
  byte[] export(String contentId, String format);
}

final class PdfExporter implements ExportPlugin {
  public byte[] export(String contentId, String format) {
    return ("PDF:" + contentId + ":" + format).getBytes();
  }
}

final class PluginConfig {
  volatile boolean dropDebug = true;
  volatile boolean prependRequestId = true;
  volatile boolean timeCalls = true;
}

final class RequestCtx {
  static final ThreadLocal<String> requestId = new ThreadLocal<>();
}

final class PluginProxy implements InvocationHandler {
  private final Object target;
```

```
  private final PluginConfig cfg;

  private PluginProxy(Object target, PluginConfig cfg) { this.target = target; this.cfg
= cfg; }

  @SuppressWarnings("unchecked")
  static <T> T wrap(Class<T> api, T impl, PluginConfig cfg) {
    return (T) Proxy.newProxyInstance(api.getClassLoader(), new Class<?>[]{api},
new PluginProxy(impl, cfg));
  }

  @Override public Object invoke(Object proxy, Method m, Object[] args) throws
Throwable {
    // 1) Optional timing
    long start = cfg.timeCalls ? System.nanoTime() : 0;

    try {
      // 2) Optional request-id prefix
      if (cfg.prependRequestId && args != null && args.length > 0 && args[0]
instanceof String s) {
        String rid = RequestCtx.requestId.get();
        if (rid != null && !rid.isBlank()) args[0] = "[rid=" + rid + "] " + s;
      }
      // 3) Optional validation hint
      if (m.isAnnotationPresent(Validate.class) ||
target.getClass().isAnnotationPresent(Validate.class)) {
        for (Object a : args == null ? new Object[0] : args) {
          if (a instanceof String str && str.isBlank()) throw new
IllegalArgumentException("blank arg");
        }
      }
      return m.invoke(target, args);
    } catch (InvocationTargetException ite) {
      throw ite.getCause();
    } finally {
      if (cfg.timeCalls) {
        long ms = TimeUnit.NANOSECONDS.toMillis(System.nanoTime() - start);
        System.out.printf("plugin=%s method=%s durationMs=%d%n",
            target.getClass().getSimpleName(), m.getName(), ms);
      }
    }
  }
}

// Usage
PluginConfig cfg = new PluginConfig();
ExportPlugin plugin = PluginProxy.wrap(ExportPlugin.class, new PdfExporter(),
cfg);
RequestCtx.requestId.set("REQ-42");
```

```
byte[] out = plugin.export("post-123", "pdf"); // gets prefixed/logged/validated per config
```

Reasoning to interviewer:

- Dynamic proxies are lightweight AOP at the interface boundary; no bytecode weaving or framework required.
- The handler centralizes cross-cutting concerns and reads runtime config, so changing behavior doesn’t require a rebuild.
- Unwrapping InvocationTargetException preserves original exceptions for proper diagnostics.

Future scope:

- Cache per-method policy (timed/validated) to reduce reflection overhead.
- Emit metrics via Micrometer; add rate limits or circuit-breakers in the handler.
- For classes without interfaces, switch to Byte Buddy/CGLIB.
- Allow a chain of decorators (policy list) so features can be enabled/ordered declaratively.

Scenario 74: Validating deserialization of sensitive financial records

The reporting service ingests serialized objects. Only approved DTOs should be instantiated; everything else—especially gadget classes—must be blocked.

Answer:

I'll attach context-specific ObjectInputFilters to every ObjectInputStream at the boundary. Filters allow only DTO packages and a small set of safe JDK containers, and they cap structure (maxdepth, maxarray, maxrefs) to stop resource bombs. I'll also install a strict global backstop in case a code path forgets to set its own filter.

Code:

```
import java.io.*;
import java.util.Set;

public final class DeserGuards {
  private static final String DTO_PKG = "com.finance.reports.dto.";
  private static final Set<String> ALSO_OK = Set.of("java.util.ArrayList",
"java.util.HashMap");

  public static ObjectInputFilter inbound() {
    return info -> {
      if (info.depth() > 20) return ObjectInputFilter.Status.REJECTED;
      if (info.arrayLength() != -1 && info.arrayLength() > 500_000) return
ObjectInputFilter.Status.REJECTED;
      if (info.references()  != -1 && info.references()  > 1_000_000) return
ObjectInputFilter.Status.REJECTED;

      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED; // primitives / null
      String n = cl.getName();

      if (Proxy.isProxyClass(cl)) return ObjectInputFilter.Status.REJECTED;
      if (n.startsWith("java.lang.invoke.") || n.startsWith("sun.") || n.startsWith("jdk.")
          || n.startsWith("java.rmi.")) return ObjectInputFilter.Status.REJECTED;

      return (n.startsWith(DTO_PKG) || ALSO_OK.contains(n))
          ? ObjectInputFilter.Status.ALLOWED
          : ObjectInputFilter.Status.REJECTED;
    };
  }

  // Global backstop (optional but recommended)
  public static void installGlobalBackstop() {
    ObjectInputFilter.Config.setSerialFilter(info -> {
```

```
      Class<?> cl = info.serialClass();
      if (cl == null) return ObjectInputFilter.Status.UNDECIDED;
      return cl.getName().startsWith(DTO_PKG)
          ? ObjectInputFilter.Status.ALLOWED
          : ObjectInputFilter.Status.REJECTED;
    });
  }
}

// Boundary usage
try (ObjectInputStream ois = new ObjectInputStream(in)) {
  ois.setObjectInputFilter(DeserGuards.inbound());
  Object obj = ois.readObject(); // rejected unless allowlisted + within limits
}

One-line pattern alternative:

ObjectInputFilter f = ObjectInputFilter.Config.createFilter(
  "maxdepth=20;maxarray=500000;maxrefs=1000000;" +
  "com.finance.reports.dto.*;java.util.*;!*");
ois.setObjectInputFilter(f);
```

Reasoning to interviewer:

- Filters execute before object graph creation, so dangerous classes never get instantiated.
- A tight allowlist plus structural caps addresses both code-execution and resource-exhaustion angles.
- Keeping filters per stream lets each topic define its DTOs; the global backstop covers oversights.

Future scope:

- Prefer schema-based formats (JSON/Avro/Protobuf) for cross-service traffic; keep Java serialization only where necessary.
- Externalize the allowlist and test with “known bad” payloads in CI.
- Pair with mTLS or signed messages to reduce the chance of tampering upstream.
- Add observability on rejections to spot probing and tune limits safely.

Scenario 75: Reducing heap fragmentation in a messaging service

High-throughput producers/consumers allocate many short-lived and some medium-lived objects. Fragmentation creeps in, GC frequency rises, and pauses get spiky.

Answer

- Prefer concurrent-compacting collectors that continuously defragment:
  - ZGC (Java 21+ generational by default) or Shenandoah on supported platforms.
  - If you must use G1, tune region sizing and mixed-GC cadence proactively.
- Fix heap size to avoid resizing churn and keep headroom for bursts.
- Keep objects “young”: right-size young gen to avoid premature promotion; reuse buffers/builders on hot paths.
- Cap metaspace and trim the runtime image to lower total footprint.
- Pre-touch pages so the first traffic spike doesn’t suffer page-fault stalls.

Code

ZGC baseline (low fragmentation / low pause):

```
java \
  -Xms8g -Xmx8g \
  -XX:+UseZGC \
  -XX:SoftMaxHeapSize=7g \
  -XX:+ZProactive \
  -XX:+AlwaysPreTouch \
  -XX:+DisableExplicitGC \
  -Xlog:gc*,safepoint:file=gc.log:time,uptime,level,tags \
  -jar messaging.jar
```

G1 tuned fallback (when ZGC isn’t available):

```
java \
  -Xms8g -Xmx8g \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=50 \
  -XX:InitiatingHeapOccupancyPercent=30 \
  -XX:G1NewSizePercent=30 -XX:G1MaxNewSizePercent=60 \
  -XX:TargetSurvivorRatio=80 -XX:SurvivorRatio=8 \
  -XX:G1HeapRegionSize=8m \
  -XX:G1ReservePercent=20 \
  -XX:+ParallelRefProcEnabled \
```

```
  -XX:+AlwaysPreTouch -XX:+DisableExplicitGC \
  -Xlog:gc*,safepoint:file=gc.log:time,uptime,level,tags \
  -jar messaging.jar
```

Hot-path hygiene (reduce allocation → less fragmentation):

```
final class Builders {
  private static final ThreadLocal<StringBuilder> SB =
      ThreadLocal.withInitial(() -> new StringBuilder(256));
  static String logLine(String topic, int size) {
    var sb = SB.get(); sb.setLength(0);
    return sb.append("t=").append(topic).append(",sz=").append(size).toString();
  }
}
```

Reasoning to interviewer

- Concurrent compaction (ZGC/Shenandoah) prevents free-list splintering by relocating objects continuously with tiny pauses.
- Fixed heap and proactive cycles prevent “cliff” behavior when bursts arrive.
- With G1, larger young regions + earlier IHOP smooth mixed collections and keep old-gen compact.
- Reducing allocation rate and promotion directly lowers fragmentation pressure.

Future scope

- Consider off-heap/pooled ByteBuffers for wire I/O; enable -XX:+UseStringDeduplication if many duplicate keys.
- Use JFR + GC logs to correlate allocation spikes to pause outliers; alert on p99 GC > budget.
- Shard hot topics/queues to spread allocation across threads; pin ultra-hot paths to single-writer actors.
- Trim the JRE with jlink and use AppCDS to cut metaspace and startup thrash.

Scenario 76: Thread safety in an e-commerce order pipeline

Multiple threads validate, price, charge, and ship orders while updating global stats (orders/sec, revenue, per-SKU counters). We want correctness without turning the pipeline into a single lock.

Answer

- Use contention-friendly atomics for hot counters: LongAdder/DoubleAdder outperform AtomicLong under high write concurrency.
- Concurrent maps with computeIfAbsent to manage per-SKU counters safely.
- Immutable DTOs between stages; mutate local state only.
- Shard updates by key (e.g., SKU or customer) to single-writer executors—no locks inside shards.
- Batch locally and flush to shared counters periodically to reduce write amplification.
- For consistent multi-field snapshots, publish immutable views instead of holding writer locks.

Code

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

record Order(String id, String sku, int qty, double price) {}

final class Stats {
  final LongAdder orders = new LongAdder();
  final LongAdder failures = new LongAdder();
  final DoubleAdder revenue = new DoubleAdder();
  final ConcurrentHashMap<String, LongAdder> perSku = new
ConcurrentHashMap<>();

  void addOrder(String sku, double amount) {
    orders.increment();
    revenue.add(amount);
    perSku.computeIfAbsent(sku, k -> new LongAdder()).increment();
  }
  void addFailure() { failures.increment(); }
}

final class ShardedUpdaters implements AutoCloseable {
  private final int shards = Math.max(2,
Runtime.getRuntime().availableProcessors());
```

```
  private final ExecutorService[] execs = new ExecutorService[shards];
  ShardedUpdaters() { for (int i=0;i<shards;i++) execs[i] =
Executors.newSingleThreadExecutor(); }
  void submit(String key, Runnable r) {
    int idx = Math.floorMod(key.hashCode(), shards);
    execs[idx].submit(r); // per-shard single-writer ⇒ no locks inside r
  }
  public void close() { for (var e : execs) e.shutdown(); }
}

final class Pipeline {
  private final Stats stats = new Stats();
  private final ShardedUpdaters sharded = new ShardedUpdaters();

  void onOrder(Order o) {
    double amount = o.qty() * o.price(); // local calc
    sharded.submit(o.sku(), () -> stats.addOrder(o.sku(), amount));
  }

  record Snapshot(long orders, long failures, double revenue) {}
  Snapshot snapshot() {
    // Non-blocking sums; if multi-field atomicity is needed, publish a periodic
Snapshot
    return new Snapshot(stats.orders.sum(), stats.failures.sum(),
stats.revenue.sum());
  }
}
```

Reasoning to interviewer

- LongAdder/DoubleAdder stripe updates internally, reducing contention on hot counters.
- Sharding converts many contending writers into N single writers; inside the shard you run without locks.
- Immutable messages and localized mutation keep pipeline stages simple and safe.
- Snapshots via sum() avoid blocking; for strict consistency, publish a scheduled immutable snapshot.

Future scope

- Replace shard executors with SPSC/MPSC lock-free queues to cut scheduler overhead.

- Expose Micrometer metrics for contention and queue depth; auto-rebalance shards when a SKU is ultra-hot.
- For complex invariants (e.g., per-user quotas), switch to per-key actors/single writers.
- Add backpressure when queue depth exceeds thresholds to keep tail latencies in check.

Scenario 77: Faster startup for microservices

Large dependency graphs + framework scanning slow cold start and autoscaling responsiveness.

Answer

- **Class Data Sharing (CDS/AppCDS):** share pre-parsed class metadata across processes; include app/framework classes in an AppCDS archive.
- **Lazy/lightweight initialization:** defer clients, caches, and heavy beans until first use; avoid eager reflection scans.
- **Trim the runtime/classpath:** remove unused starters; build a jlink image with only required modules.
- Parallel-capable class loaders and fewer static initializers to reduce single-threaded choke points at boot.
- **Profile-guided class lists:** generate from a warm run and bake into the archive.

Code

AppCDS workflow:

```
# 1) Record loaded classes from a representative run
java -Xshare:off -XX:DumpLoadedClassList=app.classlist -jar service.jar --sample-traffic

# 2) Create the shared archive with app + libs
java -Xshare:dump \
    -XX:SharedClassListFile=app.classlist \
    -XX:SharedArchiveFile=app-cds.jsa \
    -cp service.jar:lib/*

# 3) Run using the archive (faster cold start)
java -Xshare:on -XX:SharedArchiveFile=app-cds.jsa -jar service.jar

Lazy init patterns (framework-agnostic):

final class HeavyClient {
  private HeavyClient() { /* connect pools, warm caches lazily */ }
  private static class Holder { static final HeavyClient I = new HeavyClient(); }
  static HeavyClient get() { return Holder.I; } // constructed on first call
}

interface Provider<T> { T get(); }
```

```
final class MetricsProvider implements Provider<Metrics> {
  private volatile Metrics m;
  public Metrics get() {
    var local = m;
    if (local == null) {
      synchronized (this) { if (m == null) m = new Metrics(/*...*/); local = m; }
    }
    return local;
  }
}
```

Reasoning to interviewer

- CDS/AppCDS removes repeated bytecode parsing/verification and shares metadata—fleet-wide cold starts drop noticeably.
- Lazy init shortens the critical path to “ready”; most services don’t need every client warm at t=0.
- Trimming the runtime avoids loading/verifying classes you never use; fewer <clinit> blocks means fewer boot stalls.

Future scope

- Consider native images (GraalVM AOT) for select services where ultra-fast cold start > peak throughput.
- Persist warm caches between deploys when safe; precompute config/DI graphs.
- Automate classlist generation in CI from realistic traffic replays and fail PRs that regress startup budgets.
- Combine with -XX:+AlwaysPreTouch and tuned container sizing to avoid first-touch latency on memory pages.

Scenario 78: Securing service-to-service calls with JWT

In a microservices setup, Service A calls Service B over HTTP. We want B to accept only authenticated callers and to authorize per-route capabilities (scopes).

Answer:

- I'd use issuer-signed JWTs (RS256/ES256) with a centralized issuer (Auth server). A gets a token via client-credentials, then calls B with Authorization: Bearer <jwt>.
- B validates signature against the issuer's JWKS, then checks iss, aud, exp/nbf, and scope (or roles) required by the endpoint.
- I'd cache JWKS by kid to avoid hot-path network calls and optionally pair JWT with mTLS for defense-in-depth.

Code:

```
// Server-side filter using Nimbus JOSE + JWT (pom: com.nimbusds:nimbus-jose-
jwt)
import com.nimbusds.jose.jwk.source.*;
import com.nimbusds.jose.proc.*;
import com.nimbusds.jwt.proc.*;
import com.nimbusds.jwt.SignedJWT;
import javax.servlet.*;
import javax.servlet.http.*;
import java.net.URL;
import java.util.Set;

public class JwtAuthFilter implements Filter {
  private final String issuer = "https://auth.example.com";
  private final Set<String> audience = Set.of("service-b");
  private final ConfigurableJWTProcessor<SecurityContext> processor;

  public JwtAuthFilter() throws Exception {
    var jwksUrl   = new URL(issuer + "/.well-known/jwks.json");
    var keySource = new RemoteJWKSet<SecurityContext>(jwksUrl);
    processor     = new DefaultJWTProcessor<>();
    processor.setJWSKeySelector(new
JWSVerificationKeySelector<>(com.nimbusds.jose.JWSAlgorithm.RS256,
keySource));
    // Optionally configure clock skew, require exp/iat, etc.
  }

  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
```

```
      throws java.io.IOException, ServletException {
    HttpServletRequest http = (HttpServletRequest) req;
    HttpServletResponse out = (HttpServletResponse) res;
    String header = http.getHeader("Authorization");
    if (header == null || !header.startsWith("Bearer ")) { out.sendError(401); return; }

    try {
      var claims = processor.process(SignedJWT.parse(header.substring(7)), null);
      if (!issuer.equals(claims.getIssuer()) ||
          claims.getAudience().stream().noneMatch(audience::contains)) {
out.sendError(403); return; }

      var scopes = Set.of(claims.getStringClaim("scope").split(" "));
      // check route-specific scopes here, e.g., require "orders.read"
      // attach principal/claims to request context
      chain.doFilter(req, res);
    } catch (Exception e) {
      out.sendError(401);
    }
  }
}
```

Reasoning to interviewer:

- JWT gives stateless auth: B verifies with cached JWKS; no per-request call to the issuer.
- Audience binding prevents token replay across services; scopes make least-privilege enforceable per route.
- Key rotation is clean with kid—we just refetch JWKS when a new kid appears.

Future scope:

- Add short-TTL tokens and automatic refresh; consider DPoP or mTLS-bound tokens to reduce replay risk.
- Emit structured audit logs containing sub, aud, scope, and trace IDs.
- Centralize permission checks with an authz library to avoid drift across services.

Scenario 79: Sealed classes for payroll roles

Only two categories exist: full-time and contractor. We must prevent any other subclasses of Employee.

Answer:

- I'll declare a sealed base type and explicitly list the permitted subtypes. Leaves are final (or non-sealed if we need a controlled extension point). That locks the hierarchy at compile time and enables exhaustive pattern matching.

Code:

```
// Java 17+
public sealed interface Employee permits FullTime, Contractor {
  String id();
  String name();
}

public final record FullTime(String id, String name, long annualSalaryMicros)
implements Employee {}
public final record Contractor(String id, String name, long hourlyRateMicros)
implements Employee {}

// Exhaustive business logic (no default needed for sealed hierarchies)
long monthlyCost(Employee e) {
  return switch (e) {
    case FullTime ft  -> ft.annualSalaryMicros() / 12;
    case Contractor c -> c.hourlyRateMicros() * 160; // assume 160 hrs/mo
  };
}
```

Reasoning to interviewer:

- sealed … permits … prevents unapproved subtypes at compile time, not just by convention.
- Using records for the variants keeps them immutable and concise.
- Pattern-matching switch is exhaustive over the sealed hierarchy, so missing a case fails the build.

Future scope:

- Keep the sealed family in one JPMS module to avoid accidental exposure.

- If regional rules require more variants later, mark one branch non-sealed and restrict further in the module.
- Add validation in record compact constructors (e.g., non-negative pay).

Scenario 80: Validating nested JSON payloads in user registration

The request has nested structures (user + address). We want declarative validation that cascades cleanly before persistence.

Answer:

- I'd model the request with DTOs and annotate fields using Bean Validation.
- Use @Valid on nested fields to trigger cascaded validation.
- In Spring/Jakarta, controller methods annotated with @Valid automatically return 400 with structured violations when constraints fail.

Code:

```
// DTOs (Jakarta Validation API)
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;

public record RegistrationRequest(
    @Valid @NotNull UserInfo user,
    @Valid @NotNull Address address) {}

public record UserInfo(
    @NotBlank @Size(max=100) String fullName,
    @Email @NotBlank          String email,
    @NotBlank @Pattern(regexp="^[A-Za-z0-9_]{6,32}$") String username,
    @NotBlank @Size(min=8, max=64) String password) {}

public record Address(
    @NotBlank String line1,
    String line2,
    @NotBlank String city,
    @NotBlank String state,
    @NotBlank @Pattern(regexp="^[0-9]{5}(-[0-9]{4})?$") String postalCode,
    @NotBlank String country) {}
// Spring REST example
@RestController
class RegistrationController {
  @PostMapping("/register")
  ResponseEntity<?> register(@Valid @RequestBody RegistrationRequest req) {
    // If invalid, Spring returns 400 with field errors automatically
    // Persist user...
    return ResponseEntity.ok().build();
  }
}
```

Reasoning to interviewer:

- @Valid ensures deep validation of nested objects, keeping checks near the data and out of controller logic.
- Records make the payload immutable and constructor-bound, which aligns well with validation.
- Standard annotations cover most rules; custom constraints can handle domain rules (e.g., password strength).

Future scope:

- Add class-level constraints for cross-field checks (e.g., username != email).
- Localize messages via ValidationMessages.properties.
- Generate JSON Schema/OpenAPI from the annotated DTOs to keep clients in sync.

Scenario 81: Calling native libraries with the Foreign Function & Memory (FFM) API

You need to call libavcodec from Java without JNI, and you want safe, explicit control of off-heap memory.

Answer

I'll use the FFM API (Java 22+/23): Linker to create downcalls, SymbolLookup to find C symbols in libavcodec, and Arena/MemorySegment to manage native memory lifetimes. The idea is: open a confined arena per operation (or per batch), look up the function, build a MethodHandle with a matching FunctionDescriptor, prepare any arguments in native memory, call the handle, then let the arena's scope free everything deterministically when it closes.

Code

```
// Java 22+ FFM (no incubator): java.lang.foreign.*
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import static java.lang.foreign.ValueLayout.*;

public final class AvCodecFfm {
  private static final Linker LINKER = Linker.nativeLinker();

  // Load symbols from libavcodec (platform's usual name resolution applies)
  private static SymbolLookup avcodecLookup(Arena arena) {
    return SymbolLookup.libraryLookup("avcodec", arena); // e.g.,
"libavcodec.so"/"avcodec.dll"
  }

  // Example 1: call unsigned int avcodec_version(void)
  public static int avcodecVersion() throws Throwable {
    try (Arena arena = Arena.ofConfined()) {
      var lookup = avcodecLookup(arena);
      var addr   = lookup.find("avcodec_version").orElseThrow();
      var fd     = FunctionDescriptor.of(JAVA_INT);       // returns unsigned int →
map to int
      MethodHandle mh = LINKER.downcallHandle(addr, fd);
      return (int) mh.invokeExact();
    }
  }

  // Example 2: call const AVCodec* avcodec_find_encoder_by_name(const char*)
→ address
  public static MemorySegment avcodecFindEncoderByName(String encName)
throws Throwable {
```

```
    try (Arena arena = Arena.ofConfined()) {
      var lookup = avcodecLookup(arena);
      var addr   = lookup.find("avcodec_find_encoder_by_name").orElseThrow();
      var fd     = FunctionDescriptor.of(ADDRESS, ADDRESS); // (char*) → (AVCodec*)
      MethodHandle mh = LINKER.downcallHandle(addr, fd);

      // allocate a UTF-8 C string off-heap inside this arena
      MemorySegment cStr = arena.allocateFrom(encName);     // lifetime bound to 'arena'
      MemorySegment codecPtr = (MemorySegment) mh.invokeExact(cStr);
      return codecPtr; // DO NOT dereference after arena closes; copy out if needed
    }
  }

  // Example 3: tiny struct layout example (simplified "packet": { uint8_t* data; int size; })
  static final GroupLayout PACKET = MemoryLayout.structLayout(
      ADDRESS.withName("data"),
      JAVA_INT.withName("size")
  ).withName("Packet");

  public static void fillPacket(MemorySegment packet, MemorySegment data, int size) {
    packet.set(ADDRESS, PACKET.byteOffset(MemoryLayout.PathElement.groupElement("data")), data);
    packet.set(JAVA_INT, PACKET.byteOffset(MemoryLayout.PathElement.groupElement("size")), size);
  }
}
```

Reasoning to interviewer

- FFM removes JNI boilerplate: the ABI mapping is declared in FunctionDescriptor, and MethodHandle invokes directly.
- Arena scopes ownership. When the try block ends, all allocations inside it are freed—no leaks, no finalizers.
- Using typed MemoryLayout means I'm not doing pointer math by hand; offsets come from the layout.
- For pointers returned by C (e.g., AVCodec*), I treat them as MemorySegment addresses. If libavcodec transfers ownership to me, I'll call the matching free function; otherwise I keep only the pointer for the duration allowed by the native API.

Future scope

- Add upcalls for callbacks (e.g., logging) via LINKER.upcallStub and a matching FunctionDescriptor.
- Separate symbol resolution from hot calls: resolve once per process, then reuse the MethodHandles.
- Wrap common structs in tiny helper classes that hide layout/set/get to keep call sites clean.
- If I need long-lived native buffers (e.g., encoder state), use a shared arena with explicit lifecycle and document ownership clearly.

Scenario 82: Off-heap struct management with FFM arenas

You're parsing large market-data frames off-heap to reduce GC pressure. Allocation, alignment, and timely free are critical under load.

Answer

I'll use a small pool of rotating arenas (or one arena per batch/partition). Each feed slice allocates into its arena; when that slice is fully processed, I close the arena and the whole chunk is freed at once. For alignment (e.g., 64-byte for SIMD/IO), I'll either design the MemoryLayout with the right alignment or over-allocate and slice. The point is: avoid tiny, random frees—allocate in big lifetimes and free in bulk.

Code

```
import java.lang.foreign.*;
import static java.lang.foreign.ValueLayout.*;
import java.util.concurrent.atomic.AtomicInteger;

public final class OffheapFrames {
  // Example per-frame layout: header(8 bytes) + len(4 bytes) + padding + payload[]
  static final GroupLayout HEADER = MemoryLayout.structLayout(
      JAVA_LONG.withName("tsNanos"),
      JAVA_INT.withName("len")
  ).withName("Header").withBitAlignment(64); // 64-bit align for nice access

  // Ring of arenas so one thread can ingest while others still read older frames
  static final class ArenaRing {
    private final Arena[] slots;
    private final AtomicInteger cursor = new AtomicInteger();

    ArenaRing(int n) {
      this.slots = new Arena[n];
      for (int i = 0; i < n; i++) slots[i] = Arena.ofConfined();
    }

    // rotate: close next slot (free all its memory), then reuse it
    Arena next() {
      int i = Math.floorMod(cursor.getAndIncrement(), slots.length);
      slots[i].close();                // bulk free prior allocations in this slot
      slots[i] = Arena.ofConfined();   // fresh arena for the new batch
      return slots[i];
    }
  }

  // Pack a frame into off-heap memory inside the chosen arena
```

```
  public static MemorySegment allocateFrame(Arena arena, long tsNanos, byte[]
payload) {
    int len = payload.length;
    long total = HEADER.byteSize() + len;
    // Align the full frame to 64 bytes (over-allocate and slice)
    MemorySegment raw = arena.allocate(total + 63); // cheap over-allocation
    long aligned = (raw.address() + 63) & ~63L;
    MemorySegment frame = raw.asSlice(aligned - raw.address(), total);

    frame.set(JAVA_LONG,
HEADER.byteOffset(MemoryLayout.PathElement.groupElement("tsNanos")),
tsNanos);
    frame.set(JAVA_INT,
HEADER.byteOffset(MemoryLayout.PathElement.groupElement("len")),     len);

    MemorySegment payloadSeg = frame.asSlice(HEADER.byteSize(), len);
    payloadSeg.copyFrom(MemorySegment.ofArray(payload));
    return frame; // valid until arena is closed
  }
}
```

Reasoning to interviewer

- Arenas give me region allocation: many small pieces allocated, one bulk free—great for high-throughput feeds.
- The ring of arenas avoids stalls: readers can still look at the N-1 batch while I overwrite N.
- Aligning frames at 64 bytes reduces false sharing and can help downstream SIMD or DMA.
- All lifetimes are explicit: no hidden finalizers or phantom references; when a batch is done, its arena closes.

Future scope

- Use SegmentAllocator wrappers per arena for higher-level “allocate header + payload” helpers.
- Partition by symbol/source (arena per hot key) to reduce cross-thread ownership.
- Add bounds/guards: keep the length in the header and assert before parsing to avoid overruns.

- When integrating with native codecs/parsers, pass MemorySegment directly to avoid copies.

Scenario 83: Primitive classes for telemetry data (JDK 23)

You model billions of Point3f values (x,y,z floats) that don't need identity or synchronization. Regular objects waste memory (12–16B header + refs) and fragment the heap.

Answer

Primitive classes (Project Valhalla, JDK 23 preview) let me declare identity-less, immutable types whose values can be flattened in arrays/fields—no per-object header, no pointer indirection. An array of Point3f becomes a tight, contiguous blob of 3 floats per element. That slashes footprint and improves cache locality for math loops.

Code (illustrative preview syntax)

```
// Preview: syntax may evolve; idea is an identity-less, immutable class
primitive class Point3f {
  public final float x;
  public final float y;
  public final float z;

  public Point3f(float x, float y, float z) {
    this.x = x; this.y = y; this.z = z;
  }

  public float dot(Point3f o) { return x*o.x + y*o.y + z*o.z; }
  public Point3f add(Point3f o) { return new Point3f(x+o.x, y+o.y, z+o.z); }
}

// Usage: flattened storage (no per-element object headers)
Point3f[] cloud = new Point3f[n];   // laid out as x,y,z | x,y,z | ...
float s = 0f;
for (int i = 0; i < n; i++) s += cloud[i].z; // predictable, cache-friendly
```

What this buys you

- Flattened arrays/fields: fewer cache misses and much smaller memory footprint.
- No identity: comparisons default to value equality semantics; no monitor/identity hash.
- Great for telemetry/math kernels where values flow through pipelines and don't need mutation.

Limitations/caveats

- No object identity: you can’t synchronized on a primitive object; identity hash is unsupported.
- Nullability: “null-free” storage forms don’t have a null default—use a “nullable” projection or sentinel as needed.
- Some frameworks that rely on proxies or reflective construction may need adaptation.
- Generics erase to Object; specialized generics aren’t here yet, so collections like List<Point3f> won’t flatten—use arrays or specialized containers for hot paths.
- Still preview: APIs and syntax can change; enable the preview flags and test across JDK bumps.

Reasoning to interviewer

- For dense telemetry, you mostly scan/aggregate—primitive classes make each access a straight memory read with no pointer chasing.
- Eliminating object headers can cut memory by ~2–3× compared to boxed/POJO forms (workload-dependent).
- The design forces immutability, which matches telemetry’s “append-only, read-many” nature.

Future scope

- Combine primitive classes with the Vector API for tight inner loops over flattened arrays.
- Consider “struct-of-arrays” vs “array-of-structs” layouts depending on access patterns; primitive classes help with AoS, while multiple primitive arrays can handle SoA.
- Add small domain types (e.g., primitive class MeterId) to remove accidental mixing of units without overhead.
- Track the preview and migrate gradually; keep a fallback POJO path for older JDKs if needed.

Scenario 84: Comparing Records and primitive classes for performance

You're sampling telemetry at very high rates. Today each sample is a tiny immutable Record. Profiling shows a lot of small-object churn and pointer chasing in large arrays of samples.

Answer

- Records are regular identity objects with final fields. They still have an object header and live on the heap; arrays of records are arrays of references. The JIT can stack-allocate or scalar-replace a record only when it doesn't escape the method; once you put records into arrays/collections, you pay for allocation and indirection.
- Primitive classes (JDK 23 preview, Project Valhalla) are identity-free, immutable value types. Arrays and fields can be flattened: Sample[] stores the bits of each Sample back-to-back (no object headers, no references). That cuts memory, improves cache locality, and reduces GC pressure.
- When I'd switch: the hot path manipulates big arrays of small immutable values; identity isn't needed; you control the storage (arrays/fields rather than generic collections); and you can run with preview flags. For mixed code or heavy framework use (proxies/reflection/collections), I'd stick to records and rely on escape analysis where possible.

Code

```
// Today: record (simple, readable, but heap objects & refs in arrays)
public record Sample(long tsNanos, float value) {}

// Tomorrow: primitive class (preview syntax; identity-free & flattenable)
primitive class SampleP {
  public final long tsNanos;
  public final float value;
  public SampleP(long tsNanos, float value) {
    this.tsNanos = tsNanos; this.value = value;
  }
}

// Hot loop (array use shows the difference)
static float avg(Sample[] a) {
  float sum = 0f;
  for (int i = 0; i < a.length; i++) sum += a[i].value();    // pointer chasing
```

```
  return sum / a.length;
}
static float avgP(SampleP[] a) {
  float sum = 0f;
  for (int i = 0; i < a.length; i++) sum += a[i].value;      // flattened, cache-friendly
  return sum / a.length;
}
```

Reasoning to interviewer

- Records shine for clarity and immutability, and the JIT may scalar-replace them in small scopes. But they remain objects once stored.
- Primitive classes remove per-object headers and indirection in arrays/fields, which is exactly where analytics pipelines spend time. That typically lowers memory by ~2–3× for small structs and lifts throughput by improving locality.
- Limitations: no identity (can't synchronize on them), null-free storage semantics, preview-feature caveats, and collections/generics still box—arrays/fields reap the big wins.

Future scope

- Pair primitive classes with the Vector API to chew through flattened arrays faster.
- If you must use collections, create specialized columnar containers (SoA) for hot data, keep Records at the edges.
- Keep a fallback build with Records for older JDKs; gate the primitive class path behind feature flags and JMH baselines.

Scenario 85: Module import declarations in large codebases (JDK 23)

Your module-info.java files are full of requires ... lines, and source files drown in many package imports.

Answer

- Module import declarations don't replace requires in module-info.java; they simplify source-file imports. In JDK 23 (preview), you can write import module <name>; at the top of a class to import, on demand, all public types from the packages that the named module exports, including types from modules it makes readable transitively. This cuts dozens of import x.y.*; lines to one and improves readability in exploratory and application code. OpenJDK+1
- You still manage the module graph in module-info.java with requires, exports, and requires transitive. Use requires transitive in API modules to reduce repeated requires downstream.
- Practical effects: cleaner source files, faster prototyping/single-file programs, and simpler onboarding. Trade-offs: potential ambiguous simple names (compiler will flag) and it's a preview—enable with --enable-preview for compile/run. Oracle Docs+1

Code

```
// Before: many package imports
import java.sql.Connection;
import java.sql.DriverManager;
import javax.sql.DataSource;
import java.util.logging.Logger;
import java.util.logging.Level;
// ...

// After (JDK 23 preview):
import module java.sql;       // brings java.sql & javax.sql public types
import module java.logging;   // brings logging API

class Repo { /* use Connection, DataSource, Logger directly */ }
```

Reasoning to interviewer

- This feature lifts the cognitive load of tracking package names across many modules while keeping the module graph explicit in module-info.java.
- Tooling impact is positive: IDEs and javac understand the declaration; ambiguity is surfaced at compile time. It's additive—teams can adopt it gradually.

Future scope

- Combine with jdeps in CI to keep module-info.java lean and correct while enjoying tidier source files.
- Establish a convention: allow import module in app/test modules, keep library modules conservative until the feature exits preview.
- Use aggregator “facade modules” with requires transitive to shrink requires blocks in module-info.java where appropriate.

Scenario 86: JVM performance tuning with escape analysis

Your trading engine allocates lots of tiny helper objects in hot loops. GC is tuned, but allocation itself bottlenecks.

Answer

- Let HotSpot prove objects don't escape and eliminate them:
  - Escape analysis (EA) + scalar replacement can turn a short-lived object into a few registers/stack slots.
  - Lock elision removes unnecessary synchronized if the receiver doesn't escape.
- How to help the JIT: keep objects local; don't store them into fields/arrays/collections; keep methods small and inlinable; avoid passing them to unknown (virtual/non-inlined) callees; prefer immutable "builderless" patterns on hot paths.

Code

```
// Bad: object escapes via a field and a collection
final class Quote { final double bid, ask; Quote(double b, double a){ bid=b; ask=a; }
}
final class Engine {
  private final List<Quote> seen = new ArrayList<>();
  private Quote last;                          // field → escapes
  double spreadSum(List<double[]> ticks) {
    double s = 0;
    for (var t : ticks) {
      Quote q = new Quote(t[0], t[1]);         // alloc every tick
      last = q; seen.add(q);                   // definitely escapes
      s += (q.ask - q.bid);
    }
    return s;
  }
}

// Good: non-escaping locals → scalar replacement
final class EngineEA {
  double spreadSum(double[] bids, double[] asks) {
    double s = 0;
    for (int i = 0; i < bids.length; i++) {
      // no Quote object at all; JIT keeps scalars in regs/stack
      double bid = bids[i], ask = asks[i];
      s += (ask - bid);
```

```
    }
    return s;
  }
}

// If you must pass a tiny aggregate, keep it local and inlinable
record Pair(double x, double y) {}
static double sumSpreads(double[] bids, double[] asks) {
  double s = 0;
  for (int i = 0; i < bids.length; i++) {
    s += spread(new Pair(bids[i], asks[i]));    // can be scalar-replaced if inlined
  }
  return s;
}
static double spread(Pair p) { return p.y() - p.x(); } // keep small to inline
```

Reasoning to interviewer

- EA works per compilation unit: once an allocation's reference is stored globally or crosses an unanalyzable call boundary, it's considered escaping and the JIT must allocate.
- Restructuring code so aggregates stay local (or are immediately decomposed) lets C2 eliminate the allocation entirely and often the locks as well.
- The payoff is lower allocation rate, fewer barriers, and better CPU cache behavior—GC tuning can only do so much if the allocator stays hot.

Future scope

- Verify with diagnostics:
    - -XX:+UnlockDiagnosticVMOptions -XX:+PrintEscapeAnalysis -XX:+PrintEliminateAllocations -XX:+PrintEliminateLocks to see what C2 removes.
    - Use JFR ("Object Allocation in new TLAB/outside TLAB") and JITWatch to correlate allocation sites with compiler decisions.
- If objects must cross boundaries, switch shapes: columnar primitives (arrays of doubles) or primitive classes (when available) for flattening.
- Hoist/inline hot helpers (e.g., fee/rounding) to keep arguments known and analyzable; avoid megamorphic call sites in the hot loop.

Scenario 87: Diagnosing latency spikes caused by safepoints

P99 latency jumps but GC logs are clean. That usually means “non-GC” stop-the-world activity: safepoints triggered by VM operations like deoptimization, code-cache sweeps, biased-lock revocation (older JDKs), class redefinition, or heavy JVMTI tooling.

Answer

I’d confirm (a) that we’re hitting safepoints, (b) what VM operation is causing them, and (c) whether they correlate with compilation/deopt/code-cache events.

Code

```
# Start with unified logging targeted at safepoints and friends
java \
 -Xlog:safepoint=info           \  # reason + time at each STW
 -Xlog:deopt=info               \  # deoptimization (uncommon traps)
 -Xlog:codecache=info           \  # code cache sweeps/full/compilation disable
 -Xlog:compiler=info            \  # C1/C2/Tiered compilation activity
 -Xlog:class+load=info          \  # class loading surges can trigger STW
 -XX:+UnlockDiagnosticVMOptions \
 -XX:+PrintCompilation          \  # concise compile timeline
 -jar app.jar

# Deep dive (use sparingly in prod):
# -Xlog:safepoint*=debug,os+thread=trace  # very chatty
# JFR is the safest way to see end-to-end correlations:
java \
 -XX:StartFlightRecording=filename=latency.jfr,settings=profile,dumponexit=true \
 -Xlog:safepoint=info -Xlog:deopt=info -Xlog:codecache=info \
 -jar app.jar
# Inspect JFR events: Safepoint Begin/End, Deoptimization, Code cache full, Compilation, Class loading
```

Mitigations by cause:

- Biased lock revocation (JDK ≤17): disable it or avoid long-lived biased objects.
    - -XX:-UseBiasedLocking (removed entirely in newer JDKs).
- Deoptimization bursts (uncommon traps, megamorphic sites): stabilize call sites, reduce speculation.
    - Keep hot methods small/inlinable; reduce reflection/dynamic proxies at the hot edge.

    - Pin noisy methods: -XX:CompileCommand=exclude,com.foo.Hot::method or force compile.
- Code cache pressure (sweeps / “CodeCache is full. Compiler has been disabled.”):
    - Increase: -XX:ReservedCodeCacheSize=256m (or larger), keep -XX:+SegmentedCodeCache (default).
    - Trim dynamic class generation; cap frameworks that emit bytecode.
- Tooling/agent safepoints (JVMTI, frequent thread dumps): restrict or move to staging.
- Class redefinition/retransformation: avoid hot-redeploy on prod instances serving latency-critical traffic.

Reasoning to interviewer

- Not all pauses are GC. The JVM must occasionally stop all threads for VM ops; those are your P99 spikes.
- Unified logging + JFR gives a precise “why” and “when” without guesswork.
- Fixes are targeted: tame speculation (deopts), give the compiler room (code cache), and remove periodic VM ops from the hot window.

Future scope

- Add a canary instance with aggressive -Xlog:safepoint*=debug for short periods to fingerprint spikes.
- Bake a JFR profile into CI perf tests and alert on new safepoint patterns.
- Move background class loading/initialization to startup or a warm-up phase.
- For ultra-tight SLAs, isolate noisy subsystems in separate processes.

Scenario 88: Advanced module usage with services and reflection

Plugins must be discovered at runtime (ServiceLoader), while JSON mappers need reflective access to certain model packages—without exposing internals.

Answer

I split API vs implementation modules, use uses/provides for ServiceLoader discovery, and grant reflective access with narrow opens … to <mapper> (not exports). Implementations remain encapsulated—only the API is exported.

Code

```
// plugin API (stable contracts)
module com.cms.plugin.api {
  exports com.cms.plugin.api;                 // Plugin, Content, Result DTOs
  uses com.cms.plugin.api.Plugin;               // consumers can discover
implementations
}

// one plugin implementation
module com.cms.plugin.markdown {
  requires com.cms.plugin.api;

  provides com.cms.plugin.api.Plugin
      with com.cms.plugin.markdown.MarkdownPlugin;

  // allow Jackson to reflectively read/write only the model package
  opens com.cms.plugin.markdown.model to com.fasterxml.jackson.databind;
  // NOTE: we do NOT export impl packages; they stay hidden.
}

// application (composition layer)
module com.cms.app {
  requires com.cms.plugin.api;
  requires com.fasterxml.jackson.databind;     // or your chosen mapper
  uses com.cms.plugin.api.Plugin;
}
// Discovery at runtime
ServiceLoader<com.cms.plugin.api.Plugin> loader =
    ServiceLoader.load(com.cms.plugin.api.Plugin.class);
for (var p : loader) {
  // invoke plugin; mapper can reflect on allowed model packages
}
```

Reasoning to interviewer

- uses/provides lets implementations register themselves; the app doesn't depend on any impl module.
- opens is the surgical tool for reflection—unlike exports, it doesn't make packages compile-time visible.
- This keeps internals hidden, frameworks happy, and the module graph clean and acyclic.

Future scope

- Layer plugins dynamically via ModuleLayer to hot-load/unload and isolate dependencies.
- In CI, run jdeps to block accidental exports and reflection that punches global holes.
- Externalize plugin metadata (version/caps) and check compatibility before wiring.

Scenario 89: ClassLoader conflicts and split-package resolution

Two vendor JARs ship classes in the same package. That works on the classpath but fails under JPMS (split packages are illegal within a layer).

Answer

I remove the split at the packaging boundary or isolate the vendors so the JVM never sees the same package in two modules.

Code

1. Relocate one vendor's package (preferred for libraries you control transitively):

- Maven Shade / Gradle Shadow to rename com.vendor.common → com.vendorA.relocated.common.
- Then each module owns a unique package; declare clean requires/exports.

2. Merge into a single module when they're logically one library:

```
# Build a combined module via patching; both JARs become one module
java --patch-module vendor.common=vendorA.jar:vendorB.jar \
    --module-path mods -m com.app/main.App
```

- Now the package isn't split—both JARs are part of the same named module.

3. Isolate with separate module layers (advanced; keeps both originals intact):

```
ModuleLayer parent = ModuleLayer.boot();
Configuration base = parent.configuration();
Configuration cfgA =
base.resolve(ModuleFinder.of(Path.of("vendorA.jar")),
ModuleFinder.of(), Set.of("vendor.a"));
Configuration cfgB =
base.resolve(ModuleFinder.of(Path.of("vendorB.jar")),
ModuleFinder.of(), Set.of("vendor.b"));

ClassLoader clA = ClassLoader.getSystemClassLoader();
ClassLoader clB = ClassLoader.getSystemClassLoader();
```

```
ModuleLayer layerA = parent.defineModulesWithOneLoader(cfgA,
clA);
ModuleLayer layerB = parent.defineModulesWithOneLoader(cfgB,
clB);

// Use ServiceLoader in each layer to obtain implementations without
mixing packages
ServiceLoader<spi.Service> a = ServiceLoader.load(layerA,
spi.Service.class);
ServiceLoader<spi.Service> b = ServiceLoader.load(layerB,
spi.Service.class);
```

- Split-package checks apply per layer; by isolating them, each layer sees a consistent, sealed package.

Reasoning to interviewer

- Named modules implicitly seal packages; split packages break that guarantee (package-private access rules, security model).
- Relocation or merging removes the split entirely—simplest and safest.
- Layer isolation is a last resort when you can't touch vendor bits; you keep both worlds separate and communicate via services or DTOs.

Future scope

- Pin a "vendor facade" module that provides a stable API and hides the underlying vendor choice.
- Add a build-time check (jdeps or a custom plugin) to fail the build on new split packages.
- If you must stay on the classpath, use JAR sealing (Sealed: true in MANIFEST) to catch accidental splits early—then plan a proper modular fix.

Scenario 90: Pattern matching for switch with guarded patterns

We're routing a sealed message hierarchy and need guards (e.g., only some commands), explicit null handling, compile-time exhaustiveness, and no unreachable cases.

Answer

I'll use a pattern-matching switch (Java 21+; preview earlier) over a sealed base type. Guards go on the more specific cases and must appear before broader patterns so they aren't shadowed. I'll add an explicit case null to avoid accidental NPEs. With a sealed hierarchy, the compiler can check exhaustiveness—so I can omit default once all permitted subtypes (plus null) are covered.

Code

```
// Sealed hierarchy gives the compiler a closed world for exhaustiveness
sealed interface Msg permits Command, Event, Error {}

record Command(String name, String payload) implements Msg {}
record Event(String topic, int severity)    implements Msg {}
record Error(int code, String details)      implements Msg {}

enum Action { EXECUTE, IGNORE, ALERT, RETRY, BAD_REQUEST }

// Dispatcher using guarded patterns and explicit null handling
static Action dispatch(Msg m) {
  return switch (m) {
    // 1) Always handle null deliberately (otherwise NPE)
    case null -> BAD_REQUEST;

    // 2) Guarded patterns first (specific subset of a type)
    case Command c when "deploy".equalsIgnoreCase(c.name()) -> EXECUTE;
    case Command c when "noop".equalsIgnoreCase(c.name())   -> IGNORE;

    // 3) Unguarded type pattern AFTER its guarded variants
    case Command c -> BAD_REQUEST; // unknown command

    // 4) Other subtypes
    case Event e when e.severity() >= 5 -> ALERT;
    case Event e                        -> IGNORE;

    case Error err when err.code() == 409 -> RETRY;
    case Error err                        -> ALERT; // generic error policy
  };
}
```

Reasoning to interviewer

- Guard placement and dominance
  - A broader pattern dominates a narrower one if it appears first. So an unguarded case Command c placed before a guarded case Command c when ... would make the guarded case unreachable. Putting guarded cases first avoids this.
  - Type-based dominance also applies: case CharSequence cs must not precede case String s if both are present; the broader CharSequence would shadow String.
- Exhaustiveness
  - With sealed interface Msg permits ..., the compiler checks that all permitted subtypes are covered. Because null isn't part of the hierarchy, I add case null to keep the switch total without a default.
- Null handling
  - Pattern switches throw NPE on null unless you provide case null. Being explicit documents intent and prevents latent crashes.
- Readability and safety
  - Each arm both tests and binds (Command c)—no separate casts. Guards are plain boolean expressions evaluated after the type test, keeping routing logic concise and clear.
- No fall-through
  - Pattern-matching switches are non-fall-through by design; each arm yields/returns, which the compiler enforces.

Future scope

- If the set of message types grows, keep it sealed; the compiler will force you to update the switch.
- For multi-field guards, extract predicate methods (e.g., isHighPriority()) to keep cases readable.
- If you must switch on a non-sealed base (e.g., Object), keep a conservative default and log/metrics there to detect unexpected shapes.

- Combine with records and deconstruction patterns (when available) to match on inner fields directly (e.g., case Command(String name, var p) when name.equals("deploy") -> ...).